

**Université de Franche-Comté
Faculté des Lettres**

La Conjugaison du verbe français

MEMOIRE

**présenté pour l'obtention
de la Maîtrise de Linguistique
option Industries de la Langue**

par

William Kinnaird

**Sous la direction de
Mme Sylviane Cardey**

OCTOBRE 1991

TABLE DES MATIERES

PREMIERE PARTIE - LE CONTEXTE

1	Introduction	6
1.1	Guides de la conjugaison existants actuellement	7
1.1.1	Manuels	7
1.1.2	Logiciels	8
1.2	Description du programme CONJUGUE	10
1.2.1	Le module CONJUGAISON COMPLETE	10
1.2.2	Le module GENERATION	11
1.2.3	Le module INTERRO A	12
1.2.4	Le module INTERRO B	13
1.3	Les objectifs de ce travail	14
1.3.1	Contenu et limites	14
1.4	Les données	16
1.5	Le langage prolog	20
1.5.1	Quelques difficultés	20
1.5.2	Quelques avantages	21
1.6	Les systèmes experts	23
1.6.1	Les systèmes experts en général	23
1.6.2	Notre système pour la conjugaison verbale	24
1.6.3	Diagramme des divers modules	25

DEUXIEME PARTIE - ANALYSE LINGUISTIQUE

2.1	Introduction	27
2.2	Deux prédécesseurs	29
2.2.1	M.H. Gertner	29
2.2.2	L. Isaac	31
2.2.3	Conclusions	33
2.3	La problématique	34
2.3.1	Un schéma simple	35
2.3.2	Un schéma plus puissant	36
2.3.3	Un schéma fiable	38
3	Vers une formalisation	41
3.1	Deux aspects non-phonétiques	41
3.1.1	Les temps	41
3.1.2	Les personnes	45
3.2	Les terminaisons	47
3.2.1	Formalisation	49
3.3	Les radicaux	50
3.3.1	Première conjugaison	52
3.3.1.1	Particularités des radicaux	52
3.3.1.2	Voyelle thématique	52
3.3.1.3	Formalisation	52
3.3.2	Deuxième conjugaison	53
3.3.2.1	Particularités des radicaux	53

3.3.2.2	Voyelle thématique	53
3.3.2.3	Formalisation	53
3.3.3	Troisième conjugaison	55
3.3.3.1	Particularités des radicaux	58
3.3.3.2	Eléments minimaux	58
3.3.3.3	Les verbes en -ir des groupes 2 et 3	61
3.3.3.4	Regroupement dans la 3 ^e conjugaison	63
3.3.3.4.1	Les cas "faciles"	65
3.3.3.4.2	Les cas plus difficiles	68
3.3.3.5	Morphophonologie	68
3.3.3.5.1	L'indicatif présent	71
3.3.3.5.3	Residu	72
3.3.3.6	Les voyelles thématiques	75
3.3.3.6.1	Formalisation	78
4	Comment savoir où fixer les limites?	81
4.1	Un problème	82
4.1.1	Verbes defectifs, tableau	83
4.1.2	Verbes essentiellement impersonnels	

TROISIEME PARTIE - IMPLEMENTATION

5.1	Introduction	85
5.2	La création d'une boîte à outils	85
5.2.1	Traitement des éléments minimaux	87
5.2.2	Exclusion des éléments	89
5.2.3	Remplacement d'un élément par un autre	91
5.2.4	Isolation des éléments	92
5.3	Quelques remarques à propos de la taille du programme	94
5.3.1	L'importance du CODE	96
5.3.2	Deux représentations de la même opération	101
6	Description du programme	102
6.1	Vue d'ensemble du traitement	103
6.2	Vue d'ensemble selon les predicats	104
6.3	La saisie du groupe verbal	105
6.4	Transcription des regles génératives	105
6.4.1	Morphophonologie du premier groupe	108
6.4.2	Morphophonologie du troisième groupe	112
6.5	Le traitement des temps - un exemple	114
6.6	Séparation des verbes en -ir	117
6.7	Des programmes pour tester le programme	

LISTINGS DES MODULES DU PROGRAMME

7.1	Module initial	118
7.2	Menus	120
7.3	Le Noyau	125
7.4	Interro et Génération	140
7.5	Conjugaison complète	144
7.6	Concordance	148

ANNEXE

A.	Particularités du programme	151
B.	Quelques traits additionnels non-inclus	152
C.	Définitions	153
C1.	Numérotation	153
C1.1	Temps	153
C1.2	Personne	153
C2	Symboles	154
D.	Les règles principales	155
E.	Prédicats - dans l'ordre alphabétique	157
F	Résultats des programmes de test	159
F1	Séparation des 2 ^e et 3 ^e conjugaisons	159
F2	Conjugaison de la liste <i>VERBS.DBA</i>	160
BIBLIOGRAPHIE		162
INDEX		164

PREMIERE PARTIE

Le contexte

Quand on apprend une langue il y a beaucoup de choses à faire en même temps. Il faut étudier simultanément la grammaire, la prononciation, le lexique etc. Tous les outils qui peuvent alléger le fardeau pesant sur les épaules du débutant sont certes bienvenus, et l'ordinateur en est un et pas des moins efficaces. Néanmoins, il est évident que même l'ordinateur, malgré sa rapidité et ses performances dans presque tous les domaines de l'intellect, ne peut pas tout résoudre. Il offre quand même des possibilités beaucoup plus modulées et beaucoup plus étendues que n'importe quel manuel. L'ordinateur peut jouer à la fois les rôles de professeur et de document. C'est dans cet esprit là que nous nous sommes engagés dans l'élaboration d'un logiciel de conjugaison.

Nous nous sommes ainsi proposé de construire un outil qui ne serait pas limité à certains paradigmes mais qui pourrait fournir directement la conjugaison de n'importe quel verbe de la langue française à n'importe quel temps. Nous avons voulu créer quelque chose d'exhaustif, mais cela dans un domaine bien précis et délimité. Nous détaillerons plus loin ces objectifs.

Le programme est, à proprement parlé, un *didacticiel*, à vocation, comme son nom l'indique, didactique. Il va non seulement réaliser la conjugaison que l'utilisateur lui a demandée mais aussi montrer à l'utilisateur comment il l'a réalisée. Dans un deuxième temps, si l'utilisateur le souhaite, le programme l'interrogera aussi, de plusieurs façons différentes, sur ce qu'il vient

d'apprendre. Un module additionnel TUTOR, qui n'existe pour l'instant qu'en prototype, va donner au débutant l'occasion de réaliser la conjugaison d'une façon "interactive", afin de l'initier, pas à pas, à l'art de conjuguer.

1.1 GUIDES DE LA CONJUGAISON EXISTANTS ACTUELLEMENT

1.1.1. Manuels

Il existe des ouvrages destinés à aider l'apprenant à «générer» des conjugaisons verbales. Habituellement on s'attend à trouver dans les grammaires des tableaux de paradigmes permettant cette «génération», c'est-à-dire conjuguer les verbes à partir de la forme trouvée dans un dictionnaire - à savoir la forme infinitive dans le cas du français. En ce qui concerne la langue française il existe de nombreux ouvrages qui sont entièrement consacrés à la conjugaison, dont le plus connu est sans doute celui de BESCHERELLE. Au cours des dernières années sont paru plusieurs autres ouvrages du même type.

Pour les langues anciennes comme le grec et l'hébreu, à part les tableaux usuels, il existe des aides comme le SLIDAVERB de Zondervan. Il s'agit d'une carte qui ressemble à une règle à calcul (anglais *slide-rule*), avec des ouvertures et dans laquelle on "glisse" (*slide*) une autre carte sur laquelle sont rangés en colonnes des paradigmes verbaux.

Le programme élaboré dans le présent travail est uniquement un générateur de conjugaisons et il ne sera pas capable de reconnaître l'origine d'une forme fléchie.

1.1.2 Logiciels

Il existe aussi au moins trois¹ logiciels consacrés à la grammaire française, deux pour IBM PC et un pour Macintosh. Nous en ai vu fonctionner deux - le correcteur HUGO et le FRENCH ASSISTANT. Tous les deux sont des programmes qui traitent de plusieurs aspects de la langue française tel, par exemple, dans le cas de HUGO, les accords en genre et en nombre.

En ce qui concerne la conjugaison ils l'abordent d'une façon assez différente de ce que nous nous sommes proposés de réaliser. En réponse à une demande de l'utilisateur le FRENCH ASSISTANT affiche tout simplement des tableaux de formes conjuguées selon le mode et le temps choisis.² Il en est de même pour HUGO qui affiche une "page de Bescherelle", mais qui peut aussi suggérer l'infinitif du verbe à partir d'une forme conjuguée présente dans le texte. Si l'utilisateur surligne, par exemple, suis dans un texte, le programme lui affichera une liste d'analyses possibles, comme suit:

être - ind. prés, 1 sg
être - imptf, 2 sg
suivre - ind. prés 1 sg
suivre - ind. prés 2 sg
suivre - imptf 2 sg

¹ Un quatrième, un "Bescherelle Electronique", a aussi été annoncé, mais il semble qu'il ne soit pas paru.

² Une version plus récente que celle que nous avons vu serait, paraît-il, capable de conjuguer (à partir de l'infinitif?) pendant que l'utilisateur tape le texte.

Le programme SANS-FAUTE/GRAMMAIRE, crée pour le Macintosh, traite les problèmes de grammaire et ce à peu près de la même façon que HUGO à la différence qu'il est un peu plus précis³.

Les trois logiciels traitent entre autres le verbe français mais aucun d'eux n'est consacré exclusivement à la conjugaison ou à la didactique. Par exemple, HUGO et SANS FAUTE abordent les problèmes de morpho-syntaxe, et le FRENCH ASSISTANT comporte un module de traduction anglais/français très rudimentaire.

³ Voir Science Vie Micro MACINTOSH Août-Sept 1991 p 51-2.

1.2

DESCRIPTION DU PROGRAMME CONJUGUE

CONJUGUE est un programme entièrement consacré à la conjugaison. L'utilisateur peut l'interroger de plusieurs façons différentes. Imaginons qu'il veuille d'abord une vue d'ensemble de la conjugaison. Alors, il choisit le module CONJUGAISON COMPLETE, il entre son verbe à l'infinitif, *sentir* par exemple, et le programme lui donne l'écran suivant:

1.2.1

LE MODULE CONJUGAISON COMPLETE

INDICATIF présent	IMPERATIF	INDICATIF passé simple
je sens	sens	je sentis
tu sens	sentons	tu sentis
il sent	sentez	il sentit
nous sentons	PARTICIPE présent	nous sentîmes
vous sentez	sentant	vous sentîtes
ils sentent	PARTICIPE passé	ils sentirent
SUBJONCTIF présent	ayant senti, ie	SUBJONCTIF imparfait
je sente	INDICATIF futur	je sentisse
tu sentes	je sentirai	tu sentisses
il sente	tu sentiras	il sentît
nous sentions	il sentira	nous sentissions
vous sentiez	nous sentirons	vous sentissiez
ils sentent	vous sentirez	ils sentissent
INDICATIF imparfait	ils sentiront	
je sentais	CONDITIONNEL	
tu sentais	je sentirais	
il sentait	tu sentirais	
nous sentions	il sentirait	
vous sentiez	nous sentirions	
ils sentaient	vous sentiriez	
	ils sentiraient	
ESPACE: encore ESC: menu		

Par la suite, s'il veut voir en détail les démarches nécessaires pour arriver à une certaine forme à partir de l'infinitif, il choisit le mode GÉNÉRATION, puis un temps et il entre l'infinitif de son verbe, suivi d'un pronom, par exemple *sentir*

à la première personne de l'indicatif présent:*

1.2.2

LE MODULE GENERATION

ESC:menu ESPACE:continuer MODE: Génération - INDICATIF présent	
>> <i>sentir je</i> >>> <i>je sens</i>	SCORE
ANALYSE	INFO
<i>je sent - ir</i> <i>sent + s</i> <i>t --> Ø/_s</i> <i>sen + s</i> <i>je sens</i>	

Après avoir obtenu ces premières réponses, l'utilisateur peut, s'il le désire, choisir le module INTERRO par lequel l'ordinateur lui proposera une série de tests pour contrôler ses nouvelles acquisitions:

* Dans les écrans suivants tout ce qui est écrit en italique est entré par l'utilisateur.

1.2.3

LE MODULE INTERRO A

ESC:menu ESPACE:continuer MODE: Interro - SUBJONCTIF présent													
ENTRER INFINITIF >> sentir <table><tr><td>je sente</td><td>bravo!</td></tr><tr><td>tu sentes</td><td>bravo!</td></tr><tr><td>il sente</td><td>bravo!</td></tr><tr><td>nous sentons</td><td>incorrect --> sentions</td></tr><tr><td>vous sentiez</td><td>bravo!</td></tr><tr><td>ils</td><td></td></tr></table>	je sente	bravo!	tu sentes	bravo!	il sente	bravo!	nous sentons	incorrect --> sentions	vous sentiez	bravo!	ils		SCORE
je sente	bravo!												
tu sentes	bravo!												
il sente	bravo!												
nous sentons	incorrect --> sentions												
vous sentiez	bravo!												
ils													
ANALYSE vous sent-ir sent - iez vous sentiez	INFO												

Fort de ses premières acquisitions l'utilisateur pourra ensuite poursuivre les tests en laissant à l'ordinateur le choix du verbe, du temps, de la personne ou tous à la fois et ce sur plusieurs niveaux de difficultés:

1.2.4

LE MODULE INTERRO B

ESC:menu ESPACE:continuer		MODE: Interro
élever --- INDICATIF présent >> ils élèvent Bravo! La bonne réponse.		SCORE VRAI 10 FAUX 3 TOTAL 13 74%
ANALYSE ils élev - er élev + ent e --> é/__ent élèvent	INFO	

(NOTA La fenêtre INFO n'a pas encore été développée.)

1.3

LES OBJECTIFS DE CE TRAVAIL

1.3.1 Contenu et limites

Le verbe est un constituant syntagmatique très complexe - on pourrait dire le noyau de l'énoncé. Pour l'apprenant il serait donc très utile si le programme pourrait fournir des indications pertinentes sur l'emploi du verbe - le nombre d'arguments, s'il exige une préposition et si oui laquelle, s'il est utilisé d'une manière transitive ou intransitive etc. On aimerait aussi savoir dans quels contextes on peut utiliser un tel verbe et dans quels contextes son emploi n'est pas acceptable.

Il n'est pas difficile d'imaginer que les renseignements syntaxiques et sémantiques de ce genre exigeraient un travail beaucoup plus étendu que celui-ci. Le programme fonctionne d'une façon purement formelle, c'est-à-dire qu'il ne s'occupe pas des conditions d'emploi du verbe qu'il traite. Bien qu'il présente à l'utilisateur des menus qui listent les temps comme sous-ensembles des modes, il traite ces deux points de la même manière. Le passé simple et le subjonctif imparfait, par exemple, ayant le même radical et la même voyelle entre radical et terminaison, sont considérés tous les deux par le programme comme appartenant au même ensemble, bien que le TEMPS *passé simple* soit une sous-catégorie du MODE indicatif, et que le TEMPS *imparfait* du subjonctif appartienne bien sûr au MODE subjonctif. Le programme ne s'intéresse donc pas au fonctionnement du verbe mais

se concentre uniquement sur la forme de l'infinitif entrée par l'utilisateur et cherche, d'après le mode, le temps, la personne et le nombre choisis, à changer la forme selon les règles de l'algorithme de conjugaison.

Pour cette raison il est inutile d'attendre du programme qu'il sache, par exemple, que (normalement)⁵ les êtres humains n'aboient pas et que les chiens (on suppose) ne s'adressent pas à la première personne - bref, il ne faut pas être surpris de le voir conjuguer: *j'aboye, tu aboyes ... nous aboyons* etc⁶. Par contre en ce qui concerne les verbes dits défectifs et dont certaines formes sont inusitées, parce que disparues ou tombées en désuétude, le programme en avertira l'utilisateur⁷.

⁵ Bien que tout soit possible dans le monde des contes et dans certains contextes!

⁶ Les "correcteurs" HUGO et SANS FAUTE mentionnés ci-dessus sont capables de corriger quelques fautes faites sur le plan syntaxique mais la sémantique leur échappe totalement.

⁷ Quand il s'agit d'un verbe comme *pleuvoir* il se peut que l'utilisateur se demande comment le programme "sait" que la forme *je pleus* est un peu bizarre et refuse de l'accepter, tandis qu'il ignore le même phénomène avec *aboyer*. Cependant *pleuvoir* (avec *falloir*, par exemple) est un verbe à emploi impersonnel (et appartient donc à une liste finie), à la différence des verbes comme *aboyer* qui ont un sujet évident, bien que non-humain.

1.4.

LES DONNEES

Pour un travail de cette sorte nous avons eu besoin d'un corpus, c'est-à-dire une classe d'objets que l'on suppose représentative de l'ensemble des objets que l'on veut traiter. Dans le cas qui nous occupe, le corpus serait constitué par toutes les formes verbales rencontrées dans un ensemble de textes donné. Il est clair cependant que si on voulait prendre en considération chaque verbe de la langue française sous toutes ses formes on aurait besoin d'un ensemble de textes très important. Cela représenterait l'oeuvre d'une vie tel le *Bon Usage* de Grevisse, sans parler des difficultés que cela entraînerait pour un étranger n'ayant pas le français comme langue maternelle. En fin, du fait que ce travail a déjà été réalisé, non seulement dans ledit ouvrage de Grevisse mais dans d'autres du même genre, nous nous sommes déchargés de cette obligation et nous nous sommes permis de prendre comme système initial une description traditionnelle fiable, relevée dans les ouvrages signalés ci-dessous.

- LAROUSSE, *La conjugaison* ⁸
- BESCHERELLE, *La Conjugaison - 12000 verbes.*
- BENAC ET BURNEY, *Guide de Conjugaison.* ⁹
- GUERIN, *La Conjugaison visualisée.* ¹⁰
- CAPUT, *Dictionnaire des Verbes Français*

Et bien sûr aussi

- GREVISSE, *Le Bon Usage*, chapitre 5 "Le Verbe".

⁸ Nous avons utilisé les tableaux qui se trouvent dans le dictionnaire de Larousse, que nous avons sous la main.

⁹ Nous n'avons pas utilisé ce guide.

¹⁰ Nous n'avons pas trouvé cet ouvrage.

En comparant ces ouvrages on remarque tout de suite qu'ils ne sont pas tout à fait d'accord sur les éléments constitutifs d'un corpus fiable. Pour la plupart il y a peu de désaccord mais à propos des verbes dits défectifs et impersonnels (ou unipersonnels) il y a beaucoup de différence d'opinion, et on y reviendra plus loin dans la deuxième partie.

L'Editeur de Bescherelle (1990 - "nouvelle édition entièrement remise à jour") affirme dans son avertissement que l'ouvrage "fournit une liste quasi exhaustive des verbes français". Il nous informe aussi que "quelques rares verbes désuets ont été abandonnés", et qu'en revanche, "parmi les centaines de verbes introduits, figurent des verbes relevant des langues de métiers, de la langue argotique". On est tenté de croire que cette "liste quasi-exhaustive" "entièrement remise à jour, contenant quelques sept mille entrées représentant douze mille verbes" constituerait un corpus fiable et l'ouvrage a suffi pour d'autres qui ont abordé la conjugaison¹¹. Cependant Bescherelle a été critiqué parce qu'il serait "suranné", qu'il contiendrait des verbes qui ne sont plus en usage courant, et qu'il vaudrait mieux se baser sur un dictionnaire plus récent comme celui des Caput. Nous allons maintenir toutefois que loin d'être vieilli, Bescherelle nous servira comme guide à la fois fiable et très convenable. L'ouvrage des Caput, bien qu'il soit très utile dans d'autres contextes, l'est moins en ce qui concerne notre recherche parce qu'il ne comporte qu'un échantillonnage de tous les verbes. En

¹¹ Par exemple ISAAC (1985) p8, GERTNER (1973) p12.

plus on est tenté parfois de douter que cet échantillon soit le plus représentatif.

Prenons un exemple. Le *Dictionnaire des Verbes Français* de J.P. et J. Caput présente "les particularités morphologiques et syntaxiques.... les plus communes" des verbes qui appartiennent au "vocabulaire commun du français parlé aujourd'hui"¹², ainsi que des tableaux des conjugaisons. Il est donc surprenant de trouver dans ce dictionnaire le verbe *verglacer*: *il verglace*, un verbe dont Grevisse remarque qu'il "était déjà noté comme vieux par Bescherelle en 1846.... Nous n'avons relevé qu'un seul exemple au 20^e siècle, et avec la graphie *verglasse*: *Il neige et il pleut et il grêle et il vente et il verglasse* (GIRAUDOUX, *Littérature*, p. 317)"¹³.

Il est vrai que l'Editeur de Bescherelle lui aussi dans son avertissement, affirme, comme nous l'avons noté ci-dessus, que "quelques rares verbes désuets ont été abandonnés", et que néanmoins *verglacer* se trouve dans le dictionnaire orthographique de 7000 verbes. Pourtant Bescherelle vise à l'exhaustivité en matière de la conjugaison alors que les Caput se concentrent sur la morpho-syntaxe.

Pour l'analyse nous avons choisi d'utiliser principalement l'ouvrage de Bescherelle, tout en complétant ses renseignements

¹² Préface pVIII

¹³ *Bon Usage* 754a (p1186-7).

avec ceux qui se trouvent dans le dictionnaire des Caput et avec les tableaux à la fin du *Dictionnaire de la langue française* de Larousse. Bien sûr l'ouvrage bien connu de Grevisse nous a été aussi indispensable. Bescherelle et Caput présentent les formes verbales en tableaux complets, ce qui facilite la vérification des règles, mais ceux de Bescherelle sont plus commodes et -- comme nous l'avons indiqué -- pour nos besoins l'ouvrage est en général plus complet.

Etant donné que ce travail ne se base pas sur des tableaux de conjugaisons (bien que le programme soit capable d'en afficher à l'écran, si l'utilisateur le désire), la façon de grouper les verbes n'est pas importante (par exemple, Bescherelle donne 82 tableaux, Larousse 85, Caput 94 etc). Nous considérerons ces ouvrages comme point de départ et - ce qui est plus important - comme point d'appui en ce qu'ils nous fourniront des formes conjuguées que nous souhaiterons comme "output" du programme. Autrement dit, ils nous fourniront un corpus bien que leurs analyse et groupement ne soient pas utiles à notre travail.

1.5

LE LANGAGE PROLOG

Le programme CONJUGUE a été écrit en Turbo Prolog.

1.5.1. Quelques difficultés

Il est vrai que programmer en Prolog n'est pas facile. Jean-Paul Delahaye, dans son introduction¹⁴ dit que, "C'est en effet une caractéristique de Prolog qu'un débutant est absolument incapable de faire certaines choses élémentaires, et que même parfois, il les juge impossibles à réaliser en Prolog!"

En citant quelques caractéristiques de Prolog Delahaye explique en quoi ce langage déclaratif est différent des autres langages de programmation et il donne une liste d'inconvénients que cette différence entraîne. Nous en noterons et en commenterons ici quelques-uns :

- la logique rend difficile et peu naturelle la programmation de certaines tâches de type "impératif" (il est vrai que, faute de structures prédéfinies comme SI...ALORS, SINON des routines élémentaires trouvées dans d'autres langages sont rendues assez difficiles).
- les variables ne gardent pas leurs valeurs (les variables sont locales, on est obligé de transmettre leurs valeurs tout le long du traitement. La seule alternative c'est d'utiliser quelques astuces qui ne sont pas dans l'esprit de la

¹⁴ Delahaye, 1988, XIX.

programmation en logique.

- Prolog est *non-déterministe* ce qui demande une connaissance fine de son calcul (ce non-déterminisme est parfois utile dans notre programme, parfois gênant, et nous avons été obligé d'utiliser largement une astuce afin de le contrôler).

1.5.2. Quelques avantages

En revanche il y a bien sûr des avantages dont le plus remarquable est peut-être le fait que la programmation se fasse à un niveau d'abstraction assez élevé. Pour notre travail cela signifie que les règles linguistiques se traduisent facilement en programme Prolog. Considérons l'exemple suivant de notre programme¹⁵. Il s'agit de trois règles d'un sous système permettant de déterminer la voyelle thématique à utiliser au passé simple et au subjonctif imparfait pour les verbes des 2^e et 3^e conjugaisons. Les règles doivent s'appliquer dans l'ordre indiqué.

$$1. V \longrightarrow i / \left[\begin{array}{l} \text{-(taire, savoir, devoir, être, pouvoir, pourvoir)} \\ \text{+ (ill, vr, fr, pr, mp, gn, qu, nc, a, v, s, f, d, t)} \end{array} \right] \text{ --- +TEMPS} > 7$$

$$2. V \longrightarrow \emptyset / \{in, i, u, i\} \text{ --- + TEMPS} > 7$$

$$3. V \longrightarrow u / \text{ + TEMPS} > 7$$

Et voyons maintenant cette formalisation traduite en PROLOG:¹⁶

¹⁵ Les règles sont données dans l'annexe D, ainsi que quelques renseignements concernant le langage formel utilisé.

¹⁶ Les détails de la programmation seront expliqués ultérieurement (3^e partie) ainsi que cette exemple (2^e partie). Pour l'instant les comprendre n'est pas important.

```
 affecter_voy_thém(TEMPS,RAD,PERS,INF,i):-  
   TEMPS>7,  
   rad(R), quelle était la forme du radical au début du traitement?  
   exclure(INIT,ELEM,R,[tai,sav,dev,êt,mouv,pourv]),  
   not(concat(INIT,ELEM,R)),  
   appartenance(INIT2,ELEM2,[ill,vr,fr,pr,mp,gn,qu,nc,m,v,s,f,d,t],RAD),!.  
 affecter_voy_thém(TEMPS,RAD,PERS,INF,0):-  
   TEMPS>7,  
   appartenance(INIT,ELEM,[in,i,i,u],RAD),!.  
 affecter_voy_thém(TEMPS,RAD,PERS,INF,"u").
```

La ressemblance des deux schémas est bien évidente: trois clauses dans le programme pour les trois règles, les mêmes listes, bien que la liste des verbes à exclure soit dans le programme une liste de radicaux plutôt qu'une liste d'infinitifs comme ici dans les règles.

1.6. LES SYSTEMES EXPERTS

1.6.1 Les systèmes experts en général

Imaginons un système comportant toutes les connaissances d'un domaine donné et que nous voulons nous renseigner sur quelques faits en particulier. Un système expert est une base de données de cette sorte qui permet une interrogation de plusieurs types. Cette interrogation se fait uniquement au moyen d'une interface quelconque, qui peut même être une interface en langue naturelle (*natural language interface*) permettant à l'utilisateur de dialoguer avec la base de données dans sa propre langue.

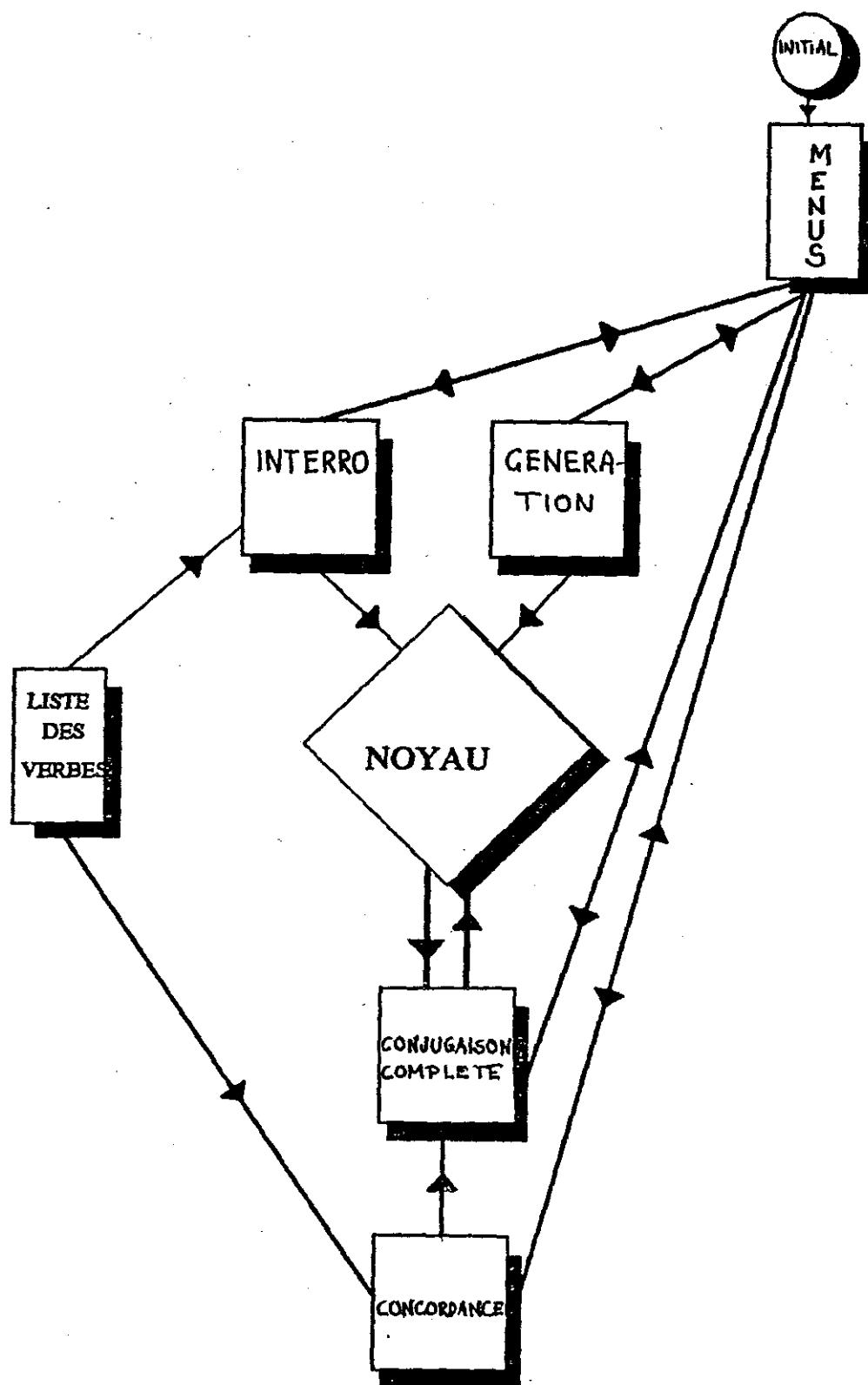
Les systèmes experts les mieux connus sont probablement les systèmes experts médicaux. Supposons que nous observions en une personne, quelques symptômes, mais que nous ne possédions pas les connaissances nécessaires pour en faire un diagnostic. Nous pourrions démarrer un système expert qui nous poserait des questions générales auxquelles nous n'aurions qu'à répondre 'oui' ou 'non'. Chaque question que poserait l'interface se baserait sur la réponse donnée à la question précédente. Les questions deviendraient de plus en plus spécifiques et le système procéderait ainsi jusqu'à ce qu'il puisse nous informer sur la nature de la maladie. Les questions que poserait l'interface pourraient être soit des phrases intégrales stockées dans le système et appelées l'une après l'autre, soit des questions générées par le système à partir d'un lexique et d'une base de règles morpho-syntaxiques et sémantiques.

Une interface en langage naturel donnerait à l'utilisateur la possibilité de procéder aussi dans l'autre sens, c'est-à-dire que l'utilisateur poserait les questions et le système répondrait. Ainsi, l'utilisateur n'a pas besoin d'initier le système, mais il peut directement faire une requête donnant des détails, par exemple, les symptômes qu'il voit, et le système lui répondra, soit directement, soit en posant quelques questions lui même afin d'obtenir des renseignements supplémentaires qui vont l'aider à répondre à la requête initiale.

1.6.2 Notre système pour la conjugaison verbale

Le système sur lequel se fonde le programme Conjugue n'a pas d'interface en langage naturel mais il est néanmoins organisé à peu près de la même manière (voir diagramme à la page suivante). Nous accédons au système par le module MENUS. Le coeur du système est la base de données, ou plutôt la base de règles (le NOYAU), autour duquel sont rangés trois différents modules (INTERRO, GENERATION, CONJUGAISON COMPLETE -- les interfaces) nous donnant accès aux règles. Les flèches montrent les directions d'entrées et de sorties possibles. La liste de verbes (un échantillon des verbes les plus fréquents) n'a rien à voir avec la réalisation de la conjugaison mais sert uniquement à fournir le module INTERRO des verbes pour interroger l'utilisateur. Nous pouvons consulter cette liste au moyen du module CONCORDANCE.

1.6.3 Diagramme des divers modules

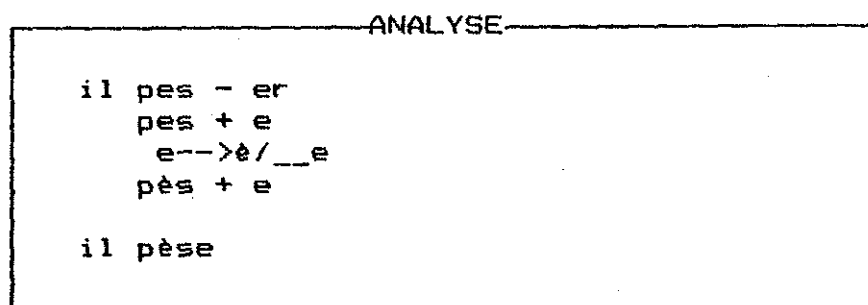


DEUXIEME PARTIE

L'analyse linguistique

2.1 Introduction

Nous avons déjà expliqué à propos du programme pour lequel a été faite l'analyse qui sera mise en oeuvre qu'il est un didacticiel. Ce programme a pour objectif d'aider l'apprenant à conjuguer à partir des formes que l'on trouve dans un dictionnaire. On suppose que l'utilisateur voudrait plutôt maîtriser la langue française, c'est-à-dire l'utiliser, que de se lancer à la découverte des lois internes qui régissent les transformations phonologiques et grammaticales. Toutefois il ne serait pas tout à fait sans intérêt pour l'apprenant d'en apprendre quelque chose, bien que ce "quelque chose" soit forcément limité, et ce programme contient une fenêtre qui présente l'analyse des étapes suivies dans le processus de la conjugaison:



Les règles comme celle-ci sont assez simples et pour ainsi dire n'exigent pas des connaissances linguistiques poussées.

Nous allons donc laisser de côté toute abstraction et nous nous concentrerons sur les formes uniquement superficielles, c'est-à-dire, dans cette analyse les formes écrites de l'orthographe française.

Toute question concernant le niveau de langue ne sera alors pas pertinente pour notre analyse. La liaison, pour nous, n'aura aucune signification¹. Nous n'avons donc pas besoin de rendre compte des formes sous-jacentes qui ne se réalisent que sous certaines conditions puisque les verbes sont traités hors contexte, sans tenir compte de ce qui les suit. On a, par exemple, le fait que la forme sous-jacente de la 3^{ème} personne de l'indicatif présent *il parle* est, en réalité /il paRlɔ(t)/, avec un t qui est réalisé uniquement lorsque la forme qui suit le verbe commence par une voyelle -- *parle-t-il* /parlɔtil/. Une analyse contextuelle devrait donc considérer cette désinence comme /ɔ(t)/, mais de notre côté, nous pouvons passer de tels phénomènes sous silence.

Il aurait certes été intéressant de faire une analyse beaucoup plus rigoureuse sur le plan morphologique et phonologique, mais j'ai considéré qu'un tel approfondissement dans le cadre de ce travail n'était pas nécessaire. Le conjugueur a été conçu comme outil pratique pour ceux qui apprennent le français, quel que soit le niveau de leurs connaissances linguistiques. Il faut donc qu'il soit capable de donner la forme conjuguée à partir d'une forme écrite selon les normes de l'orthographe et qu'il génère aussi des formes qui seront immédiatement reconnues par l'utilisateur non-spécialiste.

¹ A part la liaison entre je et un verbe qui commence en voyelle: je aime —> j'aime.

En revanche, pour certains phénomènes il aurait été bien utile d'avoir recours à une description phonologique. Cependant, l'idée de convertir une forme nécessairement orthographique en forme phonémique pour l'analyse et puis la reconvertir en forme orthographique pour la sortie nous parassait trop compliquée comme *modus operandi* pour les limites que nous nous sommes imposées.

L'analyse qui suit est une analyse assez simple, une analyse qui ne traite que des formes graphiques et qui ne serait donc pas capable d'expliquer des phénomènes et des épiphenomènes phonologiques et morphologiques. Un traitement global de la langue demanderait une analyse plus rigoureuse que le présent travail, mais nous avons jugé que celle dont il est question ici, bien que simple, serait néanmoins suffisante du moins en ce qui concerne les traits dominants de notre travail.

2.2 Deux prédécesseurs

2.2.1 GERTNER, M.H. - The Morphology of the Modern French Verb

Le travail de M.H. Gertner date de 1973. Il avait pour objet de réduire les dizaines de paradigmes verbaux à un système de règles morphophonémiques. Son modèle comporte des règles de trois types:

- des règles générales s'appliquant à tous les verbes sans exception, mais inapplicables dans certains cas
- des règles "irrégulières" ou spéciales ne s'appliquant qu'à un nombre restreint de verbes

- quatre règles uniques, remplaçant quatre règles régulières. Afin de générer correctement toutes les formes de chaque verbe il propose une série de 55 règles pour les verbes "réguliers" et 66 règles supplémentaires pour rendre compte des formes "irrégulières". Dans un deuxième temps son modèle s'adapterait aussi au français parlé qui, plus simple, ne nécessite que 33 règles.

M. Gertner prend comme système initial les divisions traditionnelles en 1^{re}, 2^e et 3^e conjugaisons mais il préfère parler des verbes "régulièrement irréguliers" (*regularly irregular*) ou "irrégulièrement réguliers" (*irregularly regular*) plutôt que d'utiliser la désignation normale "irrégulier" parce que les notions traditionnelles de régularité serait "too superficial".

Il se concentre principalement sur le français écrit, d'où l'acception de sa part des définitions traditionnelles des sons du français. Pourtant il ne se limite pas à l'orthographe mais utilise une notation quasi-phonologique, laissant de côté les traits distinctifs de la phonologie à proprement parler. Le fait qu'il tient compte des verbes archaïques et défectifs et qu'il consacre deux chapitres aux affixes dérivationnels pourrait indiquer qu'il s'intéresse à la *diachronie*. Dans notre travail nous avons en gros, en notant ses remarques sur la défectivité, adopté cette approche.

Selon L. Isaac, M. Gertner aurait implémenté son modèle comme programme ce qui "lui a permis de faire conjuguer un ordinateur"², mais, à part cette observation, il ne nous a guère été possible de relever d'autres détails concernant ledit programme.

2.2.2 ISAAC, Luc - Calcul de la flexion verbale en français contemporain

L'ouvrage de L. Isaac est plus récent (1985) et a profité non seulement du travail de M. Gertner mais aussi d'une oeuvre allemande concernant la conjugaison du verbe français, le Generative Morphologie des Neufranzösischen de F. Kiefer (1973)³. Le travail de L. Isaac est donc plus rigoureux que celui de Gertner surtout dans son traitement de la phonologie. Cette rigueur l'a obligé à consacrer une grande partie de l'ouvrage à des explications sur la démarche suivie avant d'aborder le corpus propre et l'élaboration de l'algorithme. L'ouvrage a légèrement influencé notre travail mais nous l'avons obtenu trop tard pour qu'il put influencé vraiment la structure de notre algorithme. Quelques précisions, concernant notamment le langage formel⁴, ont néanmoins été retenues.

L. Isaac, lui aussi, aurait dans un précédent travail construit un programme de conjugaison "qui fournit les dérivations des formes verbales des conjugaisons chanter et finir. A propos de

² ISAAC 1.4.1 p 14, et une note: "Avec la collaboration de A. Zingis ; cf. Computers and the Humanities, 10.1 (janvier-février 1976), p. 46."

³ Nous n'avons pas vu cet ouvrage.

⁴ Pour son langage formel voir chapitre 6 de son ouvrage.

DEUXIEME PARTIE

son *Calcul de la flexion verbale* il dit que "nous envisageons la construction d'un programme plus puissant, qui rende la machine capable de traduire elle-même les règles linguistiques que nous dégagerons en instructions auxquelles elle puisse se conformer". Il veut "apprendre" à un ordinateur non seulement à conjuguer, mais surtout à analyser le verbe français, c'est-à-dire à préciser lui-même quelles règles il aura appliquées dans le calcul de chaque forme verbale"⁵.

L'apprenant n'est pas visé dans son travail et en fait il en ignore l'aspect didactique: "Nous avons refusé de prendre d'emblée une optique pédagogique". "Notre démarche constitue essentiellement une recherche fondamentale puisque son but immédiat est la description et l'explication scientifique d'un corpus". Bien qu'il soit d'accord sur le fait que "des applications plus pratiques ne sont pas inconcevables: d'abord dans l'enseignement de la langue à des locuteurs non francophones..." il maintient, par exemple, que "personne n'ira soutenir que la physique aurait dû être fondée sur des bases pédagogiques. Bien au contraire, le professeur de physique modifie ce qu'il connaît de cette science, et il exprime sa pensée en termes trop simplifiés d'un point de vue scientifique strict, mais compréhensibles pour ses étudiants. Il doit en aller ainsi de la linguistique."⁶ Néanmoins il note qu'on pourrait "se laisser fasciner par les règles formelles au point d'oublier

⁵ 1.1.2.3, pp 5-6.

⁶ 1.1.3.2 p 7

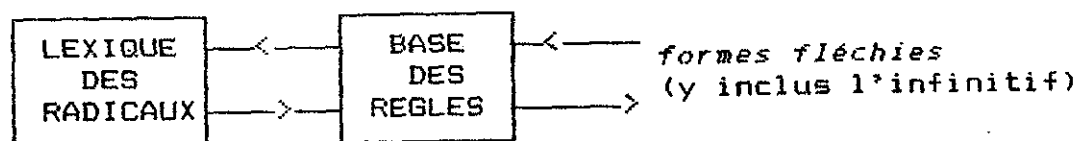
qu'elles servent avant tout à comprendre et à expliquer le mieux possible une certaine réalité"⁷.

Nous voyons ici nettement en quoi les objectifs du modèle de L. Isaac diffèrent du ceux du présent travail.

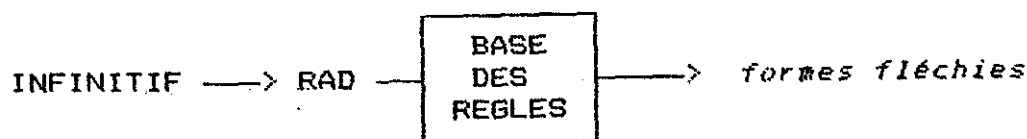
2.2.3

Conclusions

Ces deux travaux prennent comme point de départ un radical sous-jacent ce qui leur permet de traiter l'infinitif comme forme fléchie et, on suppose, de réaliser la reconnaissance des flexions ainsi que leur génération:



1. Système bidirectionnel permettant la génération et la reconnaissance des formes fléchies. Un lexique comportant des radicaux d'un niveau d'abstraction élevé sert comme point de départ pour générer les formes conjuguées et aussi comme référence pour leur identification.



2. Système unidirectionnel limité à la génération des formes conjuguées et cela à partir d'une forme déjà conjuguée, l'infinitif. Notre modèle est de cette sorte.

L'algorithme du présent travail a été élaboré quasi-indépendamment de ces deux prédécesseurs. A la limite on pourrait dire que son langage formel ressemble à celui de L. Isaac, mais il est peut-être plutôt structuré d'après l'algorithme de M. Gertner.

⁷ 13.2.1.1 p 427

2.3 La problématique

La conjugaison du verbe français comporte trois personnes, deux nombres, et sept temps⁹, ce qui donne 42 formes conjuguées. En plus il y a deux participes (l'un d'eux ayant 4 formes possibles) un impératif à trois formes (2^e pers du singulier, 1^{er} et 2^e pers du pluriel) et un infinitif. Cela donne un total de 51 formes. Il y a aussi les temps composés, comme *je suis allé*, ayant pour auxiliaire soit *être*, soit *avoir*. Toutefois ces dernières n'introduisent aucune forme nouvelle, et nous les laisserons de côté dans notre analyse et dans le programme.

Comment donc générer toutes ces formes, et à partir de quoi? Nous avons déjà déclaré qu'étant donné que le programme est un didacticiel il sera inutile de prendre comme système initial une forme abstraite. Il vaudrait mieux que l'utilisateur puisse commencer avec une forme qui lui est familière. Traditionnellement quand on parle d'un verbe on en parle en utilisant la forme qu'on trouve dans un dictionnaire, on utilise l'infinitif. On dit plutôt "le verbe *finir*" que "le verbe *fini-*". Nous allons donc prendre comme point de départ l'infinitif du verbe, ce qui signifie que nous n'aurons pas besoin de le générer et que notre liste de formes cibles comportera une forme en moins.

⁹ Pour le sens du mot "temps" dans ce travail voir section 3.1.1 "Les temps".

2.3.1 Un schéma simple

Il serait concevable de faire un conjugueur sans s'occuper du problème des règles. En PROLOG, on pourrait réaliser un système expert où chaque personne de chaque temps et mode est entrée comme suit:

```
verbe(aimer,indic_prés,1,aime).
verbe(aimer,indic_prés,2,aises).
verbe(aimer,indic_prés,3,aime).
verbe(aimer,indic_prés,4,aimons).
verbe(aimer,indic_prés,5,aisez).
verbe(aimer,indic_prés,6,aiment).
verbe(aimer,indic_imparf,1,aimais).
verbe(aimer,indic_imparf,2,aimais).
    etc
verbe(tenir,indic_prés,1,tiens).
verbe(tenir,indic_prés,2,tiens).
verbe(tenir,indic_prés,3,tient).
verbe(tenir,indic_prés,4,tenons).
verbe(tenir,indic_imparf,1,tenais).
verbe(tenir,indic_imparf,2,tenais).
    etc
verbe(avoir,subj_imparf,1,eusse).
verbe(avoir,subj_imparf,2,eusses).
verbe(avoir,subj_imparf,3,eût).
verbe(avoir,subj_imparf,4,eussions).
verbe(avoir,subj_imparf,5,eussiez).
verbe(avoir,subj_imparf,6,eussent).
    etc
```

Bien qu'il soit simple à réaliser (sinon laborieux), on voit immédiatement qu'on aurait besoin d'une base de données énorme pour contenir toutes les informations concernant toutes les conjugaisons possibles de tous les verbes de la langue française. Nous pouvons nous limiter à un échantillon de, par exemple, 600 verbes - ceux notés dans l'index de Bescherelle comme particulièrement fréquents. Mais même en omettant les formes composées et l'infinitif (qui va nous servir comme point de départ), en partant du principe que chaque verbe a 50 formes un simple calcul nous montre que nous aurions besoin de 600 x 50

soit 300000* lignes dans notre base de données.

En plus un tel système générerait beaucoup de redondance, beaucoup de répétition, et - faute plus grave - ne nous éclairerait pas sur ce qui se passe "intérieurement", c'est-à-dire les relations internes entre telle et telle forme. Il ne serait pas capable de nous montrer la marche à suivre pour conjuguer un verbe. Si jamais nous voulions entrer un verbe absent de notre base de données le programme insisterait sur le fait qu'il n'existe pas.

2.3.2 Un schéma plus puissant

Les ouvrages comme Bescherelle ou Larousse ne fournissent pas pour chaque verbe de la langue, considérée dans sa totalité, un tableau, mais choisissent plutôt quelques verbes modèles qui servent à en représenter également plusieurs autres. Un index quasi-exhaustif en fin de volume aide à résoudre les problèmes de conjugaison par le renvoi aux tableaux servant de modèles. Nous pourrions donc procéder de la même manière avec notre programme.

Pour les verbes de type *aimer*, par exemple:

```
verbe(RAD,indic_prés,1,e).
verbe(RAD,indic_prés,2,es).
verbe(RAD,indic_prés,3,e).
verbe(RAD,indic_prés,4,ons).
verbe(RAD,indic_prés,5,ez).
verbe(RAD,indic_prés,6,ent).
    etc
verbe(RAD,indic_passé_simple,1,a).
verbe(RAD,indic_passé_simple,2,as).
verbe(RAD,indic_passé_simple,3,a).
```

* En réalité un peu moins, puisque plusieurs de ces verbes les plus fréquents sont défectifs et manquent donc quelques formes.

```
verbe(RAD,indic_passé_simple,4,âmes).  
verbe(RAD,indic_passé_simple,5,âtes).  
verbe(RAD,indic_passé_simple,6,èrent).
```

Au lieu d'avoir chaque verbe entré séparément, notre schéma contient une variable RAD, mais encore une fois on constate qu'il est peu efficace car il entraîne une redondance évidente. Nous avons certainement réussi à réduire la taille de la base de données mais nous avons comme effet de bord créé plusieurs autres problèmes que nous commenterons maintenant.

1. Il est évident que bien qu'un tel système soit à la limite fiable pour les verbes dits *réguliers*, ceux qui appartiennent traditionnellement à la première et à la deuxième conjugaison, mais nous aurions du mal à rendre compte de l'abondance de particularités pour les verbes de la 3^e conjugaison. Dans un premier temps il y a le fait que le radical subit des changements non seulement selon le temps et le mode, par exemple:

```
tenir 1ère pers, indic_prés      --> tien-  
                  indic_imparf   --> ten-  
                  indic_futur    --> tiend-  
                  indic_passé_simple --> t-
```

mais aussi à l'indicatif présent, il change selon la personne:

```
tenir indic_prés, 1 ---> tien-  
                  2 ---> tien-  
                  3 ---> tien-  
                  4 ---> ten-  
                  5 ---> ten-  
                  6 ---> tienn-
```

Dans un deuxième temps il y a le problème des exceptions (même avec quelques verbes de la première conjugaison), sans parler des

DEUXIEME PARTIE

verbes défectifs et des verbes unipersonnels.

2. Notre système soulève aussi le problème de catégorisation: comment le programme saura-t-il à quelle conjugaison appartient chaque verbe? Nous aurions besoin d'une liste de tous les verbes, numérotés comme dans le dictionnaire orthographique de Bescherelle, qui elle, encore une fois signifierait une base de données importante.

3. A propos de la redondance nous remarquons que très souvent nous serions obligés de répéter les mêmes terminaisons .

En bréf, un logiciel modélé sur les guides de la conjugaison "sous forme de document" ne s'avère pas très intéressant pour un traitement par ordinateur. Il nous faudra un autre modèle.

2.3.3 Un schéma fiable

Nous avons déjà remarqués en regardant les sorties de nos deux premières tentatives, qu'il y a beaucoup de choses qu'on pourrait régulariser ou organiser. Les mêmes phénomènes se repètent maintes fois et on peut même relever des schèmes généraux. En gros on voit que la conjugaison se compose toujours d'une racine et d'une terminaison, schème que l'on peut représenter comme suit:

FORME CONJUGUEE -----> RACINE + TERMINAISON

Ici on a trois variables FORME CONJUGUEE, RACINE et TERMINAISON.
 La règle signifie qu'une forme conjuguée se réécrit en une racine et une terminaison. Ces deux dernières peuvent être elles mêmes décomposées comme suit:

RACINE -----> RADICAL + VOYELLE THEMATIQUE
 TERMINAISON -----> { TEMPS + PERSONNE
 FORMES "NON-CONJUGUEES" }

et on peut continuer ainsi:

TEMPS -----> { indicatif présent
 indicatif imparfait
 indicatif futur simple
 indicatif passé simple
 conditionnel
 subjonctif présent
 subjonctif imparfait
 impératif }

FORMES NON-CONJUGUEES -----> { infinitif
 participe présent
 participe passé }

VOYELLE THEMATIQUE -----> { a
 e
 i
 u
 ø }

PARTICIPE -----> { présent
 passé }

PARTICIPE PASSE -----> GENRE + NOMBRE

PERSONNE -----> { 1 sg
 2 sg
 3 sg
 1 pl
 2 pl
 3 pl }

Ainsi, en employant des règles que nous allons bientôt mettre au point, nous pourrions catégoriser et conjuguer correctement

DEUXIEME PARTIE

ANALYSE

n'importe quel verbe de la langue française, cela à tous les temps et toutes les personnes, sans avoir recours à une liste exhaustive et sans crainte que le programme ne rejette le verbe que nous avons entré¹⁰.

¹⁰ Il faut en même temps admettre que notre système ne sera pas capable d'exclure des "verbes" qui n'existent pas. Si un graphème non-réalisé dans la langue française se ressemble à un graphème qui existe, le programme l'acceptera et le conjuguera - par exemple, *tenir* par analogie avec *tenir*. Même une forme complètement impossible en français comme *lxsdjcer* sera acceptée puisqu'elle se termine en *-cer* et elle sera conjuguée comme *placer*.

3

VERS UNE FORMALISATION

3.1 Deux aspects non-phonétiques

Avant de considérer le verbe même nous devons formaliser deux aspects non-phonétiques du processus - les temps et les personnes.

3.1.1 Les temps

Nous considérerons comme *temps* chaque réalisation temporelle d'un *mode*. Ainsi même les formes "non-conjuguées", les participes, seront des temps. Toutefois traditionnellement les temps sont regroupés selon le *mode*, comme suit:

INDICATIF	SUBJONCTIF	IMPERATIF	PARTICIPE	CONDITIONNEL
présent	présent	présent	présent	présent
imparfait	imparfait		passé	
passé simple				
futur				

Bien qu'un tel regroupement soit linguistiquement satisfaisant il n'est pas du tout suffisant pour un traitement formel. Pour ce dernier il nous faudra une analyse qui se concentre sur les formes dans lesquelles les conjugaisons se présentent et qui n'a pas besoin de s'intéresser à leur emploi. Nous allons alors considérer un temps comme existant indépendamment, sans tenir compte de la notion de mode.

Pour une première tentative d'une formalisation prenons l'ensemble suivant:

ENSEMBLE_A = {indicatif_présent, indicatif_imparfait,
indicatif_passé_simple, indicatif_futur,
subjonctif_présent, subjonctif_imparfait,
impératif, participe_présent, participe_passé,
conditionnel}

On constate qu'il existe une certaine symétrie entre quelques formes. M. Grevisse remarque: "Quand le futur est usité, le conditionnel l'est aussi (et avec le même radical). De même, si le passé simple existe, le subjonctif imparfait existe aussi"¹¹. Comme nous le dit Grevisse, le passé simple et le subjonctif imparfait ont le même radical, et ils ont aussi la même voyelle entre radical et terminaison. De même pour le futur et le conditionnel:

AIMER			
	radical	voyelle	terminaison (1 p sg)
passé simple	aim-	-a-	-i
subjonct. impf	aim-	-a-	-sse
futur	aim-	-e-	-rai
conditionnel	aim-	-e-	-rais

VIVRE			
	radical	voyelle	terminaison
passé simple	véc-	-u-	-s
subjonct. impf	véc-	-u-	-sse
futur	viv-	Ø	-rai
conditionnel	viv-	Ø	-rais

DORMIR			
	radical	voyelle	terminaison
passé simple	dorm-	-i-	-s
subjonct. impf	dorm-	-i-	-sse
futur	dorm-	-i-	-rai
conditionnel	dorm-	-i-	-rais

¹¹ Bon Usage 846 note [p1273].

	ALLER		
	radical	voyelle	terminaison
passé simple	all-	-a-	-s
subjonct. impf	all-	-a-	-sse
futur	i-	Ø	-rai
conditionnel	i-	Ø	-rais

En outre on remarque que les temps ci-dessus prennent une voyelle entre le radical et la terminaison¹² tandis que les autres conjugaisons n'en prennent pas:

AIMER		
	radical	terminaison (1 pers sg)
indicatif présent	aim-	-e
indicatif imparfait	aim-	-ais
participe présent	aim-	-ant
participe passé	aim-	-é
subjonctif présent	aim-	-e
impératif	aim-	-e (2ème pers)

DORMIR		
	radical	terminaison (1 pers pl)
indicatif présent	dorm-	-ons
indicatif imparfait	dorm-	-ions
participe présent	dorm-	-ant
participe passé	dorm-	-i
subjonctif présent	dorm-	-ions
impératif	dorm-	-ons

On peut donc diviser ensemble A en deux sous-ensembles:

ENSEMBLE_B = {indicatif_futur, conditionnel, passé_simple, subjonctif_imparfait}

ENSEMBLE_C = {indicatif_présent, subjonctif_présent, indicatif_imparfait, impératif, participe_présent, participe_passé}

Ensemble_B, selon ce qu'on a déjà dit, se compose lui même de deux sous-ensembles:

ENSEMBLE_D = {indicatif_futur, conditionnel}

ENSEMBLE_E = {indicatif_passé_simple, subjonctif_imparfait}

Considérons maintenant l'ensemble C. Il contient les deux participes, qui ne prennent pas de terminaisons à marque de

¹² En principe, bien que quelques verbes, comme aller ci-dessus, n'en prennent pas.

personne, et l'impératif, qui est utilisé normalement sans pronom. Ainsi on peut grouper ces trois ensembles par opposition aux autres trois qui exigent un pronom.

ENSEMBLE_F = {impératif, participe_présent, participe_passé}
ENSEMBLE_G = {indicatif_présent, indicatif_imparfait,
 subjonctif_présent}

Ce regroupement, qui n'a rien à voir avec le sens et qui est uniquement basé sur les formes dans lesquelles les divers temps se réalisent, va nous aider à gérer les 10 différents temps et modes. On pourrait représenter chaque temps comme terme Prolog, soit comme nous l'avons fait ci-dessus - "indicatif_présent", par exemple - soit en affectant un "code" à deux lettres - IP pour indicatif présent, PR pour participe présent etc. Toutefois un tel système n'est pas le plus rentable. Nous serions obligés de maintenir plusieurs ensembles, comme ci-dessus, les appeler et les tester afin de vérifier si le temps demandé figure dans telle ou telle liste. Beaucoup plus efficace serait de les représenter d'une façon numérique, ce qui nous aiderait à faire des calculs. Affectons alors à chacun des 10 temps un numéro, de 0 à 9:

No.	Temps	Code
0	Indicatif présent	IP
1	Subjonctif présent	SP
2	Indicatif imparfait	II
3	Impératif	IM
4	Participé présent	PR
5	Participe passé	PA
6	Indicatif futur	FS
7	Conditionnel	CD
8	Indicatif passé simple	PS
9	Subjonctif imparfait	SI

Cela s'avérera plus efficace et beaucoup plus économique qu'un traitement utilisant des ensembles. Supposons qu'au début du traitement une telle valeur numérique est toujours affectée au verbe (nous verrons dans la troisième partie comment) et alors pour identifier l'ensemble B nous n'avons qu'à déclarer "TEMPS>5", pour ensemble C "TEMPS<6", pour ensemble D "TEMPS>6 AND TEMPS<8" et pour un temps en particulier, par exemple l'indicatif présent, "TEMPS=0".

Les codes à deux lettres

Nous avons aussi donné ici à chaque temps un code à deux lettres, uniquement pour les descriptions qui suivent afin d'éviter la répétition de "l'indicatif présent" etc. Le programme ne traite les temps que comme des numéros et ne sait rien de ces codes.

3.1.2 Les personnes

Les personnes ne nous posent normalement pas de problème de représentation puisqu'elles sont déjà considérées comme numéros, 1^e, 2^e pers, sg et pl etc. Cependant, cette numérotation selon le sens ne nous convient pas pour notre traitement formel. Considerons les données suivantes:

je meurs	je dois	je peux	je veux
tu meurs	tu dois	tu peux	tu veux
il meurt	il doit	il peut	il veut
nous mourons	nous devons	nous pouvons	nous voulons
vous mourez	vous devez	vous pouvez	vous voulez
ils meurent	ils doivent	ils peuvent	ils veulent

L'alternance eu - ou et oi - e est phonétiquement conditionnée. Le déplacement de l'accent à la 1^e et la 2^e personne du pluriel

DEUXIEME PARTIE

permet la réalisation de la voyelle /u/ au lieu de /œ/ dans le cas où la terminaison est muette. Mais notre analyse n'est pas capable de tenir compte des effets phonétiques ou phonologiques et cela nous prive de la possibilité de spécifier une règle comme: ¹³

$$\text{œ} \longrightarrow \text{u}/[+\text{accent}]$$

En plus nous prenons l'infinitif comme point de départ ce qui nous oblige à considérer *ou* et *e* comme formes de base et donc à procéder dans le sens contraire à la règle ci-dessus. Nous voyons néanmoins que la troisième personne du pluriel a le même radical que les personnes du singulier et nous pouvons reformuler notre règle comme suit:

$$\text{ou} \longrightarrow \text{eu}/ \begin{bmatrix} \text{s} \\ \text{x} \\ \text{t} \\ \text{ent} \end{bmatrix}$$

Ou bien comme suit:

$$\text{ou} \longrightarrow \text{eu}/ \begin{bmatrix} 1 \text{ sg} \\ 2 \text{ sg} \\ 3 \text{ sg} \\ 3 \text{ pl} \end{bmatrix}$$

Faute alors d'une représentation phonétique il nous serait parfois utile de pouvoir considérer la troisième personne du pluriel comme appartenant au même ensemble que les trois

¹³ Des définitions du langage formel utilisé sont présentées dans l'annexe.

personnes au singulier. Dans ce cas nous pourrions le faire avec une numérotation parallèle à celle qui a été faite pour les temps, et afin de maintenir la symétrie avec celle-là nous commencerons avec 0:

sg.	pl.
1 pers = 0	1 pers = 4
2 pers = 1	2 pers = 5
3 pers = 2	3 pers = 3

La règle peut alors être réécrite ainsi:

ou ---> eu/[+PERS<4]

Dans le cas où nous voudrions grouper ensemble soit toutes les personnes plurielles, et cela se ferait à l'aide de l'expression PERS>2, soit toutes les personnes au singulier ce qui s'exprime ainsi: PERS<3.

3.2 Les terminaisons

Ayant formalisés les temps et les personnes nous avons ce qu'il nous faut pour procéder à une formalisation des terminaisons. L. Isaac (p29) critique "les généralisations manquées" de M. Gertner, qui n'avait pas observé entre autres que les terminaisons du conditionnel peuvent être dérivées de celles de l'imparfait. Nous ne pouvons entièrement profiter des remarques d'Isaac puisque l'orthographe obscurcit quelques détails, lesquels seraient plus clairs à un niveau plus abstrait, mais elles nous aideront quand même à réduire le nombre de terminaisons.

Les ouvrages de conjugaison comme Bescherelle, ne s'intéressant pas aux voyelles thématiques, sont obligés de maintenir une grande liste de terminaisons. Nous laisserons à un traitement ultérieur l'affectation de ces voyelles ainsi que le double s pour les verbes de la 2^e conjugaison. Nous pourrions alors nous limiter aux listes suivantes:

	0	1	2	3	6	7	8	9
0	e s, x	e	ais		rai	rais	i, s	sse
1	es s, x	es	ais	e, s	ras	rais	s	sses
2	e t	e	ait		ra	rait	Ø, t	^t
4	ons	ions	ions	ons	rons	rions	^mes	ssions
5	ez	iez	iez	ez	rez	riez	^tes	ssiez
3	ent	ent	aient		ront	raient	èrent	ssent

Participe présent 4 ant
passé 5 é, i, s, t, Ø

En sus de la dérivation conditionnel ----> r [+ imparfait] les terminaisons des 1 et 2 pers. pluriels (PERS = 4, PERS = 5) se répètent plusieurs fois. Le subjonctif présent a les mêmes terminaisons pour tous les verbes quelque soit leur groupe (à part être) et ne diffèrent de l'indicatif présent que par l'utilisation du i pour les 1 et 2 pers. du pl. Nous pouvons donc facilement dériver le SP de l'IP en ajoutant un i à ces deux:

subjonct prés ----> [+ind prés]/[-(avoir, être)] ____

Ø ----> i/____ $\left[\begin{array}{l} [+subj prés] \\ [+PERS > 3] \end{array} \right]$

Par la suite, le subjonctif imparfait peut être dérivé du subj présent, à l'exception de la 3^e pers sg:

subjonct impf ----> ss $\left[\begin{array}{l} [+ \text{ subj prés}] \\ [- \text{ PERS}=2] \end{array} \right]$

$\left[\begin{array}{l} \text{subjonct impf} \\ \text{PERS} = 2 \end{array} \right] \text{----> } ^t$

Nous pouvons aller plus loin en utilisant le même procédé, mais en fait une structure plus simple pour les données s'avérera un peu plus économique du point de vue de l'utilisation de la mémoire de l'ordinateur. Néanmoins pour cette analyse linguistique nous présenterons une formalisation quasi-complète des terminaisons:

3.2.1 Formalisation

1. IP1 ----> $\left\{ \begin{array}{l} 0 \text{ e} \\ 1 \text{ es} \\ 2 \text{ e} \\ 4 \text{ ons} \\ 5 \text{ ez} \\ 3 \text{ ent} \end{array} \right\} / \left\{ \begin{array}{l} - \left[\begin{array}{l} \text{all} \\ \text{PERS}<3 \end{array} \right] \\ + \left\{ \begin{array}{l} \text{ouvr} \\ \text{ffr} \\ \text{cueill} \\ \text{sail} \end{array} \right\} \\ \left[\begin{array}{l} \ll- [\text{er}] \gg \\ -\text{PERS}<3 \end{array} \right] \end{array} \right\} - \left\{ \begin{array}{l} \ll- [\text{er}] \gg \\ -\text{PERS}<3 \end{array} \right\} \left\{ \begin{array}{l} \ll+ [\text{er}] \gg \end{array} \right\}$

2. IP2 ----> $\left\{ \begin{array}{l} 0 \text{ s} \\ 1 \text{ s} \\ 2 \text{ s} \end{array} \right\} / \left[\begin{array}{l} -\{\text{pouv, voul, val}\} \\ - [\text{PERS}<2] \end{array} \right]$

3. IP3 ----> x

4. SP1 ----> $[+IP] / \left\{ \begin{array}{l} - \left[\begin{array}{l} \text{avoir} \\ \text{PERS}=2 \end{array} \right] \\ - \left[\begin{array}{l} \text{être} \\ \text{PERS}<3 \end{array} \right] \end{array} \right\} -$

4b. $\emptyset \text{ ----> } i / \left[\begin{array}{l} [+SP] \\ [\text{PERS}>3] \end{array} \right]$

5. SP2 ----> $\left\{ \begin{array}{l} 0 \text{ s} \\ 1 \text{ s} \\ 2 \text{ t} \end{array} \right\}$

6. II ----> {ais,ais,ait,ions,iez,aient}
5. FS ----> {rai,ras,ra,rons,rez,ront}
7. CD ----> r [+II]
8. PS1 ----> {s,s,t,^mes,^tes,èrent} / ____ [<-[er]>]
[PERS<3>]
9. PS2 ----> {i,s,t}
10. SI ----> ss [[+SP]
[- PERS=2]]
11. [SI
PERS = 2] ----> ^t

3.3 Les radicaux

Le programme prendra comme point de départ la forme du verbe telle qu'on peut le trouver dans un dictionnaire. La description traditionnelle qui classe les verbes en fonction de leur infinitif et de leur participe présent suffira donc quant aux objectifs fixés, et nous la garderons dans sa quasi-totalité. Par contre, étant donné que les divisions en tableaux, quelque en soit leur numéro, ne nous conviennent pas, nous ne les utiliserons que comme point de départ.

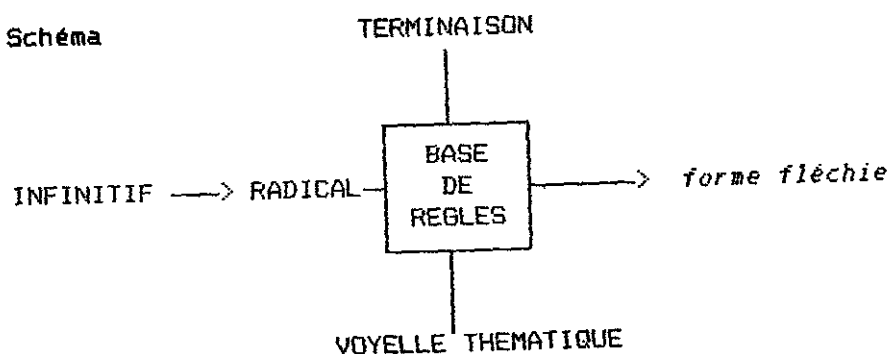
Les tableaux entraînent une certaine redondance. Par exemple, un tableau spécial pour les verbes de type éger (tableau 14 de Bescherelle, 23 de Caput) est inutile. Le schéma de Larousse est plus économique en ce qu'il ne présente que des tableaux pour les verbes de type -ger et -éCer. Lorsque nous avons défini les règles pour les verbes de ces deux derniers types, nous n'avons plus besoin de nous occuper du type -éger, puisque les mêmes règles s'appliquent ici, à la seule différence qu'elles

s'appliquent toutes les deux à la fois. Il en est de même pour les tableaux 18, 19 et 20 de Larousse, les verbes de type *dormir*, *mentir* et *servir*, que nous pouvons traiter ensemble dès que nous aurons défini quelques règles "morphophonologiques"¹⁴.

Dans notre formalisation nous parlerons de la *racine* du verbe, ce qui signifie le verbe moins la désinence. La racine se décompose en un *radical* et une *voyelle thématique*. Le radical n'est effectivement que ce qui reste du verbe dès qu'on a enlevé la terminaison de l'infinitif. Il y a quatre de ces terminaisons: *-er, -re, -oir* et *-ir* (y inclus *-ir*). Il y a quatre voyelles thématiques possibles: *a, i, u* et *e*.

En ayant toujours présent à l'esprit le modèle du traitement présenté ci-dessous, procédons maintenant à l'élaboration de notre corpus.

A. Schéma



B. Règle

INFINITIF ----> RADICAL+VOYELLE+TERMINAISON ----> FORME CONJUGUEE

¹⁴ Voir section 3.3.3.5 plus loin

3.3.1

PREMIERE CONJUGAISON

Marque de l'infinitif: -er (tous les verbes en -er, sauf aller).

3.3.1.1 Particularités des radicaux

plac	c --> q devant {a, o, â}
mang	g --> ge devant {a, o, â}
pes	e --> è devant e-muet
céd	é --> è devant e-muet (mais pas devant er-)
jet	t --> tt devant e-muet (exceptions)
pel	l --> ll devant e-muet (exceptions)
pay	y --> i devant e-muet (alternative paye) (exceptions -eyer)
broy	y --> i devant e-muet
envoy	y --> i devant e-muet (futur enver et pas de voy. thém.)

3.3.1.2 Vowelle thématique

TEMPS 0 - 5	- toujours Ø
au passé simple et subj. imp.	- toujours -a-
au futur et conditionnel	- toujours -e-

3.3.1.3 Formalisation

LISTE1(modeler, celer, déceler, receler, ciseler, démanteler, écarteler, se_encasteler, geler, dégeler, congeler, surgeler, marteler, peler, acheter, racheter, bégueter, corseter, crocheter, fileter, fureter, halet)).

1.
$$\begin{matrix} c \\ g \end{matrix} \longrightarrow \begin{matrix} q \\ ge \end{matrix} / ____ \# [+ \{a, o, â\}]$$
2.
$$\begin{matrix} t \\ l \end{matrix} \longrightarrow \begin{matrix} tt \\ ll \end{matrix} / [\text{-(LISTE1)}] ____ \# [+e_muet]$$
3.
$$e \longrightarrow \text{è} / [+ \{l, t, c, m, n, p, r, s, v, vr\}] ____ \# [+e_muet]$$
4.
$$\text{é} \longrightarrow \text{è} / [+ C\{ \}] ____ \# \left[\begin{matrix} [+ \text{TEMPS} > 5] \\ [+e_muet] \end{matrix} \right]$$
5.
$$\text{oy} \longrightarrow \text{er/ env} ____ \# \left[\begin{matrix} + \text{TEMPS} > 5 \\ + \text{TEMPS} < 8 \end{matrix} \right]$$
6.
$$y \longrightarrow i / [\text{- ey}] ____ \# [+ e_muet]$$
7.
$$V \longrightarrow \text{e} / [\text{- envoy}] ____ \# \left[\begin{matrix} + \text{TEMPS} > 5 \\ + \text{TEMPS} < 8 \end{matrix} \right]$$
8.
$$V \longrightarrow \text{a} / ____ \# [+ \text{TEMPS} > 7]$$

Et bien qu'une définition phonologique d'*e-muet* nous manque, notre numérotation des temps et personnes nous permet de le définir ainsi:

$$9. \quad e_muet \longrightarrow \left[\begin{array}{l} [+ PERS<4] \\ [+ TEMPS=\{0,1,3,6,7\}] \end{array} \right]$$

3.3.2

DEUXIEME CONJUGAISON

Marque de l'infinitif: *-ir* (Ca. 320 verbes), *ir* (*amuir* et *hair*).

3.3.2.1 Particularités des radicaux

Se terminant en:

i ou *amuï*

$\emptyset \longrightarrow$ ss au II, SP, PR et au pl de IP, IM

hai

i $\longrightarrow$ *i* au IP 1,2,3 sg, IMP 2 sg.

3.3.2.2 Voyelle thématique - toujours $\emptyset$

3.3.2.3 Formalisation

$$10. \quad i \longrightarrow i/ha_ \left\{ \begin{array}{l} [+ TEMPS=0] \\ [+ TEMPS=3] \end{array} \right\} [+ PERS<3]$$

$$11. \quad \emptyset \longrightarrow ss/(i,i)_ [+ TEMPS<5] - \left[\begin{array}{l} [PERS<3] \\ \{ [TEMPS=0] \\ [TEMPS=3] \} \end{array} \right]$$

3.3.3

TROISIEME CONJUGAISON

Marques de l'infinitif: *-ir* (ca. 90 verbes), tous les verbes en *-oir* et *-re*.

La troisième conjugaison nous présente de nombreuses irrégularités. Afin de bien cerner les problèmes à résoudre nous présentons un tableau comportant tous les paradigmes des verbes concernés. Ce tableau se base sur ceux de *Bescherelle* mais les divisions diffèrent beaucoup.

DEUXIEME PARTIE

Le tableau comporte sous une forme sommaire tout ce que nous voulons comme sortie de la première partie de notre conjugueur, avant le traitement morphophonologique détaillé plus loin. Toutes les formes que nous voulons passer à la deuxième étape du traitement y sont rangées, et il s'agit des formes les plus abstraites de l'ensemble du traitement. Pour un verbe comme *sentir*, par exemple, nous considérerons *sent-* comme la forme "sous jacente" du radical et la flexion (je) *sens* comme la fusion du radical et la terminaison *sent + s* avec suppression du *t*:

t ---> Ø/sen___#s

Pour d'autres verbes l'orthographe nous empêchera de trouver facilement une forme sous-jacente et nous nous contenterons d'en donner des radicaux différents, par exemple, dans le cas d'*acquérir* dans le tableau nous avons les radicaux suivants:

	acquier-	(IP 1-3 sg)
	acquièr-	(IP 3 pl, SP)
acquérir ---->	acquér-	(IP 1-2 pl, II etc)
	acquér-	(FS, CD)
	acqu-	(PS, SI)

Le tableau donne aussi les voyelles thématiques pour chaque verbe modèle. Les règles seront présentées ultérieurement.

3.3.3.1 Particularités des radicaux

Elément minimal + marque d'infinif.	TEMPS									
	0	1	2	3	4	5	6	7	8	9
aller	va all	aill all	all	all	all	allé	i-ø	i-ø	all-a	all-a
-enir	-ien -en -ienn	-ienn -en	-en	-en	-en	-enu	-iend-ø	-iend-ø	-in-ø	-in-ø
-quérir	-quer -quér -quér	-quér -quér	-quér	-quier -quér	-quér	-quis	-quer-ø	-quer-ø	-qu-i	-qu-i
-ntir -rtir	-nt -rt	-nt -rt	-nt -rt	-nt -rt	-nt -rt	-nti -rti	-nt-i -rt-i	-nt-i -rt-i	-nt-i -rt-i	-nt-i -rt-i
-vêtir	-vêt	-vêt	-vêt	-vêt	-vêt	-vêtu	-vêt-i	-vêt-i	-vêt-i	-vêt-i
-vrir -ffrir	-vr+e -ffr+e	-vr -ffr	-vr -ffr	-vr -ffr	-vr -ffr	-vert -ffert	-vr-i -ffr-i	-vr-i -ffr-i	-vr-i -ffr-i	-vr-i -ffr-i
-cueillir	-cueill+e	-cueill	-cueill	-cueill	-cueill	cueilli	cueill-e	-cueill-e	-cueill-i	-cueill-i
-saillir	-saill+e	-saill	-saill	-saill	-saill	-sailli	-saill-i	-saill-i	-saill-i	-saill-i
-faillir	-fau+x -faill	-faill	-faill	-	-faill	-failli	-faill-i	-faill-i	-faill-i	-faill-i
-bouillir	-bou -bouill	-bouill	-bouill	-bou -bouill	-bouill	bouilli	bouill-i	-bouill-i	-bouill-i	-bouill-i
-rair	-ra	-ra	-ra	-ra	-ra	-rai	-ra-i	-ra-i	-ra-i	-ra-i
-courir	-cour	-cour	-cour	-cour	-cour	-couru	-cour-ø	-cour-ø	-cour-u	-cour-u
mourir	mour mour	mour mour	mour	mour mour	mour	mort	mour-ø	mour-ø	mour-u	mour-u
-servir	-serv	-serv	-serv	-serv	-serv	-servi	-serv-i	-serv-i	-serv-i	-serv-i
-fuir	-fui	-fui	-fui	-fui	-fui	-fui	-fui-ø	-fui-ø	-fui-ø	-fui-ø
ouïr	oi	oi	oi	oi	oi	ouï	ouï	ouï	ouï	ouï
gésir	gi	-	gis	-	gis	-	-	-	-	-
-cevoir	-çoiv -cev	-çoiv -cev	-cev	-çoiv -cev	-cev	-çu	-cev-ø	-cev-ø	-ç-u	-ç-u
-voir	voi	-voi	-voi	-voi	-voi	-vu	-ver-ø	-ver-ø	-v-i	-v-i
pourvoir	pourvoi	pourvoi	pourvoi	pourvoi	pourvoi	pourvu	pourvoi-ø	pourvoi-ø	pourv-u	pourv-u
-savoir	sai sav	-sach	-sav	-sach	-sach	-su	-sau-ø	-sau-ø	-s-u	-s-u
-devoir	dev doiv	-dev	-dev	-dev	-dev	-dû	-dev-ø	-dev-ø	-d-u	-d-u

DEUXIEME PARTIE

ANALYSE

pouvoir	peuv+ pouv peuv	puiss	pouv	-	pouv	pu	pour-Ø	pour-Ø	p-u	p-u
-mouvoir	mouv meuv	-meuv -mouv	-mouv	-mouv	-mouv	-mû	-mouv-Ø	-mouv-Ø	-m-u	-m-u
-pleuvoir	pleu	-pleuv	-pleuv	-	-pleuv	-plu	-pleuv-Ø	-pleuv-Ø	-pl-u	-pl-u
falloir	fau	faill	faill	-	-	fallu	faud-Ø	faud-Ø	fall-u	fall-u
-seoir	-sie+d -sié	-sié	-sey	-	(-sey) -sé	-	-sié-Ø	-sié-Ø	-	-
-valoir	-vau+ -val	-vaill -val	-val	-vau -val	-val	-valu	-vaud-Ø	-vaud-Ø	-val-u	-val-u
-vouloir	-veu -voul -veul	-veuil -voul	-voul	(veuil) -veu -voul	-voul	-voulu	-voud-Ø	-voud-Ø	-voul-u	-voul-u
-asseoir	-assied -assey -assoï	-assey -assoï	-assey -assoï	-assey -assoï	-assey -assoï	-assis	-assié	-assié	-ass-i	-ass-i
surseoir	sursoi	sursoi	sursoi	sursoi	sursoi	sursis	surseoi	surseo	surs-i	surs-i
choir	choi	-	-	-	-	chu	choi	choi	ch-u	ch-u
échoir	échoi (éche) (éché)	échoi	-	-	éché	échu	échoi (écher)	échoi (écher)	éch-u	éch-u
déchoir	déchoi (déche)	déchoi	-	-	-	déchu	déchoi (décher)	déchoi (décher)	déch-u	déch-u
-prendre	-prend -pren -prenn	-prenn -pren	-pren	-prend -pren	-pren	-pris	-prend	-prend	-pr-i	-pr-i
-ndre -rdre -mpre -fout	-nd -rd -mp -fout	-nd -rd -mp -fout	-nd -rd -mp -fout	-nd -rd -mp -fout	-nd -rd -mp -fout	-ndu -rdu -mpu -foutu	-nd -rd -mp -fout	-nd -rd -mp -fout	-nd-i -rd-i -mp-i -	-nd-i -rd-i -mp-i -
-battre	-bat -batt -batt	-batt	-batt	-bat -batt	-batt	-battu	-batt	-batt	-batt-i	-batt-i
-mettre	-met -mett -mett	-mett	-mett	-met -mett	-mett	-mis	-mett	-mett	-m-i	-m-i
-indre	-in -ign -ign	-ign	-ign	-in -ign	-ign	-int	-ind	-ind	-ign-i	-ign-i
-vaincre	-vain -vainqu -vainqu	-vainqu	-vainqu	-vain -vainqu	-vainqu	-vaincu	-vainc-Ø	-vain-Ø	-vainqu-i	-vainqu-i

-faire	-fais -faites	-fass	-fais	-fais -faites	-fais	-fait	-fe-ø	-fe-ø	-f-i	-f-i
-raire	-rai	-rai	-rai	-rai	-rai	-rait	-rai	-rai	-	-
-plaire	-plais (3s + ^)	-plais	-plais	-plais	-plais	-plu	-plai	-plai	pl-u	pl-u
-taire	-tais	-tais	-tais	-tais	-tais	tu (inv)	-tai	-tai	-t-u	-t-u
repâitre	repaiss (3s + ^)	repaiss	repaiss	repaiss	repaiss	repu	repait	repait	rep-u	rep-u
paître	paiss (3s + ^)	paiss	paiss	paiss	paiss	(pu)	paît	paît	-	-
-naître	-naiss (3s + ^)	-naiss	-naiss	-naiss	-naiss	-né	-naît	-naît	-naqu-i	-naqu-i
-aître	-aiss (3s + ^)	-aiss	-aiss	-aiss	-aiss	-u	-aît	-aît	-u	-u
-croître	-croiss	-croiss	-croiss	-croiss	-croiss	-crû	-croît	-croît	-cr^-u	-cr^-u
-croire	-croi	-croi	-croi	-croi	-croi	-cru	-croi	-croi	-cr-u	-cr-u
-boire	-boi -buv -boiv	-boiv -buv -boiv	-buv	-boi -buv	-buv	-bu	-boi	-boi	-b-u	-b-u
-clore	-clos (3s + ^) (pas 1+2p)	-clos	-	-clos (seul2s)	-clos	-clos	-clo	-clo	-	-
-clure	-clu	-clu	-clu	-clu	-clu	-clu	-clu	-clu	-clu-ø	-clu-ø
-soudre	-sou -solv -solv	-solv	-solv	-sou -solv	-solv	-sous	-soud	-soud	-	-
-coudre	-coud -cous -cous	-cous	-cous	-coud -cous	-cous	-cousu	-coud	-coud	-cous-i	-cous-i
-moudre	-moud -moul -moul	-moul	-moul	-moud -moul	-moul	-moulu	-moud	-moud	-moul-u	-moul-u
-suivre	-suiv	-suiv	-suiv	-suiv	-suiv	-suivi	-suiv	-suiv	-suiv-i	-suiv-i
-vivre	-viv	-viv	-viv	-viv	-viv	-vécu	-viv	-viv	-véc-u	-véc-u
-lire	-lis	-lis	-lis	-lis	-lis	-lu	-li	-li	-l-u	-l-u
-dire	-dis (2p-dites)	-dis	-dis	-dis 2p dites	-dis	-dit	-di	-di	-d-i	-d-i
-crire	-criv	-criv	-criv	-criv	-criv	-crit	-cri	-cri	criv-i	criv-i
-rire	-ri	-ri	-ri	-ri	-ri	-ri	-ri	-ri	-ri-ø	-ri-ø
-fire	-fis	-fis	-fis	-fis	-fis	-fit	-fi	-fi	-f-i	-f-i
-uire	-uis	-uis	-uis	-uis	-uis	-uit	-ui	-ui	-uis-i	-uis-i

3.3.3.2 Eléments minimaux

La notion d'élément minimal se répète maintes fois dans les explications qui suivent. Il s'agit de la partie d'un verbe que l'on pourrait considérer soit comme élément unique, soit comme facteur commun à un groupe de verbes qu'on veut soumettre au même traitement. Prenons, par exemple, les verbes que Bescherelle range dans son tableau 23. Nous voyons ici qu'ils ont en commun l'élément *-en-* et nous pouvons donc identifier ainsi ce groupe, c'est-à-dire *tenir* et *venir* et tous leurs composés. En retenant la marque de l'infinitif nous pouvons élaborer une règle applicable à l'ensemble de la langue, ceci de la façon suivante: "Tout verbe en *-en-* ayant *-ir* comme marque d'infinitif se conjugue ainsi...." Il en est de même pour les trois tableaux 57 *peindre*, 58 *joindre* et 59 *craindre*. Ils peuvent être traités de la même manière, donc l'élément minimal ou facteur commun sera *-ind-*. Les tableaux ci-dessus sont rangés selon ces éléments, rattachés à leurs marques d'infinitif respectives.

Dans la troisième partie de ce travail nous élaborerons une méthode pour utiliser cette notion sur ordinateur.

3.3.3.3 Les verbes en *-ir* des groupes 2 et 3

Comment identifier les verbes de la troisième conjugaison? Tous ceux qui se terminent en *-re* ou en *-oir* y appartiennent automatiquement mais les verbes en *-ir* chevauchent la 2^e et la 3^e conjugaisons et nous n'avons pas de méthode immédiate et évidente pour les différencier les uns des autres. La 2^e conjugaison

comporte à peu près 320 verbes, tous en -ir sauf deux (haïr et amuir). La 3^e conjugaison ne possède que 91 verbes en -ir et il nous semble donc plus efficace de les séparer de ceux de la 2^e conjugaison que l'inverse. Puisque nous ne voulons pas une simple liste comportant ces 91 verbes, nous allons chercher les éléments minimaux qui les distinguent de ceux de la 2^e conjugaison. Faute d'un seul élément minimal pour le groupe entier, nous sommes obligés d'en chercher un pour chaque sous-groupe.

tenir, venir et leurs composés	-en-
acquérir et les autres de son type	-quér-
sentir, mentir et leurs composés	-nt-
partir, sortir et leurs composés ¹⁵	-rt-
vêtir et ses composés	-vét-
ouvrir, couvrir et leurs composés	-vr-
offrir, souffrir	-ffr-
cueillir et ses composés	-cueill-
assaillir et les autres de son type ¹⁶	-saill-
faillir et défailir	-faill-
bouillir et ses composés	-bouill-
dormir et ses composés	-rm-
courir et ses composés	-cour-
mourir	-mour-
servir et ses composés	-rv-
fuir et ses composés	-fui-
ouïr	-oui-
gésir	-gés-

Passons maintenant les verbes de la 2^e conjugaison à travers de la "grille" que nous avons établie ci-dessus. Heureusement c'est un travail qu'on peut réaliser par ordinateur¹⁷. Nous allons faire une liste de tous les 327 verbes (selon Bescherelle) appartenant à la 2^e conjugaison (en omettant, pour ce test, haïr, amuir, ainsi que maudire) et nous allons vérifier à l'aide de

¹⁵ sortir et ressortir peuvent se conjuguer soit d'après la 2^e soit la 3^e conjugaison. Les problèmes sémantiques de ce genre ne sont pas traités par le programme. Il conjuguent les deux verbes d'après la 3^e conjugaison.

¹⁶ saillir à l'emploi transitif se conjugue d'après la 2^e conjugaison.

¹⁷ Voir la troisième partie.

notre "grille" s'il y a des verbes qui sont retenus.

Résultats

L'élément -nt identifiera *alentir, ralentir et retentir*

L'élément -rt identifiera *impartir, répartir, tartir,*
sortir, ressortir,
amortir,
assortir et ses composés
10 verbes en -ertir

L'élément -vr identifiera *appauvrir*

L'élément -rv identifiera *asservir*

L'élément -rm identifiera *affermer, raffermir*
saillir peut être soit 2^e soit 3^e conjugaison.

Puisque une bonne trentaine de verbes de la 2^e conjugaison ont été retenus, il nous faut donc en premier lieu exclure les verbes qui se confondent avec ceux de la 3^e conjugaison. L'élément -ert- retiendra les dix verbes en -ertir, de même que -lent- pour *alentir* et *ralentir*. L'élément -dorm- au lieu de -rm- dans la liste pour la 3^e conjugaison exclura *affermer* et *raffermer*, ainsi que -ouvr au lieu de -vr- pour éviter la confusion avec *appauvrir*. Pour le reste il faut tout simplement les lister les uns après les autres, et notre liste *type_finir* des verbes qui peuvent se confondre avec ceux du 3^e groupe comportera les éléments suivants:

type_finir(ert,lent,retent,amort,assort,asserv,impart,répart,tart,
gueill,vieill,jaill)

Et maintenant notre liste pour les verbes de la 3^e conjugaison est valable:

troisième_groupe(en,quér,bouill,mour,gés,ouï,ent,rt,vêt,ouvr,ffr,
ill,dorm, cour,serv)

Bien que nous voulions retenir pour la troisième partie toute explication qui concerne l'implémentation, nous présenterons ici l'algorithme:

- a. Le verbe appartient-il à la liste *type_finir*?
Si oui - conjuguer d'après la 2^e conjugaison
Si non - aller à b.
- b. Le verbe appartient-il à la liste *sentir_etc*?
Si oui - conjuguer d'après la 3^e conjugaison
Si non - conjuguer d'après la 2^e conjugaison

La liste de tous les verbes de la 2^e conjugaison avec les verbes modèles de la 3^e conjugaison que nous avons utilisé dans ce test ainsi que les résultats peuvent être consultés dans l'annexe F.

3.3.3.4 Le regroupement dans la 3^e conjugaison.

M. Gertner préfère parler des verbes "irrégulièrement réguliers" plutôt que des verbes irréguliers, mais lui aussi, face à quelques unes des particularités qui sont maintenant sous nos yeux, il est obligé d'utiliser la terminologie traditionnelle. Il est certes possible de ranger les quelque 370 verbes du 3^e groupe en petits sous-groupes comportant des verbes qui se conjuguent de la même façon. Mais le problème reste que ces sous-groupes ne fonctionnent pas pour tous les temps et personnes - ils se chevauchent mais ils sont rarement conformes aux mêmes schémas.

La plupart des particularités concernent l'indicatif présent et les terminaisons muettes¹⁰. Phonétiquement il est clair que l'alternance *mour/meur* dans *nous mourons / je meurs* est due au

¹⁰ Bien qu'elles soient parfois réalisées dans la langue, si elles sont suivies d'une voyelle, dans notre analyse hors contexte ces terminaisons sont tout à fait muettes.

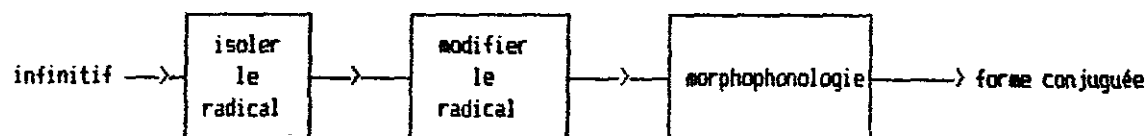
déplacement de l'accent et que la suppression de *v* dans *serv/ser* (*nous servons / je sers*) est l'effet de la terminaison muette. Nous n'avons pas la possibilité dans cette analyse de vérifier où tombe l'accentuation. Nous pouvons cependant établir une condition terminaison muette / non-muette: si la terminaison appartient à la liste {s,t,x} elle est muette, sinon elle est "non-muette". En effet nous avons déjà en main une méthode plus facile, la méthode de calcul établie ci-dessus. Les personnes (*je, tu, il*) qui prennent les terminaisons dans notre liste ont toutes un numéro inférieur à 3, donc la condition $PERS < 3$ suffira. Il y a des cas où il faut inclure la 3^e personne plurielle et là nous pouvons utiliser la condition $PERS < 4$.

La suppression des éléments ne pose pas de problèmes insurmontables. Un petit nombre de règles générera les formes correctes. Toutefois il y a des cas plus délicats tel que l'alternance vocalique dans *mour/meur* ou *dev/doiv*, les changements du radical dans *joind/joign* ou *ten/tien* et surtout le remplacement entier du radical dans *aller (all/va/i)* ou *naft/naqu*. Ces formes exigent un traitement à part.

Il est évident que dériver le radical *i* à partir d'*all* en postulant une forme sous-jacente quelconque ne serait ni productif, parce qu'il ne s'applique que dans ce seul cas, ni scientifique, parce que historiquement ils sont de deux origines différentes. Il vaudrait mieux effectuer un simple changement de l'un à l'autre dans des contextes spécifiques et, comme nous

l'avons notés ci-dessus, nous procéderons ainsi.

Nous allons traiter de ces deux phénomènes de changement ("modification") et de suppression ("morphophonologie") séparément, ce qui nous donne le schéma suivant:



Séparons donc les radicaux que nous pouvons traiter "facilement" de ceux qui exigent un traitement plus compliqué:

Liste 1 représentant 201 verbes

-ent, -rt, -êt, -ouvr, -ffr, -cueill, -saill, -défaill, -dorm, -cour, -serv, -fui, -nd, -rd, -mp, -fout, -batt, -suiv, -mett, -rai, -tai, -plai, -croi, -viv, -lui, -ui, -cri, -di, -ri, -li, -fi, -ci

Liste 2 représentant 169 verbes (+ avoir et être)

all, -en, -quer, -faill, -bouill, -mour, -ouï, -gés, -cev, -pourv, -v, -sav, -dev, -pouv, -mouv, -pleuv, -fall, -val, -voul, -asse, -surse, -se, -ch, -éch, -déch, -prend, -ind, -vainc, -fai, -rai, -repaît, -paît, -naît, -aît, -croît, -boi, -soud, -coud, -moud

3.3.3.4.1 Les cas "faciles"

La première liste doit être-elle même séparée en deux listes parce que quelques radicaux exigent une petite modification avant de passer au traitement morphophonologique:

Liste 1a. aucune modification - 110 verbes

-ent, -rt, -êt, -ouvr, -ffr, -cueill, -saill, -dorm, -cour, -serv, -fui, -nd, -rd, -mp, -fout, -batt, -suiv

Liste 1b. une modification quelconque - 91 verbes

-mett, -rai, -tai, -plai, -croi, -viv, -lui, -ui, -cri, -di, -ri, -li, -fi, -ci, -clo

DEUXIEME PARTIE

ANALYSE

La liste 1a comporte tous les verbes en *ir* sauf *-en, -quer, faillir*¹⁹, *-bouill, mour, gés, ouï*. Nous sommes donc obligés de les exclure d'abord de la liste *sentir_etc*. En les excluant, on obtient la liste suivante de radicaux qu'on peut laisser sans traitement:

sentir_etc(ent,rt,vêt,ouvr,ffr,ill,dorm,cour,serv)

Pour les verbes en *-re* il faut séparer ceux dans la liste 1b pour un traitement à part. Il s'agit des cas tel que, par exemple, celui de *mettre* qui se conjugue comme *battre*, à la différence qu'au PS et SI le radical est *m-*, ou bien d'*écrire* pour lequel nous voulons établir une "forme sous-jacente" *écriv-*.

Pour tous il y a un changement de radical pour les temps "littéraires", le PS et le SI (TEMPS=8 et TEMPS=9, selon notre numérotation). Deux conditions donc suffiront: TEMPS<8 ou TEMPS>7. Selon la condition TEMPS<8 dans les cas des radicaux *lui, -ui, -ci, -fi, -di, -li, -plai, tai, -clo* il s'agit d'ajouter un *s*. A *-cri* et ses composés on va ajouter un *v*. Les autres (*-viv, -croi, -ri, -rai, -mett*) passeront sans traitement. Pour bien comprendre ce qui se passe ici voyons le schéma suivant du traitement entier:

¹⁹ Seulement *faillir*, puisque *défaillir* se conjugue d'après *saillir*.

condition TEMPS<8

INF ----> RAD1---->RAD2 ----> MORPHOPHONOLOGIE --> FORME FLECHIE

dire ----> di ----> dis ----> dis + s --> dis + s--> --> dis
dis + ons -----> --> disons

Selon la condition *TEMPS>7* il y a d'avantage de particularités et nous établirons deux listes afin d'effectuer des simples changements:

LISTE_A		LISTE_B
ui	-->	uis
lui	-->	lui
ci	-->	cis
fi	-->	f
fri	-->	fr
cri	-->	criv
di	-->	d
li	-->	l
viv	-->	véc
croi	-->	cr
ri	-->	ri
rai	-->	Ø
plai	-->	pl
tai	-->	t
mett	-->	m

Les autres verbes en *-re* passeront à la section MORPHOPHONOLOGIE sans traitement.

3.3.3.4.2 Les cas plus difficiles

Des verbes tels *tenir/venir, acquérir, prendre* etc et leurs composés sont des cas plus difficiles en ce qu'ils possèdent plusieurs radicaux différents:

	1 tien		1 sai		1 prend
	2 ten		2 sav		2 pren
tenir --->	3 tienn	savoir --->	3 sach	prend --->	3 prenn
	4 tiend		4 sau		4 pr
	5 tin		5 s		

Dans les cas de *tenir* et *savoir* il y a cinq radicaux soit quatre en sus de celui de l'infinitif. Cependant ce n'est pas la simple prolifération des radicaux différents qui crée des problèmes avec ces verbes, mais aussile fait que les conditions de changements des radicaux diffèrent presque pour chaque verbe. Par exemple, *tenir* et *savoir*, bien qu'ils aient le même nombre de variants de radical et cela pour les mêmes temps, il y a un détail qui nous empêche de les traiter ensemble. Le radical du subjonctif présent de *tenir* est le même que pour la troisième personne du pluriel de l'indicatif présent. Il change de la même manière, selon l'alternance muet/non-muet de la terminaison, que le radical de l'indicatif présent. Par contre celui de *savoir* change pour le subjonctif et puis reste le même, indépendamment de la terminaison:

INDICATIF 3pl	tienn -e	sav -ent
	tienn -e	sach -e
	tienn -es	sach -es
SUBJONCTIF	tienn -e	sach -e
	ten -ions	sach -ions
	ten -iez	sach -iez
	tienn -ent	sach -ent

Toutefois nous trouverons d'autres verbes que nous pouvons traiter de la même manière que *tenir/venir*. Voyons lesquels à l'aide du tableau à la page suivante:

	IP			SP		II	PP	FS+CD	PS+SI
	<3	3	>3	<4	>3				
1.									
mour	meur	meur	mour	meur	mour	mour	mort	mour	mour
-cev	çoiv	çoiv	cev	çoiv	cev	cev	cu	cev	c
-dev	doiv	doiv	dev	doiv	dev	dev	dû	dev	d
-mouv	meuv	meuv	mouv	meuv	mouv	mouv	mû	mouv	m
-boi	boiv	boiv	buv	boiv	buv	buv	bu	boi	b
paît	paiss	paiss	paiss	paiss	paiss	paiss	pu	paît	-
repaît	repaiss	repaiss	repaiss	repaiss	repaiss	repaiss	repu	repaît	rep
-pouv	pourvoi	pourvoi	pourvoi	pourvoi	pourvoi	pourvoi	pourvu	pourvoi	v
-v	voi	voi	voi	voi	voi	sursoi	v	ver	ver
-surse	sursoi	sursoi	sursoi	sursoi	sursoi	sursoi	sursis	surseoi	sursis
-naît	naiss	naiss	naiss	naiss	naiss	naiss	né	naît	naqu
-aît	aiss	aiss	aiss	aiss	aiss	aiss	ttu	aît	tt
-croît	croiss	croiss	croiss	croiss	croiss	croiss	crû	croît	cr^
2.									
-bouill	bou	bouill	boill	bouill	boill	bouill	bouill	bouill	bouill
-prend	prend	prenn	pren	prenn	pren	pren	pris	prend	pr
-ind	in	ign	ign	ign	ign	ign	int	ind	ign
-vainc	vainc	vainqu	vainqu	vainqu	vainqu	vainqu	vaincu	vainc	vainqu
résoud	résou	résolv	résolv	résolv	résolv	résolv	résolu	résoud	résol
-coud	coud	cous	cous	cous	cous	cous	cousu	coud	cous
-moud	moud	moul	moul	moul	moul	moul	moul	moud	moul
-asse	assied	assei	assei	assei	assei	assei	assis	assied	ass
-soud	asso	solv	solv	solv	solv	solv	sous,oute	soud	-
-en	ien	ien	en	ien	en	en	enu	iend	in
-quér	quér	quér	quér	quér	quér	quér	quis	quér	qu
-rai	rai	rai	ray	rai	ray	ray	rait	rai	ø
3.									
all	va	va	all	aill	all	all	allé	i	all
-sav	sai	sav	sav	sach	sach	sav	su	sau	s
pouv	peuv	peuv	pouv	puiss	puiss	pouv	pu	pour	p
-val	vau	val	val	vaill	val	val	valu	vaud	val
voul	veul	veul	voul	veuil	voul	voul	voulu	voud	voul
-fai	fais	fais	fais	fass	fass	fais	fait	fe	f
avoir	av	av	av	ai	ai	av	eu	au	e
être	es	es	es	soi	soi	ét	été	se	f
faill	-	-	-	-	-	-	failli	faill	faill
fall	fau	-	-	faill	-	fall	fallu	faud	fall
-pleuv	pleuv	-	-	pleuv	-	pleuv	plu	pleuv	pl

† aussi puis

†† ex. élément minimal = 0. Soit l'infinitif *conn-aît-re*, RAD du PS = conn, élém. minia = 0

††† PR gis, ét, ay

Particularités de l'impératif

être - sois, soyons, soyez

avoir - aie ayons ayez

vouloir - veux/veuille, voulons, voulez/veuillez

particularités d'être - RAD = ét- au PR et II

3.3.3.5 Morphophonologie

3.3.3.5.1 L'indicatif présent

Les radicaux à consonne finale.

On peut régulariser certaines modifications parce qu'elles se répètent de la même manière pour plusieurs verbes. Il serait bien sûr plus économique d'établir quelques règles aussi générales que possibles. Ainsi pourrait-on traiter plusieurs phénomènes ensemble plutôt que de les prendre en considération séparément. Dans le tableau 3.3.3.1 nous avons réduit le nombre potentiel de radicaux en présupposant quelques formes "sous-jacentes", dans le cas, par exemple, d'*écrire* et ses composés, la forme abstraite -*criv*. Au lieu de deux radicaux pour chaque verbe:

j' écri	-s	je ser	-s
tu écri	-s	tu ser	-s
il écri	-t	il ser	-t
nous écriv	-ons	nous serv	-ons
vous écriv	-ez	vous serv	-ez
ils écriv	-ent	ils serv	-ent

on en aura qu'un seul *écriv* plus une règle $v \rightarrow \emptyset / __\# \{s, t\}$ (où # représente une frontière de morphème). Cette approche est valable aussi pour *servir*, *recevoir*, *mouvoir*, *boire*, *suivre* et *vivre*, et si nous ajoutons la terminaison *x* à la liste - $\{s, x, t\}$ - également pour *pouvoir*.

Bien que *s*, *x*, et *t* provoquent pour la plupart du temps les mêmes suppressions, *t* a quelques particularités qui nécessitent un traitement à part pour cette terminaison. Voyons d'abord un tableau comportant toutes les suppressions que nous voulons effectuer dans cette partie du programme:

CORPUS

Radicaux se terminant en une consonne

A. Dernière cons. supprimée

-sent	+s	---	ɥ	---	sens
	+t	---		---	sent
-fout	+s	---	ɥ	---	fous
	+t	---		---	fout
-dorm	+s	---	ɑ	---	dors
	+t	---		---	dort
-serv	+s	---	ʏ	---	sers
	+t	---		---	sert
-goiv	+s	---	ʏ	---	gois
	+t	---		---	goit
peuv	+x	---	ʏ	---	peux
	+t	---		---	peut
-meuv	+s	---	ʏ	---	meus
	+t	---		---	meut
-boiv	+s	---	ʏ	---	bois
	+t	---		---	boit
-suiv	+s	---	ʏ	---	suis
	+t	---		---	suit
-viv	+s	---	ʏ	---	vis
	+t	---		---	vit
-criv	+s	---	ʏ	---	cris
	+t	---		---	crit
-lis	+s	---	ɛ	---	lis
	+t	---		---	lit
-dis	+s	---	ɛ	---	dis
	+t	---		---	dit
-fis	+s	---	ɛ	---	fis
	+t	---		---	fit
-uis	+s	---	ɛ	---	uis
	+t	---		---	uit
veul	+x	---	ɥ	---	veux
	+t	---		---	veut
-ind	+s	---	ɑ	---	ins
	+t	---		---	int

B. #s et #t traités différemment

-clos	+s	---	ɛ	-----	-clos
	+t	---	s	--> ^	-clôt
-batt	+s	---	ɥ	-----	-bats
	+t	---	ɥɥ	-----	-bat
-iss	+s	---	ɛɛ	-----	-is
	+t	---	ss	--> ^	-ît
croiss	+s	---	ɛɛ	-----	croïs
	+t	---	ɛɛ	-----	croît

C. Terminaison supprimée

	+s----->	-sieds
-sied	+t-----> ɥ ----->	-sied
	+s----->	-rends
-rend	+t-----> ɥ ----->	-rend
	+s----->	-vêts
-vét	+t-----> ɥ ----->	-vét
	+s----->	-vaincs
-vainc	+t-----> ɥ ----->	-vainc
	+s----->	-couds
-coud	+t-----> ɥ ----->	-coud

D. Pas de changement

-ien	+s	-----	-iens
	+t	-----	-ient
-romp	+s	-----	-romps
	+t	-----	-rompt
-cour	+s	-----	-cours
	+t	-----	-court
-meur	+s	-----	-meurs
	+t	-----	-meurt

Formalisation

A. suppression de la dernière consonne du radical

{v,s,l,t}	---	∅ / ___ # {s,x,t} ¹
m	---	∅ / C___ # {s,x,t}
d	---	∅ / in___ # {s,x,t}

¹ Le symbole # représente une frontière de morphème. Deux ## = frontière de mot.

DEUXIEME PARTIE

ANALYSE

Etant donné que le seul cas de *m* en position finale est *-dorm* nous pouvons supprimer la 3^e règle et ajouter *m* à la liste de consonnes finales de la 1^e règle:

$\{v,s,l,t,m\} \rightarrow \emptyset / __\# \{s,x,t\}$
 $d \rightarrow \emptyset / in__\# \{s,x,t\}$

- B. #s et #t traités différemment
- C. suppression de la terminaison t

$t \rightarrow \emptyset / (d,c,t)\# __\$

Il est bien évident dans le cas du groupe C que, pour la plupart, c'est la terminaison et non la dernière consonne qui est supprimée, mais dans le cas de *vêtir* on peut hésiter. On pourrait se demander s'il n'appartient plutôt au groupe B, où il faut supprimer deux consonnes finales. Dans ce dernier cas on doit traiter les radicaux en deux passes afin de déterminer s'il n'y a pas encore une consonne à supprimer. Nous établirons une série de règles à deux niveaux et la groupe C sera traité au deuxième niveau. *Vêtir* sera donc exclu de la première passe et traité dans la deuxième, où la terminaison *t* sera supprimé. *Battre*, *mettre* etc perdra une de leurs lettres finales à la première passe et la terminaison *t* à la deuxième:

Première passe

$\{v,s,l,t,m\} \rightarrow \emptyset / -[v\emptyset]__\# \{s,x,t\}$
 $d \rightarrow \emptyset / in__\# \{s,x,t\}$

Deuxième passe

$t \rightarrow \emptyset / __\{d,c,t\}\#$
 $s \rightarrow \emptyset / i__\#t$
 $s \rightarrow \emptyset / __\# \{s,t\}$

Et nous pouvons ajouter ici deux particularités de la troisième personne du singulier et une particularité qui concerne toutes

les trois personnes - les cas de *plaire* et ses composés (*plaît*, *déplaît* etc), de *clorre* (*clôt*, mais pas ses composés, donc l'utilisation dans la règle de ## pour indiquer une frontière de mot), et de *croître* (*crois*, *crois*, *croît*, mais pas ses composés, *décrois*, *décroît*, *accrois* etc).

```
i ----> i/pla___#t
o ----> o/##cl___#t
is----> i/##cro_#(s,t)
```

3.3.3.5.3 Residu

Quelques conjugaisons exigent des règles tout à fait *ad hoc*. Nous les laisserons jusqu'à la fin où ils seront traités comme suit:

```
av + s ----> ai
av + s ----> as
av + t ----> a
êt + s ----> suis
êt + s ----> es
êt + s ----> est
```

```
fais + ent ----> font
es + ent ----> sont
va + ent ----> vont
av + ent ----> ont
```

```
ai + ions ----> ayons
ai + iez ----> ayez
```

```
va + s ----> vais
va + s ----> vas
va + t ----> va
```

```
-fais + ez ----> -faites
es + ez ----> êtes
dis + ez ----> dites
redis + ez ----> redites
```

3.3.3.6 Les voyelles thématiques

CORPUS

-en	-->	-in	+ Ø	(tenir, venir etc)
-quer	-->	-qui	+ Ø	(acquérir etc)
-fui	-->	-fui	+ Ø	(fuir etc)
oui	-->	oui	+ Ø	(ouir)
-clu	-->	-clu	+ Ø	(conclure etc)
-ri	-->	-ri	+ Ø	(rire etc)
-di	-->	-di	+ Ø	(dire etc)
-lui	-->	-lui	+ Ø	(luire, reluire)
-fi	-->	-fi	+ Ø	(confire, suffire etc)
circonci	-->	circonci	+ Ø	(circoncire)
-nt	-->	-nt	+ i	(sentir etc)
-rt	-->	-rt	+ i	(partir etc)
-êt	-->	-êt	+ i	(vêtir etc)
-vr	-->	-vr	+ i	(couvrir etc)
-fr	-->	-fr	+ i	(souffrir etc)
-cueill	-->	-cueill	+ i	
-saill	-->	-saill	+ i	
-faill	-->	-faill	+ i	
-bouill	-->	-bouill	+ i	
-rm	-->	-rm	+ i	(dormir etc)
-rv	-->	-rv	+ i	(servir etc)
-v	-->	-v	+ i	(voir etc)
-asse	-->	-ass	+ i	(asseoir etc)
surse	-->	-surs	+ i	
-prend	-->	-pr	+ i	(prendre etc)
-nd	-->	-nd	+ i	(rendre etc)
-rd	-->	-rd	+ i	(perdre etc)
-mp	-->	-mp	+ i	(rompre etc)
-batt	-->	-batt	+ i	
-mett	-->	-m	+ i	(mettre etc)
-ind	-->	-ign	+ i	(craindre, peindre, joindre etc)
-vainc	-->	-vainqu	+ i	
-fai	-->	-f	+ i	
-naît	-->	-naqu	+ i	(naître, renaître)
-coud	-->	cous	+ i	(coudre etc)
-suiv	-->	suiv	+ i	
-cri	-->	-criv	+ i	(écrire etc)
-ui	-->	-uis	+ i	(cuire, conduire, détruire, nuire etc)
-cour	-->	-cour	+ u	
-mour	-->	-mour	+ u	
-cev	-->	-c	+ u	(recevoir etc)
-pouv	-->	-pouv	+ u	
-sav	-->	-s	+ u	(savoir etc)
-dev	-->	-d	+ u	(devoir etc)
pouv	-->	p	+ u	(pouvoir)
-mouv	-->	-m	+ u	(mouvoir)
-pleuv	-->	-pl	+ u	
fall	-->	fall	+ u	
-val	-->	-val	+ u	(valoir etc)
voul	-->	voul	+ u	
-ch	-->	ch	+ u	(choir etc)
-plai	-->	-pl	+ u	
tai	-->	-t	+ u	
repait	-->	-rep	+ u	(repâitre)
-connaît	-->	-conn	+ u	
-paraît	-->	-par	+ u	
-croît	-->	-cr^	+ u	(croître etc)
-croi	-->	-cr	+ u	(croire etc)
-boi	-->	-b	+ u	(boire etc)
résoud	-->	résol	+ u	(résoudre)
-moud	-->	-moul	+ u	(moudre etc)
-viv	-->	-véc	+ u	(vivre etc)
-li	-->	-l	+ u	(lire etc)
être	-->	f	+ u	
avoir	-->	e	+ u	

ni	PS	ni	SI
absoudre,			
dissoudre,			
seoir,			
traire etc,			
paître,			
clore etc,			
gésir,			

Nous allons faire trois listes des dernières lettres des radicaux:

Tableau selon la dernière lettre

prennent_i	prennent_u	prennent_ø
t <----->	t	
r <----->	r	
l <----->	l	
m <----->	m	
v <----->	v	
s <----->	s	
d <----->	d	
p <----->	p	
n <----->	n <----->	n
u <----->		u
f <----->	f	
c <----->	c	
	h	
	^	
	b	
	e	
		i

Nous voyons ici que presque tous les membres des deux premières listes sont identiques. Dans ce cas il nous faut prendre en considération les deux dernières lettres des radicaux:

Table selon les deux dernières lettres

prennent_i	prennent_u	prennent_0
et		
nt		
rt		
tt		
	t	
	ar	
	cr	
fr		
pr		
	ur	
vr		
	al	
	l	
ll <----->	ll	
	ol	
	pl	
	ul	
f <----->	f	
m <----->	m	
rm		
iv		
rv <----->	rv	
v		
	s	
is		
rs		
ss		
us		
	d	
nd		
rd		
	p	
	ep	
mp		
gn		
	nn	in
qu		
	c	
	éc	
nc		
	ch	
	b	
	r^	
	e	
		ui
		ri
		di
		lu

En ayant toujours présent à l'esprit le tableau ci-dessus, commençons par rédiger nos listes de conditions. Les radicaux ayant t pour lettre finale peuvent être différenciés les uns des autres soit en incluant et,nt,rt,tt dans la liste prennent_i,

soit en éliminant *taire* (PS t+u+PERS) avant de parcourir la liste. Prenons la deuxième alternative. De même que pour les *s* et les *d* terminaux, nous allons éliminer *savoir* et *devoir*, et pour *f* et *m*, *être* (*fus*) et *mouvoir* (*mus*). Pour les *v* en position finale nous exclurons *pouvoir*. Les *l* en position finale ne posent pas de problèmes puisqu'en spécifiant les trois lettres finales *ill* on inclut tous les radicaux prenant *i*. Si nous incluons *mp* dans la liste *prennent_i* nous pouvons supprimer *p* et *ep* de la deuxième liste. Les *r* sont plus difficiles à traiter et nous serons obligés d'inclure *fr*, *pr* et *vr*, dans la liste *prennent_i*. Finalement, nous ajouterons *gn* et *nc* à cette même liste et nous supprimerons *nn* ainsi que *c* et *éc*.

Nos listes sont maintenant établies de la manière suivante:

1. A éliminer avant de parcourir la liste *prennent_i*:
(*taire,savoir,devoir,être,mouvoir,pourv*)
2. *prennent_i* {*t fr pr vr ill f m v s d mp gn qu nc*}
3. *prennent_ø* {*in i u i*}

Nous n'avons pas besoin de la liste *prennent_u* parce que, par défaut, elle ne contient que ce qui n'est pas dans les listes 2 et 3. La liste 3 servira également pour la deuxième conjugaison, puisque tous ses radicaux se terminent en *i*, à part *haïr*, et alors il suffira d'y ajouter *i*.

3.3.3.6.1 Formalisation

Procédons maintenant à une formalisation plus compacte de notre analyse. Puisque nous ne comparons pas deux listes l'ordre des éléments dans les listes n'est pas très important. Nous suivrons néanmoins ici aussi notre convention de mettre les éléments les

plus spécifiques en tête, devant les éléments plus généraux.

1. V ----> a/___«er» [+ TEMPS > 7]

2. V ----> i / $\left[\begin{array}{l} \text{-(taire,savoir,devoir,\text{être},\text{avoir,pouvoir})} \\ \text{+(ill,vr,fr,pr,mp,gn,qu,nc,m,v,s,f,d,t)} \end{array} \right] \text{ --- } [+\text{TEMPS}>7]$

3. V ----> Ø/(in,i,u,i)___ [+ TEMPS>7]

4. V ----> u/___ [+ TEMPS>7]

Et voyons maintenant cette formalisation traduite en sous-système

PROLOG:

```

affecter_voy_thém(TEMPS,RAD,PERS,er,a):- TEMPS>7, asserta(v("a"),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,i):-
    TEMPS>7,
    rad(RAD_ORIG),/* quelle était la forme du radical au début du traitement */
    exclure(INIT,ELEM,RAD_ORIG,[tai,sav,dev,ét,mouv,pourv]),
    not(concat(INIT,ELEM,R)),
    appartenance(INIT2,ELEM2,[ill,vr,fr,pr,mp,gn,qu,nc,m,v,s,f,d,t],RAD),
    asserta(v("i")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,"Ø"):-
    TEMPS>7,
    appartenance(INIT,ELEM,[in,i,i,u],RAD), asserta(v("")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,"u"):- asserta(v("u")).

```

Explication

La question **TEMPS>7** vérifie qu'il s'agit bien du passé simple ou du subjonctif imparfait; **rad(RAD_ORIG)**, appelle le radical de l'infinitif qu'on a sauvgardé au début du traitement; **exclure** essaie d'unifier ce radical avec un des éléments de la liste (le radical peut être un composé d'un élément - savoir ou resavoir, par exemple) et, si cette unification a lieu, **not** exclura le radical; s'il n'a pas lieu, **appartenance** va essayer d'unifier le radical avec un élément dans la liste, et s'il réussit, la voy-

elle *i* sera affecté au radical. Asserta sauvegarde la voyelle pour un traitement ultérieur.

Dans le cas d'exclusion par *exclure* dans la première clause - ce qui sera le cas avec *taire*, par exemple - le radical *t* du PS ou SI passera à la prochaine clause qui, après avoir encore vérifié qu'il s'agit du PS ou du SI, tentera de l'unifier avec un élément de notre liste *prennent_Ø*. Echouant il passera à la prochaine clause où la voyelle *u* y sera affectée.

Dans le cas de *tenir* (RAD du PS = *tin*), le radical échouera dans la première clause mais il sera accepté dans la deuxième puisque la liste contient l'élément *in*, et une "voyelle" *Ø* y sera affectée (représenté par "").

4 COMMENT SAVOIR OÙ FIXER LES LIMITES?

L'inclusion des verbes défectifs et impersonnels dans notre programme est non seulement difficile mais aussi discutable. Bien qu'il ne soit pas possible d'omettre un verbe impersonnel comme *pleuvoir* il faut que nous décidions si nous voulons inclure des verbes comme *verglacer* et *ouïr*.

Où faut-il fixer les limites? S'agit-il uniquement de ces verbes qui sont assez fréquemment utilisés dans la langue contemporaine, ou faut-il admettre aussi ceux que l'on peut rencontrer dans la littérature, bien qu'ils ne soient plus utilisés de manière très fréquente?

Nous avons, il paraît, trois alternatives. Nous pouvons exclure un verbe défectif soit en le laissant sans traitement soit en l'excluant d'une manière active ("Ce verbe n'existe plus. Veuillez entrer un autre verbe"), ou nous pouvons l'accepter en donnant des renseignements concernant les éventuelles formes désuètes et d'autres particularités.

Laisser une forme verbale sans traitement nous paraît le moins satisfaisant de ces trois choix. Le programme conjugue selon la forme du verbe et il va quand même tenter de conjuguer *avenir* ou *bienvenir* par analogie avec *venir* et *chaloir* selon *valoir*, par exemple, et cela malgré que l'on sache que beaucoup des formes résultantes n'ont jamais été réalisées dans la langue. Taper un

infinitif impossible comme *Ikjsdfoir* et regarder les conjugaisons résultantes est amusant et ne fait de mal à personne. Entrer un infinitif qui existe ou qui, au moins, existait autrefois et obtenir des conjugaisons qui n'ont jamais existées est beaucoup plus grave.

Le deuxième choix exigerait une liste de "verbes non-désirés" tout au début du traitement. Un verbe appartenant à cette liste serait bloqué, refusé à l'entrée du système. L'affichage d'un simple message du genre "n'existe pas" ou "désuét" a cependant l'air d'un message d'erreur de DOS, le système d'exploitation des ordinateurs PC, censé expliquer un tas de situations différentes sans vraiment clarifier grande chose.

Le système de calcul que nous avons développé plus haut va nous permettre d'adopter le troisième choix. Tout en suivant les renseignements que l'on trouve dans les ouvrages sur le verbe nous pourrions en principe noter les formes qui sont réalisées afin de les accepter. Les autres seront bloquées. A l'aide d'une matrice nous pourrions facilement identifier les coordonnées à calculer pour chaque verbe:

		gésir									
P E R S O N N E	TEMPS	0	1	2	3	4	5	6	7	8	9
	0	+		+		+					
	1	+		+							
	2	+		+							
	3	+		+							
	4	+		+							
	5	+		+							

RESULTATS: TEMPS=0,2,4

		chaloir									
P E R S O N N E	TEMPS	0	1	2	3	4	5	6	7	8	9
	0										
	1										
	2	+	+	+				+	+		
	3										
	4										
	5										

PERS=2, TEMPS (<3), (>5 <8)

Ainsi, par un simple test, on peut facilement bloquer toutes les formes qui ne sont pas conformes au schéma. En PROLOG nous pouvons écrire ce schéma comme suit:

```
verbe_defectif(TEMPS,PERS,RAD1,RAD2).
```

où, dans le cas d'une forme non-réalisée RAD2 représentera "rien" ou "NUL":

```
verbe_defectif(TEMPS,2,chal,chal):- TEMPS<3;   TEMPS>5,TEMPS<8.
verbe_defectif(TEMPS,PERS,chal,"").
```

Ici dans le cas de *chaloir* nous ne voulons accepter que la troisième personne (représentée par 2). Le verbe ne se réalise pas à l'impératif, au participe présent, au passé simple et au subjonctif imparfait. Il faut donc que la variable TEMPS soit inférieure à 3, OU supérieure à 5 ET inférieure à 8. Si oui le test aboutit et le radical reste *chal*-. Sinon le test échoue à la première clause et aboutit à la deuxième, où la valeur "Ø" est affectée au radical lequel est traité par la suite comme NUL et

produira à la sortie un message comme "inusité" ou bien un blanc, selon ce qu'on veut voir s'afficher à l'écran.

4.1 Un problème

Nous avons dit ci-dessus que nous pouvons "en principe" noter les formes qui sont réalisées et celles qui ne le sont pas. Du point de vue de la programmation il n'y a pas de problème, mais il y en a au niveau du corpus. Il n'y a pas d'accord entre les grammaires sur ce qui est un verbe défectif et ce qui n'en est pas un. Cela se voit nettement dans le tableau qui suit. En plus, c'est seulement chez Grevisse que toutes les formes sont notées. Bescherelle nous avertit que tel ou tel verbe est défectif mais ne nous en donne pas de détails. Caput ne traite que quelques uns de ces verbes.

DEUXIEME PARTIE

VERBES DEFECTIFS ET IMPERSONNELS

VERBES DEFECTIFS selon:

	Bescherelle	Caput	Grevisse
accroire	+		+
adire			+
ardre			+
apparoir	+		+
assavoir	+		
avenir	+		
béer	+		+
bienvenir	+		+
braire	-	+	+
bruire	+	+	
chaloir	+		+
choir	-+		+
clore	-+	+	+
déclore	-		+
enclore	-	+	+
renclore			+
éclore	-	+	+
forclore	+		+
comparoir	+		+
contondre			+
contrefoutre	+		
courre	+		
déchoir	-+	+	+
douer	+		
échoir	-+	+	+
endêver	+		
ensuivre	+		
ester	+		+
férir	+		+
forclore	+		
forfaire	+		
foutre	+		
frire	-	+	+
gésir	+		+
issir	+		+
malfaire	+		
méfaire	+		
occire	+		+
ouir	-+		+
paître	-+	+	+
parfaire	+	+	
partir	+		+
poindre	-		+
quérir	+		+
raire	-		+
raller			+
rassir	+		
ravoir	+		+
reclure	+		+
renaître	+	+	
résulter	+	+	
saillir	+		
semondre			+
seoir	-+		+
méssoir	-+		+
sortir	+		
souloir			+
sourdre	+		+
stupéfaire	+		
tître	+		+
tistre	+		
traire	-+	+	+
portraire			+
rentraire	-		+
attraire			+
transir	-		+
urger	+		

4.1.2 Verbes essentiellement impersonnels

Il faut faire la distinction entre les verbes *défectifs* et les verbes *impersonnels* (ou *unipersonnels*). D'une part il y a ceux, comme *neiger*, qui sont uniquement impersonnels(#), et ceux, comme *agir*, par exemple, qui peuvent être utilisés soit d'une façon personnelle: "agissez vite"; "agir contre un projet", soit impersonnelle: "il s'agit de votre situation" (*).

		Bescherelle	Caput	Grevisse
advenir		+		
agir	*	+	+	
brouillasser	#	+	+	
bruiner	*	+	+	
brumasser		+		
brumer		+		
crachiner		+		+
geler	*	+	+	
glacer	*	+	-	+
grêler	#	+	+	+
grésiller		+	-	
neigeoter		+		+
neiger	#	+	+	
pleuvasser		+		
pleuviner		+		+
pleuvoir	#	+	+	
repleuvoir	#	+		
pleuvoter		+		
pluviner		+		
pouvoir	*	+	+	
répugner	*	+	+	
souvenir	*	+	-	
suffire	*	+	+	
tarder	*	+	+	
tonner	*	+	+	
vaser		+		
venter		+		
verglacer	#	+	+	+

TROISIEME PARTIE

Implémentation

5.1 Introduction

Nous procéderons ici à la mise en oeuvre de l'analyse élaborée dans la partie précédente. Nous examinerons des notions telles que la création d'une "boîte à outils", le bon ménage de la mémoire de l'ordinateur, la transcription des règles linguistiques en programme Prolog et quelques méthodes pour tester les algorithmes résultants. Nous examinerons en gros l'ensemble du module NOYAU et en détail quelques sous-modules.

Il nous est impossible de donner, dans l'espace qui nous est imparti, une explication exhaustive du fonctionnement de Prolog. Donc, nous partirons du principe que le lecteur possède quelques notions de base. Malgré tout, des précisions seront données chaque fois que cela semblera nécessaire. D'autre part, tous les exemples extraits du programme seront commentés.

Quand nous parlons des *prédicats prédéfinis* nous entendons les prédicats qui appartiennent au système Turbo Prolog. Tandis que le terme *prédicats* englobera tous les autres, à savoir ceux que nous avons développé pour notre programme.

5.2 La création d'une boîte à outils

Nous voulons bien sûr que notre algorithme soit le plus efficace possible. Il est donc important que nous créions une "boîte à outils" - c'est-à-dire un jeu de prédicats dont nous pouvons nous

servir à n'importe quelle étape du traitement d'un mot.

Si, par exemple, nos données sont stockées dans des listes, nous aurons besoin d'un prédicat qui trouvera le mot ou le morceau du mot que nous cherchons. Un prédicat bien connu nous servira ici:

```
appartient(T,[T:_]).  
appartient(T,[_:Q]):- appartient(T,Q).
```

Appelé comme suit, par exemple,

```
appartient(cev,[en,quér,fai,cev,ind]).
```

ce prédicat cherche à unifier l'élément *cev* avec le premier élément de la liste et, en échouant, essaye avec le deuxième élément jusqu'à ce qu'il réussisse l'unification ou, si l'élément n'est pas trouvé, jusqu'à ce qu'il épuise la liste.

5.2.1 Traitement des éléments minimaux

En utilisant le prédicat de cette façon, on est toujours obligé de préciser l'élément à chercher. Toutefois, pendant le traitement d'un verbe, il n'est souvent pas possible de spécifier l'élément. En outre, il est préférable d'avoir un moyen de dire "tenir et ses composés" ou "quérir et ses composés" plutôt que de ranger chacun des dérivés de "tenir ou quérir" dans des listes. Il est en fait bien possible d'utiliser *appartient* sans préciser l'élément à chercher. Supposons que nous voulions tester si un radical appartient ou non à une liste quelconque. Nous avons

comme radical par exemple *conquer* et la liste *[en,quér,fai,cev,ind]*. Si nous laissons comme variable libre le premier argument d'*appartient*, il cherchera un élément de la liste sans essayer de l'unifier. Et puis, par la suite, nous pouvons faire le test très simple avec *concat*: est-ce qu'il y a quelque chose qui, uni avec cet ELEMENT, constituera RADICAL?

```

appartient(ELEMENT,LISTE), concat(INITIAL,ELEMENT,RADICAL)

appartient(_,[en,quér,fai,cev,ind])
appartient(en,[en,quér,fai,cev,ind])
  concat(_,en,conquérir)
FAIL essayer encore avec l'élément suivant
appartient(_,[quér,fai,cev,ind])
appartient(quér,[quér,fai,cev,ind])
  concat(con,quér,conquér)

RETURN:  INITIAL = con
         ELEMENT = quér
         RADICAL = conquér

```

Appartient est un prédicat récursif et il sera intéressant d'utiliser encore plus cette récursivité afin d'éviter la nécessité de réécrire chaque fois *appartient* et *concat*. Cela peut être réalisé à l'aide de notre prédicat *appartenance*:

```

appartenance(INITIAL,ELEMENT,LISTE,RADICAL):- appartient(ELEMENT,LISTE),
                                                concat(INITIAL,ELEMENT,RADICAL).

```

Une fois instancié l'appel est fait comme ci-dessous:

```

appartenance(_,_,[en,quér,fai,cev,ind],conquér)

  appartient(en,[quér,fai,cev,ind])
  concat(_,en,quér)
  FAIL
  appartient(quér,[fai,cev,ind])
  concat(con,quér,conquér)
  RETURN appartenance(con,quér,[en,quér,fai,cev,ind],conquér)

```

5.2.2 Exclusion des éléments

Dans les cas où l'on a une règle générale qui a quelques exceptions, on traite habituellement les exceptions d'abord et les autres mots qui observent la règle en suite. Il existe néanmoins des cas où il y a plusieurs règles et l'ordre n'est pas tout à fait évident.

Dans notre programme nous souhaiterions séparer tous les verbes de la 2^e groupe en -ir de tous ceux du troisième groupe. Nous pourrions traiter séparément tous les 320 verbes de la 2^eme conjugaison, qui se conjuguent d'après *finir*, si nous avions un moyen de les séparer des autres verbes en -ir.

Le moyen le plus facile et le plus direct est de dresser des listes d'exceptions et puis à nouveau à l'aide du prédicat *appartient* de chercher si le présent radical est dans la liste. Pour exclure l'élément il suffit de le faire en utilisant le prédicat prédéfini *NOT*:

```
radical(RAD),  
exceptions(LISTE),  
not(appartient(RAD,LISTE)), ..... traitement...
```

Si le radical est trouvé dans la liste il sera exclu, car avec *NOT* le succès est paradoxalement l'échec. Pourtant nous cherchons à établir comme ci-dessus un moyen de spécifier "tenir et ses composés". Cela paraît d'abord assez simple,

```

rad(RADICAL),
exceptions(LISTE),
appartient(ELEMENT,LISTE),
not(concat(INITIAL,ELEMENT,RADICAL)).

```

mais bien que CONCAT puisse être utilisé ainsi, TURBO PROLOG ne permet pas de variables libres dans NOT, ce qui est le cas ici avec la variable INITIAL. Nous avons donc besoin d'instancier INITIAL avant d'essayer de l'exclure avec NOT. En fait, il nous faut d'abord tester comme ci-dessus s'il y a un élément dans la liste qui, ajouté à INITIAL, formera le radical et puis, si cette unification a lieu il faut rejeter le radical; si l'unification n'a pas lieu le radical peut être accepté.

- | | |
|--|--|
| (A) chercher ELEM de la liste | |
| (a) la liste est-elle épuisée? | Oui - aller à (B)
Non - aller à (b) |
| (b) <quelque_chose> + ELEM = RAD? | Oui - aller à (c)
Non - retourner à (A) |
| (B) affecter valeur "xxx" à INIT et ELEM et..... aller à (c) | |
| (c) not(<quelque_chose> + ELEM = RAD)? | Oui - aller à (D)
Non - aller à (C) |
| (C) rejeter le radical | |
| (D) accepter le radical et continuer le traitement | |

Cela pourrait être exprimé ainsi en TURBO PROLOG:

```

exclure(INIT,ELEM,RAD,LISTE):- appartenance(INIT,ELEM,LISTE,RAD),!.
exclure(xxx,xxx,RAD,LISTE).

not(concat(INIT,ELEM,RAD)).

```

Un exemple

Dans le programme, avant le traitement principal des "cas difficiles" du 3^e groupe il faut exclure plusieurs verbes qui se terminent par *v* et qui se confondraient avec *voir*, dont le

radical à cette étape du traitement n'est que *v*. Les verbes tels que *suivre*, *vivre*, *servir* et *pleuvoir*, ainsi que leurs composés, n'exigent pas de traitement dans cette partie du programme et on peut les laisser passer sans modification:

```
exclure(INIT,ELEM,RAD,[suiv,viv,serv,pleuv]),  
not(concat(INIT,ELEM,RAD)),
```

ce qui donne instancié, par exemple dans le cas de *poursuivre*:

```
exclure(_,_,poursuiv[suiv,viv,serv,pleuv]),  
not(concat(pour,suiv,poursuiv)),
```

qui bien sûr, dans ce cas là, échouera toujours.

5.2.3 Remplacement d'un élément par un autre

Parfois le traitement morphophonologique que nous avons établi dans la 2^e partie n'est pas assez puissant pour rendre compte des changements que subissent certains radicaux. Dans ce cas là nous nous contenterons d'un simple passage d'un élément à un autre. La plupart des cas "faciles" et tous les cas "difficiles" de la section 3.3.3.4 seront traités de cette façon.

Nous voulons à tout prix éviter pour nos données des structures du type suivant:

```
radical(a,A).  
radical(b,B).  
radical(c,C).  
etc
```

Un traitement utilisant des listes sera plus économique (voir plus loin). A propos des cas "faciles" nous établirons deux

listes: la liste A des formes "sources" et la liste B des formes "cibles":

```
liste A(ui,ci,fi,fri,cri,di,li,viv,croi,ri,rai,plai,tai,mett)
```

```
liste B(uis,cis,f,fr,criv,d,l,véc,cr,ri,Ø,pl,t,m)
```

A chaque élément de la première liste nous voulons faire correspondre un élément de la deuxième. Un simple système de numérotation nous servira à calculer la position de l'élément cherché et ensuite à repérer l'élément correspondant. Soit les deux prédicats suivants:¹

```
calc_pos(X,[X!_],1).
calc_pos(X,[_!L],N):- calc_pos(X,L,Y),N=Y+1.

repérer([X!_],1,X):-!.
repérer([_!Q],N,X):- N>1, N1=N-1, repérer(Q,N1,X).
```

5.2.4 Isolation des éléments

Le prédicat prédéfini *concat* nous permet d'isoler des parties d'un mot, soit un seul caractère, soit un groupe de caractères. Par exemple, écrire et ses composés, au futur et au conditionnel, perdent le *v* de la forme sous-jacente *criv* dans l'environnement du *r* de la terminaison, à la différence des verbes comme *vivre* ou *suivre*. Pour isoler *écrire* etc, on peut le faire comme suit,

```
concat(INIT,criv,RAD)
```

où INIT peut être *ins* ou *dé* etc. Il n'y a pas de problème si on sait auparavant quel élément on cherche mais parfois on ne le

¹ Voir la section 5.3 pour un exemple de leur utilisation.

sait pas. Pour la morphophonologie on doit isoler la dernière consonne du radical afin de tester si'il faut la supprimer ou non. Le prédicat prédéfini *frontstr* ("chaîne avant") permettrait l'isolation des premières consonnes, mais il nous manque une méthode directe pour les dernières à moins que nous ne puissions isoler le mot entier sauf le dernier caractère. *Frontstr* exige un paramètre indiquant le nombre de caractères à isoler, mais étant donné la variété de la longueur des verbes il faut d'abord mesurer cette longueur à l'aide du prédicat prédéfini *str_len* ("longueur de chaîne"):

```
str_len(poursuiv, LONGUEUR)
```

La variable *LONGUEUR* dans ce cas est 8 et on doit donc isoler 7 caractères afin de séparer le *v*:

```
frontstr(7, poursuiv, INIT, FIN)
```

ce qui donne instancié: *INIT* = *poursui*, *FIN* = *v*. Cependant ce chiffre reste toujours variable selon la longueur du mot et ce n'est pas cette longueur qui nous intéresse mais le/les dernier(s) caractère(s), soit $Y=X-Z$ où *Y* représente le nombre de caractères que va isoler *frontstr*, *X* égale la longueur du mot mesurée par *str_len* et *Z* égale le nombre de caractères que nous cherchons à la fin du mot. Supposons que nous ne voulions que le dernier caractère, nous affectons donc à Z la valeur 1:

```
str_len(poursuiv,8), Y=8-1  
frontstr(7,poursuiv,poursui,v)
```

et nous avons v comme sortie.

Nous aurons évidemment besoin d'un tel outil à plusieurs reprises dans le programme et afin d'éviter la répétition de la même chose nous en créerons un prédicat spécial *isol_élem*:

```
isol_élem(Z,RAD,INIT,FIN):- str_len(RAD,X), Y=X-Z,  
                             frontstr(Y,RAD,INIT,FIN).
```

La section suivante montre une application pratique des outils que nous avons élaborés ici.

5.3 Quelques remarques à propos de la taille du programme

5.3.1 L'importance du CODE

Lors de la programmation on a parfois à se plaindre du manque de mémoire. Le problème, bien qu'il soit réel, n'est pas toujours la faute de l'ordinateur mais souvent celle du programmeur.

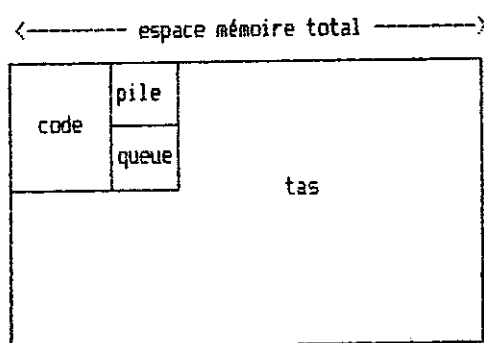
Turbo Prolog divise la mémoire de l'ordinateur en quatre zones différentes:

- le CODE - zone pour le code machine (la version binaire du programme lisible par l'ordinateur)
- le TAS - mémoire dynamique, base de données, autres données

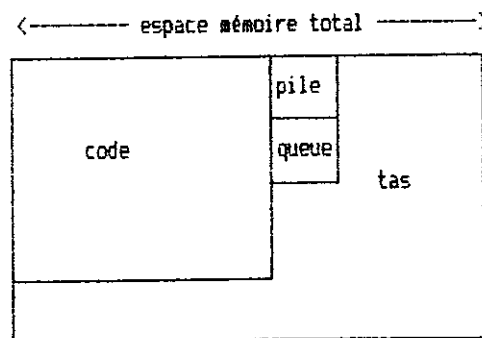
la PILE - zone pour le passage des paramètres et des adresses de retour

la QUEUE - zone pour le passage des variables "par référence"

Tout programmeur doit rendre compte du code et du tas mais habituellement il n'a pas besoin de s'occuper de la pile ni de la queue et ainsi nous les laisserons de côté dans les explications qui suivent. Le système donne au programmeur la possibilité de modifier la taille du code, de la pile et de la queue. La taille du tas est donc tout ce qui reste après l'allocation de mémoire aux autres zones:



Représentation approximative de l'allocation de mémoire par défaut au CODE, PILE, QUEUE et ce qu'il reste pour le TAS.



Après l'élargissement du CODE l'espace pour le TAS est sensiblement réduit.

La plupart du temps c'est donc à la taille du code que le programmeur doit penser. Le système va l'avertir si son programme dépasse la taille affectée au code et il a la possibilité de l'élargir. Pourtant chaque élargissement de la taille du code entraîne la diminution de l'espace mémoire pour le tas, c'est-à-dire pour les données. On risque de créer un grand programme qui ne peut lire qu'une petite base de données.

Plusieurs fois pendant la programmation de CONJUGUE le système nous a averti qu'il manquait de mémoire. Inutile dans ce cas là d'étendre sans cesse la taille du code parce que la mémoire de l'ordinateur n'est pas extensible à l'infini. Il faut penser à réduire la taille du programme afin qu'il ne dépasse pas les limites de mémoire disponible. La boîte à outils que nous avons présenté plus haut est un moyen d'éviter la répétition inutile des mêmes structures, donc l'allongement inutile du programme. Nous allons à l'aide de l'exemple suivant examiner comment ces outils peuvent être utilisés avec efficacité.

5.3.2 Deux représentations de la même opération

La façon la plus simple de déterminer le traitement de chaque groupe de verbes en Prolog est de ranger chacun séparément sur une ligne, comme dans les schémas 1 et 2 dans les sections 2.3.1 et 2.3.2 de la deuxième partie. Par exemple, pour identifier les verbes de type *tenir/venir* ou de type *acquérir* on peut utiliser le prédicat prédéfini `CONCAT` comme dans l'exemple suivant. La variable `RAD` représente le radical (par exemple *mainten-*, *souven-* et la variable `INIT` la partie initial avant "l'élément minimal" (par exemple *maint*, *souv*)²:

```
concat(INIT,en,RAD)    ---> traitement
concat(INIT,quer,RAD) ---> traitement
etc
```

² On suppose ici que chaque verbe a déjà été soumis à un traitement qui le sépare de sa marque d'infinitif *er*, *ir* etc):

Une autre méthode consiste à ranger ces tronçons de verbes dans une liste:

```
verbe([en,quér.....etc]).
```

Elle nécessiterait un traitement plus compliqué mais nous démontrerons qu'elle rend plus compacte une série importante de données.

Un petit test que nous avons fait montre bien la différence d'efficacité entre ces deux méthodes. Prenons un échantillon de six verbes. Nous nous proposons de vérifier si le verbe que nous traitons appartient à une certaine classe (dans l'exemple les verbes ont été choisis arbitrairement):

La première méthode

```
GOAL readln(RAD), verbe(RAD), write(RAD).
```

CLAUSES

```
verbe(RAD):- concat(INIT,en,RAD),!.
verbe(RAD):- concat(INIT,quér,RAD),!.
verbe(RAD):- concat(INIT,fai,RAD),!.
verbe(RAD):- concat(INIT,cev,RAD),!.
verbe(RAD):- concat(INIT,aît,RAD),!.
verbe(RAD):- concat(INIT,ind,RAD),!.
```

La deuxième méthode

```
GOAL readln(RAD),
      verbe(LISTE),
      appartenance(INIT,ELEM,LISTE,RAD),
      write(RAD).
```

CLAUSES

```
appartenance(INIT,ELEM,LISTE,RAD):- appartient(ELEM,LISTE),
                                     concat(INIT,ELEM,RAD).
appartient(X,[X: _]).
```

```
appartient(X,[_!L]):- appartient(X,L).
```

```
verbe([en,quér,fai,cev,aît,ind]).
```

Avec la directive `DIAGNOSTICS` le programmeur peut voir non seulement le comportement des prédicats mais aussi leur taille (du code, en paragraphes). Le test donne les tailles suivantes:

METHODE_1		METHODE_2	
goal =	91	goal =	228
verbe =	441	liste =	40
		appartenance =	107
		appartient =	75
TOTAL	532	TOTAL	450

La différence de taille est peut être surprenante, mais si on élargit l'échantillon à, par exemple, 24 verbes et si on modifie le traitement pour changer le radical aussi, la différence devient vraiment étonnante.

METHODE_1

```
GOAL readln(RAD), verbe(RAD,RAD2), write(RAD2).
CLAUSES
1  verbe(RAD,RAD2):- concat(INIT,en,RAD),RAD2=tienn,!.
2  verbe(RAD,RAD2):- concat(INIT,quér,RAD),RAD2=quière,!.
3  verbe(RAD,RAD2):- concat(INIT,fai,RAD),RAD2=fass,!.
4  verbe(RAD,RAD2):- concat(INIT,cev,RAD),RAD2=goiv,!.
5  verbe(RAD,RAD2):- concat(INIT,aît,RAD),RAD2=aiss,!.
6  verbe(RAD,RAD2)
    etc
24 verbe(RAD,RAD2):- concat(INIT,ind,RAD),RAD2=ign,!.

```

METHODE_2

Ici il faut ajouter deux prédicats de notre boîte à outils et une liste. Le prédicat `calc_pos` calcule la position de l'élément déjà vérifié comme dans la liste. Le prédicat `repérer` trouve l'élément

correspondant dans la deuxième liste ³.

```
GOAL verbe1(LISTE), verbe2(LISTE2),
      readln(RAD),
      appartenance(INIT,ELEM,LISTE,RAD),
      calc_pos(ELEM,LISTE,X),
      repérer(LISTE2,X,RAD2),
      write(RAD2).

CLAUSES
repérer([X!_],1,X):-!.
repérer([_!Q],N,X):- N>1, N1=N-1, repérer(Q,N1,X).

calc_pos(X,[X!_],1).
calc_pos(X,[_!L],N):- calc_pos(X,L,Y),N=Y+1.

appartient(X,[X!_]).
appartient(X,[_!L]):- appartient(X,L).

appartenance(INIT,ELEM,LISTE,RAD):- appartient(ELEM,LISTE),
                                     concat(INIT,ELEM,RAD).

verbe1([en,quér,fai,cev,aît, -- etc -- ind]).
verbe2([tien,quière,fass,goiv,aiss, -- etc -- ign]).
```

Et les résultats:

METHODE_1	METHODE_2
goal = 92	goal = 228
verbe = 2259	verbe1 = 40
	verbe2 = 40
	appartenance = 107
	appartient = 75
	calc_pos = 128
	repérer = 96
TOTAL 2341	TOTAL 714

Bien qu'il faille ajouter quelques prédicats et que la méthode 1 soit plus simple, la méthode 2 est nettement plus économique comme traitement. Si on eut davantage de sous-conditions avec la méthode 1, il faudrait les répéter sur chaque ligne, tandis

³ A noter ici c'est que, quelles que soient leur longueurs, les listes sont toujours de la même taille (du code).

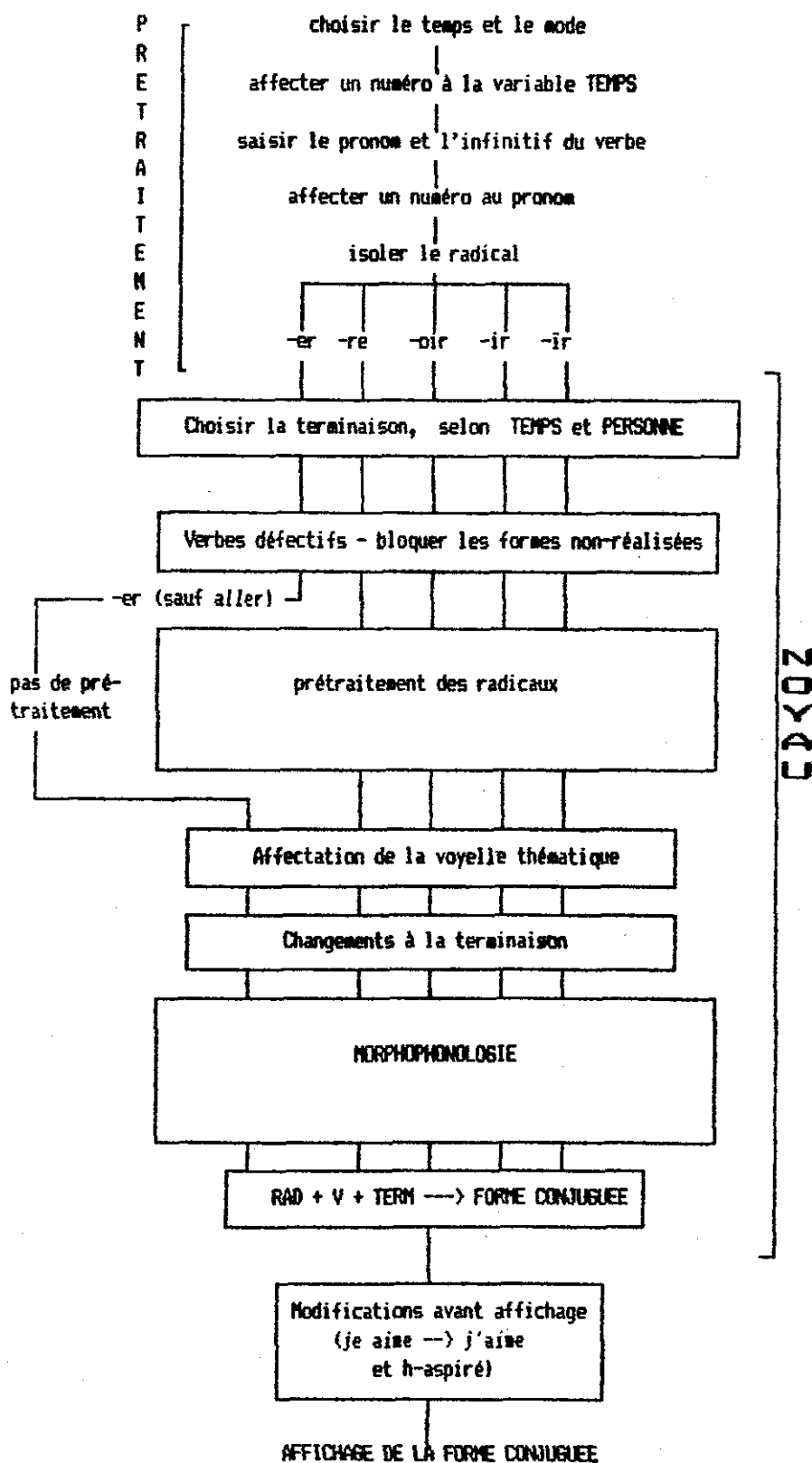
qu'avec la méthode 2 elles ne sont indiquées qu'une seule fois, ce qui représente une différence considérable en taille du code. En utilisant notre boîte à outils et en développant des méthodes comme la méthode 2 on a la possibilité de créer des programmes plus grands et plus modulés, sans avoir besoin d'un ordinateur comportant davantage de mémoire.

DESCRIPTION DU PROGRAMME

Le logiciel CONJUGUE est un programme assez complexe. Dans la description qui suit nous ne pouvons pas tout expliquer mais nous nous concentrerons, sur le module central, le "noyau". C'est ce module qui réalise la conjugaison, les autres étant arrangés autour de lui comme interfaces. (Voir diagramme, section 1.6.3.)

Aucune description ne sera donc donnée des modules CONCORDANCE, MENUS et INTERRO. Par contre, les modules GENERATION et CONJUGAISON COMPLETE comportent quelques prédicats essentiels au pré- et post-traitement et ils seront pris en compte uniquement à cet égard.

Les diagrammes du traitement qui ont déjà été présentés sont censés se compléter. Celui de la section 1.6.3 dans la première partie donne une vue d'ensemble du programme entier. Ceux de la deuxième partie montraient les étapes du développement de notre analyse. Dans cette troisième partie nous présenterons d'abord une vue d'ensemble du traitement du module NOYAU suivi par une autre vue en fonction des prédicats, montrant en gros notre algorithme. Enfin nous examinerons deux parties du programme en détail. En fait, nous ne nous concentrerons que sur un seul module du programme, et de surcroît, dans l'espace qui nous est imparti, nous ne pourrions même pas envisager l'intégralité de ce module. Nous nous contenterons de n'examiner en détails que quelques sous-modules.

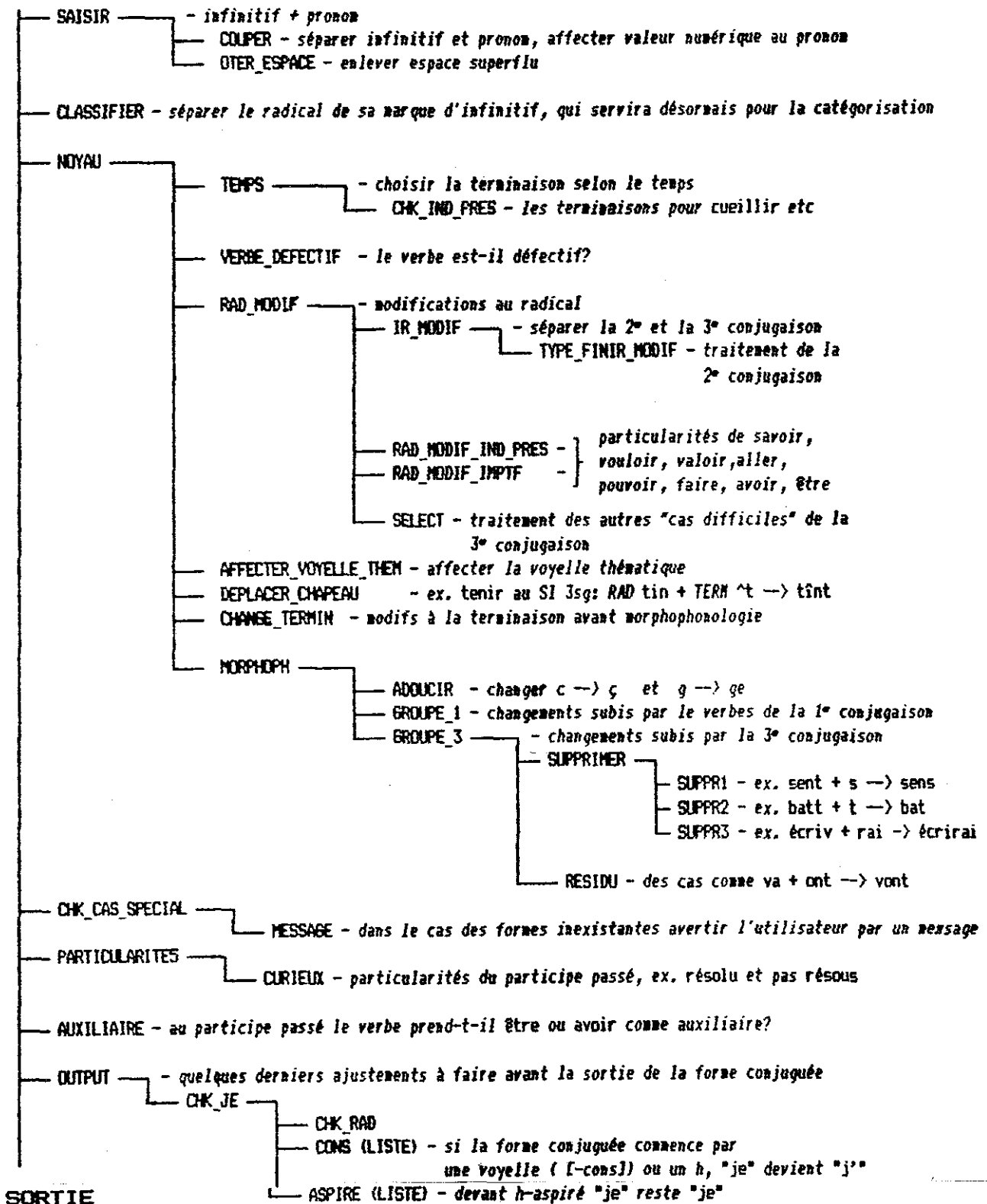


6.2

VUE D'ENSEMBLE EN FONCTION DES PREDICATS

Prédicats en MAJUSCULES, commentaires en italique

ENTREE



6.3 La saisie du groupe verbal

Le verbe est entré à l'infinitif et suivi d'un pronom. Par exemple, on tape "venir je" ou "voir nous" plutôt que l'inverse⁴. La première étape après la saisie même, consiste à séparer l'infinitif et le pronom (couper) ce qui se réalise à l'aide du prédicat prédéfini *fronttoken*. Avant d'affecter un numéro au pronom il faut supprimer l'espace qui y reste attaché (*ôter_espace*). La catégorisation du verbe et l'isolation du radical se fait à l'aide du prédicat *classifier*.

```
saisir (PRON, NO, RAD, TERM) :- readln (GROUPE), saisir verbe + pronom
                                upper_lower (GROUPE, GROUPE2), si jamais l'utilisateur tape en majuscules
                                                                ce prédicat prédéfini les convertira en
                                                                miniscules.
                                couper (GROUPE2, VB, PRON, NO), Séparer verbe et pronom
                                classifier (VB, RAD, TERM), !. Séparer radical et marque de l'infinitif
saisir ("", 00, "", "").

couper (GROUPE2, VB, PRN, NO) :- fronttoken (GROUPE2, VB, PRON),
                                ôter_espace (PRON, PRN),
                                pron (PRN, NO, _), !.
couper (GROUPE2, VB, "", 0) :- GROUPE2 = VB, !.

ôter_espace (PRON, PRN) :- frontchar (PRON, ' ', PRN), !.
ôter_espace (PRON, PRN) :- PRN = PRON.

classifier (INF, RAD, er) :- concat (RAD, er, INF), !.
classifier (INF, RAD, oir) :- concat (RAD, oir, INF), !.
classifier (INF, RAD, ir) :- concat (RAD, ir, INF), !.
classifier (INF, RAD, re) :- concat (RAD, re, INF), !.
classifier (INF, RAD, ir) :- concat (RAD, ir, INF), !.
classifier (INF, "", "") :-
    write("\n                >> Mot non reconnu <<"),
    write("\n    Veuillez retapez avec l'INFINITIF du verbe."),
    write("\n                Appuyez sur ESPACE"),
    readchar (C), C = 'Q'.
```

⁴ Nous avons remarqué qu'au lieu de taper "je venir" les gens ont tendance à entrer "je viens" qui, n'étant pas un infinitif est, bien sûr, rejeté par le programme.

6.4 Transcription des regles génératives

Nous avons déjà vu comment la traduction des règles linguistiques en langage Prolog se fait sans beaucoup de difficulté. Dans cette section nous continuerons l'implémentation de notre analyse linguistique, en particulier en ce qui concerne les "cas difficiles" de la troisième conjugaison.

Commençons cependant par les particularités du premier groupe dont les règles sont présentées dans la section 3.3.1.3

6.4.1 Morphophonologie du premier groupe

Bien que l'on puisse maintenir à peu près les mêmes structures en Prolog qu'à l'intérieur même des règles, on doit cependant préciser plus explicitement le traitement à suivre. Examinons le premier exemple du programme, soit la règle nro 1 de la section 3.3.1 et sa transcription en Prolog (la version Prolog a été simplifiée pour cette explication):

Règle 1. type "manger/placer"

c	---	g	/---	#[+(a,o,â)]
g		ge		

```
adoucir(RAD,RAD2,TERMIN):-
    cons_douce(FIN,FIN2),    appel cons_douce
    concat(FRONT,FIN,RAD2),  la consonne finale est-elle une des "consonnes douces"?
    frontstr(1,TERMIN,INITIAL,FIN3), la première voyelle de la terminaison
    appartient(INITIAL,[a,o,â]),    est-elle a, o ou â ?
    concat(FRONT,FIN2,RAD2 ). Si oui, faire changement approprié.
```

Les consonnes "douces" sont stockées séparément
 cons_douce(c,g).
 cons_douce(g,ge).

On considère qu'une voyelle dite "dure" adoucit les consonnes *g* ou *c* quand elle les précède. Pour préciser l'environnement il nous faut deux prédicats prédéfinis: un (CONCAT²) pour séparer la dernière consonne du radical, et un autre (FRONTSTR) pour isoler la première voyelle de la terminaison. Voyons maintenant le même extrait du programme mais supposons que nous traitons le verbe *manger* et le pronom *nous*:

```
adoucir(mang,RAD2,ons):-      La variable RAD2 est pour l'instant vide.
    cons_douce(g,ge),        appel cons_douce, d'abord c,ç, ne vas pas,
                              alors appel g,ge, qui est bon.
    concat(man,g,mang),      La consonne finale est-elle g?
    frontstr(1,ons,o,ns),    Séparer la 1re voyelle de la terminaison.
    appartient(o,[a,o,â]),  La voyelle est-elle a, o ou â ?
    concat(man,ge,mange).    Faire changement g → ge. RAD2 prend valeur mange.
```

La transcription des autres règles pour le premier group se fait comme suit (simplifiée, encore une fois, pour l'exemple):

2. type "eler/eter"

```
t  --->  tt
1  --->  11  / [ -(LISTE1)] ___ #[+e_muet]
```

```
groupe_1(TEMPS,PERS,RAD,RAD2)
    isol_elem(2,RAD,FRONT,END),
    frontstr(1,END,V,C), V=e, appartient(C,[1,t]),
    eler_eter(LISTE2),
    not(appartient(RAD,LISTE2)), exclure le verbe s'il appartient à la liste d'exceptions
    concat(C,C,CC), concat(V,CC,END2), concat(FRONT,END2,RAD2).
```

² Nous n'avons pas besoin ici de notre outil *isol_élé* puisque nous savons quels éléments nous cherchons à isoler, c'est-à-dire le *g* et le *c*, donc le prédicat prédéfini *concat* est bien adéquat.

3. type "peser"

e ---> e/ [+{l,t,c,m,n,p,r,s,v,vr}]__# [+e-muet]

```
groupe_1(TEMPS,PERS,RAD,RAD2),
  appartenance(INIT,ELEM,[el,et,ec,em,en,ep,er,es,ev,evr],RAD),
  frontchar(ELEM,'e',FIN), concat("è",FIN,ELEM2),
  concat(INIT,ELEM2,RAD2).
```

4 a.type "céder" b. type "régler"

é ---> è/[+C?]__ # [[+TEMPS>5]]
[[+e_muet]]

a.

```
groupe_1(TEMPS,PERS,RAD,RAD2):- TEMPS<6,
  isol_elem(2,RAD,FRONT,END),
  frontstr(1,END,FRONT2,END2), FRONT2=é,
  concat(è,END2,FIN2), concat(FRONT,FIN2,RAD2).
```

b.

```
groupe_1(TEMPS,PERS,RAD,RAD2):-
  isol_elem(3,RAD,FRONT,END),
  not(END=éci), not(END=éfi), not(END=éri),
  frontstr(1,END,FRONT2,END2), FRONT2=é, TEMPS<6,
  concat(è,END2,FIN2), concat(FRONT,FIN2,RAD2).
```

5. type "envoyer"

oy ---> er/env__ # [+ TEMPS > 5]
[+ TEMPS < 8]

```
groupe_1(TEMPS,PERS,RAD,RAD3,TERMIN2,TERMIN3):-
  TEMPS>5, TEMPS<8,
  frontstr(1,TERMIN2,INIT,FIN), INIT=e,
  concat(FRONT,envoy,RAD), concat(FRONT,enver,RAD3),
  concat("",FIN,TERMIN3).
```

6. type "payer"

y ---> i/[- ey]__ # [+ e-muet]

```
groupe_1(TEMPS,PERS,RAD,RAD2):-
  exclure(INIT,ELEM,RAD2,[ey]),
  not(concat(INIT,ELEM,RAD2)),
  concat(FRONT,y,RAD),
  mutes_1(LISTE), appartient(TERMIN2,LISTE),
  concat(FRONT,i,RAD2).
```

Le cas général c'est-à-dire aucun traitement
groupe_1(TEMPS,PERS,RAD,RAD2):- RAD=RAD2.

Les règles 7 et 8 de l'analyse sont traitées ailleurs dans le programme. La règle 4 doit être traitée en deux parties,

puisque'on ne peut pas préciser en Prolog "é suivi d'une OU de deux consonnes".

6.4.2 Morphophonologie du troisième groupe

Nous ne présenterons pas ici tous les détails des changements subis, à l'aide de notre morphophonologie, par les verbes de la 3^e conjugaison. Nous nous contenterons d'une description de la suppression des éléments à l'indicatif présent (voir section 3.3.3.5.1) et nous la présenterons en détail.

Soit les règles suivantes:

Première passe

(v,s,l,t,m) ----> Ø/[-{vê,gi}]__# {s,x,t}
d ----> Ø/in__# {s,x,t}

Deuxième passe

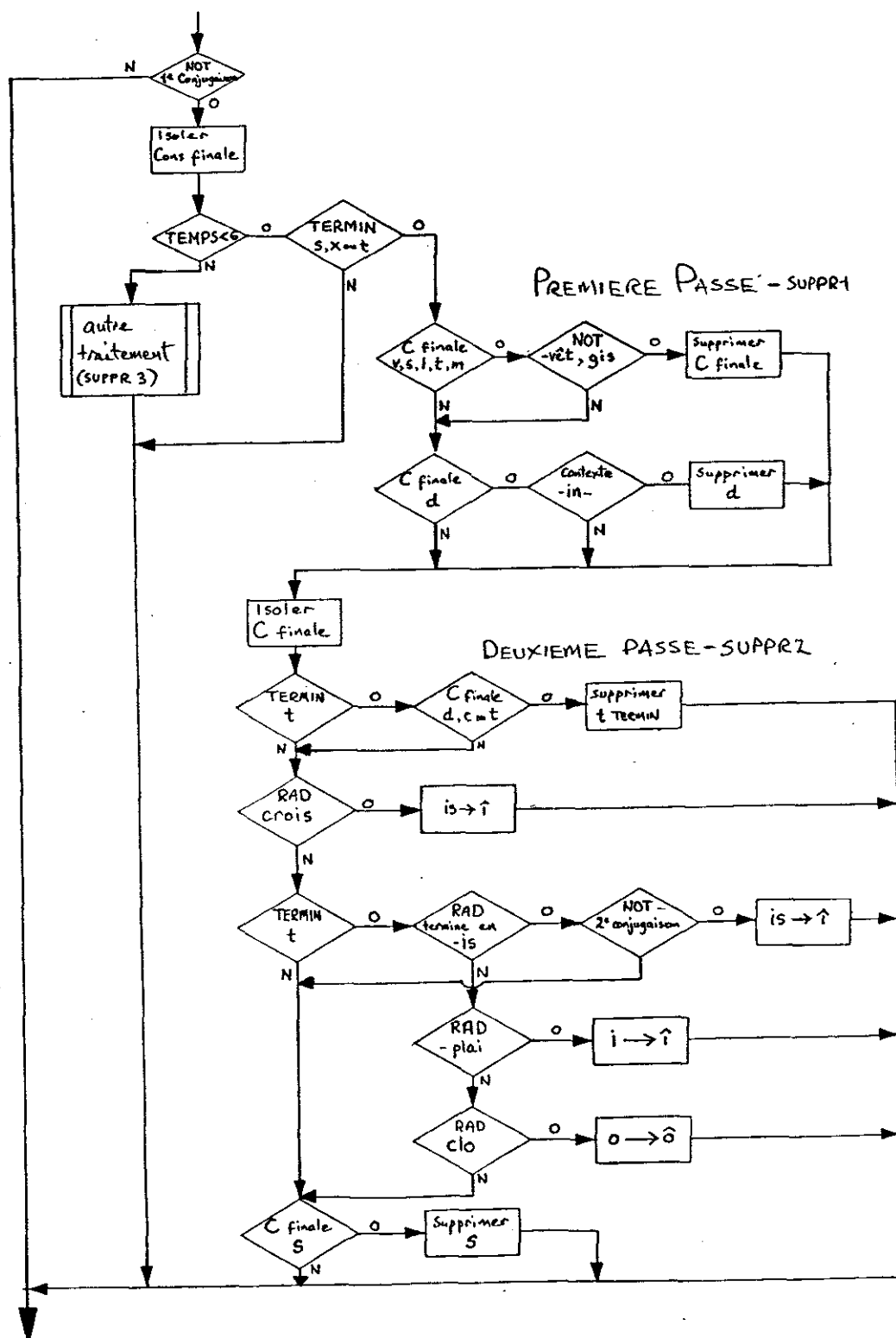
t ----> Ø/___{d,c,t}#
is ----> î/##cro_#(s,t)
s ----> ^/i__#t
i ----> î/pla__#t
o ----> ô/##cl__#t
s ----> Ø/___#(s,t)

Voyons d'abord, à la page suivante, l'algorithme afin de bien comprendre le traitement à suivre:

Organigramme du sous-programme SUPPRIME

Voir "Vue d'ensemble en fonction des prédicats" plus haut (6.2).

Le sous-sous-programme SUPPR3 n'est pas détaillé ici.



Prenons par exemple *boire* à la 1^{re} personne du singulier de l'indicatif présent. Le radical RAD pour ce temps et cette personne sera *boiv-* et la forme cible, la forme que nous désirons à la sortie, est *bois*. A la première passe le *v* est supprimé:

boiv ---> *boi*

La deuxième passe n'a donc aucun effet et le radical changé RAD2 *boi* sort et est lié à la terminaison *s*: *boi + s* ---> *bois*.

Le cas de *battre* à la troisième personne est plus compliqué. A la première passe un *t* est supprimé, à la deuxième passe la terminaison *t* est supprimée ce qui donne *bat+Ø*:

batt + t ---> *bat + t* ---> *bat + Ø* ---> *bat*

Dans le cas de *connaître*, à la troisième personne du singulier, RAD = *connaiss*. A la première passe un *s* est supprimé - *connaiss* + *t*. A la deuxième passe *is* change à *i*:

connaiss + t ---> *connaiss + t* ---> *connaî + t* ---> *connaît*

L'écran à la page suivante montre la sortie d'un programme de test que nous avons créé afin de vérifier les règles et leurs transcriptions. On peut voir les particularités des verbes tels que *croître*, ou *plaire* et *taire*. A noter aussi le traitement différent exigé dans les cas de *foutre* et *vêtir* et leurs

composés.

Ce sous-programme traite aussi le seul changement morphophonologique nécessaire pour le deuxième groupe, par exemple

```
finis + s ---> finis
finis + t ---> finit
```

mais le test présenté ci-dessous ne prend en considération que les verbes du 3^e groupe.

sent + s--> sens	dis + t--> dit	assied + s--> assieds
sent + t--> sent	confis + s--> confis	assied + t--> assied
fout + s--> fous	confis + t--> confit	prend + s--> prends
fout + t--> fout	conduis + s--> conduis	prend + t--> prend
dorm + s--> dors	conduis + t--> conduit	vainc + s--> vaincs
dorm + t--> dort	plaine + s--> plains	vainc + t--> vainc
serv + s--> sers	plaine + t--> plaint	coud + s--> couds
serv + t--> sert	peind + s--> peins	coud + t--> coud
reçoiv + s--> reçois	peind + t--> peint	tien + s--> tiens
reçoiv + t--> reçoit	clos + s--> clos	tien + t--> tient
meuv + s--> meus	clos + t--> clôt	romp + s--> romps
meuv + t--> meut	batt + s--> bats	romp + t--> rompt
boiv + s--> bois	batt + t--> bat	cour + s--> cours
boiv + t--> boit	connaiss + s--> connais	cour + t--> court
suiv + s--> suis	connaiss + t--> connaît	meur + s--> meurs
suiv + t--> suit	accroiss + s--> accrois	meur + t--> meurt
viv + s--> vis	accroiss + t--> accroît	plai + s--> plais
viv + t--> vit	croiss + s--> crois	plai + t--> plaît
écriv + s--> écris	croiss + t--> croît	tai + s--> tais
écriv + t--> écrit	croi + s--> crois	tai + t--> tait
lis + s--> lis	croi + t--> croît	gis + s--> gis
lis + t--> lit	vêt + s--> vêts	gis + t--> gît
dis + s--> dis	vêt + t--> vêt	

Voyons maintenant la transcription en Prolog de cette analyse.

```
groupe_3(TEMPS,RAD,RAD3,TERMIN,TERMIN2):-
    isol_elem(1,RAD,RA,D), Isolier le dernier caractère
    supprimer(TEMPS,RAD,RAD3,RA,D,TERMIN,TERMIN2).
```

```
supprimer(TEMPS,RAD,RAD3,RA,D,TERMIN,TERMIN2):-TEMPS<6,
    appartient(TERMIN,[s,x,t]),
    suppr1(RAD2,RA,D,TERMIN),
    suppr2(RAD2,RAD3,TERMIN,TERMIN2),!.
supprimer(TEMPS,RAD,RAD2,RA,D,TERMIN,TERMIN2):-
    TEMPS>5,TEMPS<8,
    suppr3(RAD2,RA,D,TERMIN),TERMIN=TERMIN2.
```

BRANCHEMENT

A - Ind Présent au sg.

B - Futur et conditionnel.
(Cette partie sera laissée de côté)

INDICATIF PRESENT - première passe

```
suppr1(RAD2,RA,D,TERMIN):- appartient(D,[v,s,l,t,m]),
    exclure(INIT,ELEM,RA,[v,e,gi]),
    not(concat(INIT,ELEM,RA)),RAD2=RA,!.
suppr1(RAD2,RA,d,TERMIN):- isol_elem(2,RA,R,A), A=in, RAD2=RA,!.
suppr1(RAD2,RA,D,TERMIN):- concat(RA,D,RAD), RAD=RAD2.
```

- deuxième passe

```
suppr2(RAD2,RAD3,t,""):- isol_elem(1,RAD2,RA,D), appartient(D,[d,c,t]),
    concat(RA,D,RAD),RAD3=RAD,!.
suppr2(crois,crois,T,T2):- T=T2,!.
suppr2(RAD2,RAD3,t,t):- not(flag(3)), concat(INIT,is,RAD2), concat(INIT,i,RAD3)

suppr2(RAD2,RAD3,t,t):- concat(INIT,plai,RAD2), concat(INIT,plai,RAD3),!.
suppr2(clo,clô,t,t):-!.
suppr2(RAD2,RAD3,T,T2):- concat(INIT,s,RAD2), RAD3=INIT,T=T2,!.
suppr2(RAD2,RAD3,T,T2):- RAD2=RAD3,T=T2.
```

6.5 Le traitement des temps - un exemple

Le participe passé PP (TEMPS=5, selon notre numération) d'un nombre de verbes du troisième groupe est formé avec le même radical et la même voyelle que le passé simple PS (TEMPS=8), et le subjonctif imparfait SI (TEMPS=9). Lire, par exemple,

```
PS  l + u + terminaison
PP  l + u + Ø
```

ou bien vivre

```
PS  véc + u + terminaison
PP  véc + u + Ø
```

Comment alors donner à l'ordinateur une instruction du genre "pour certains verbes le participe passé est formé en prenant la

même racine et la même voyelle que pour le passé simple, en laissant tomber la terminaison"? Cela s'avère très simple avec notre méthode de calcul. Pour choisir le bon ensemble de terminaisons il faut tester quel est le TEMPS choisi par l'utilisateur. Pour le présent cas nous n'avons pas besoin de terminaison, donc nous ne faisons pas appel à terminaison(TEMPS,TERMIN,NO) si TEMPS=5. La liste *part_passé*, ne contient que des morceaux de mots mais à l'aide du prédicat *appartenance* cette liste est valable non seulement pour les verbes "de base" mais aussi pour n'importe quels composés - dans le cas de *vivre* aussi pour *revivre* et *survivre*. Soit la liste *participe passé*:

```
part_passé1([cour,croi,ill,rm,rv,nt,rt,pleuv,fu,fall,ch,clu,uiv,viv,li,ri,
             plai,tail]).
```

et le sous-programme *temps*, qui choisit les terminaisons selon TEMPS et PERS. La partie qui nous intéresse est dans la boîte.

```
temps(0,0,RAD,oir,PRON,PERS,x):- PERS<2,
    appartenance(INIT,ELEM,[pouv,voul,val],RAD),!.
temps(0,0,RAD,er,PRON,PERS,TERMIN):- not(RAD=all),
    terminaison(TEMPS,TERMIN,PERS),!.
temps(3,3,RAD,er,PRON,PERS,TERMIN):- terminaison(3,TERMIN,PERS),!.
temps(TEMPS,TEMPS2,RAD,INF,PRON,PERS,TERMIN):-
    chk_ind_pres(TEMPS,RAD,INF,PRON,PERS,TERMIN), TEMPS2=TEMPS,!.
temps(1,1,êt,INF,PRON,PERS,TERMIN):- terminaison2(_,TERMIN,PERS),!.
temps(4,4,RAD,INF,PRON,0,ant):- !.
temps(5,5,RAD,er,PRON,0,é):- not(RAD=all),!.
```

```
temps(5,8,RAD,INF,PRON,0,""):-
    exclure(INIT,ELEM,RAD,[fri,cri]), } exclure frire et écrire et composés
    not(concat(INIT,ELEM,RAD)),
    part_passé1(LISTE),
    appartenance(INIT2,ELEM2,LISTE,RAD),!.
```

```
temps(5,5,RAD,INF,PRON,0,""):-!.
temps(7,7,RAD,INF,PRON,PERS,TERMIN):-terminaison(2,TERM,PERS),
```

Dans le cas de vivre nous aurons pour sortie par exemple:

où les variables PRON, PERS et TERMIN sont vides, représenté par deux guillemets.

```
rad_modif(8,re,,viv,,0)
```

$$vec + u + \phi \longrightarrow vecu$$

6.6 Séparation des verbes en -ir

Nous avons déjà examiné plus haut (3.3.3.3) la séparation des deux types de verbes en -ir qui chevauchent les 2^e et 3^e

conjugaisons. Ayant toujours présent à l'esprit cette analyse voyons maintenant comment la séparation s'est réalisée dans notre programme (quelques étapes sont omises).

Les deux listes

```
type_finir([ert,lent,retent,amort,assort,asserv,impart,répart,tart,
            queill,vieill,jail])
sentir_etc([lent,rt,vêt,ouvr,ffr,ill,dorm,cour,serv])
```

PREMIERE ETAPE: EXCLURE TOUS LES VERBES EN -IR QUI SONT DES CAS "DIFFICILES":

```
rad_modif(TEMPS,INF,TERMIN,TERMIN2,RAD,RAD3,PRON):-
    not(appartient(INF,[er,re,oir])),    Nous ne retenons que des verbes en -ir et ir
                                         Pour les autres, aller à la clause suivante.
    exclure(INIT,ELEM,RAD,[en,quér,bouill,mour,faill,gés,ouï]),
    not(concat(INIT,ELEM,RAD)), Si le verbe n'appartient pas à cette liste, procéder
    ir_modif(TEMPS,RAD,RAD3),TERMIN=TERMIN2,!. à IR_MODIF.
rad_modif(TEMPS,INF,TERMIN,TERMIN2,maudi,RAD3,PRON):-
    ir_modif(TEMPS,maud,RAD3),TERMIN=TERMIN2,!. Le verbe est-il maudire? Si oui,
                                         aller à IR_MODIF, si non, autre traitement.
```

DEUXIEME ETAPE: SEPARER LES VERBES EN 2 GROUPES

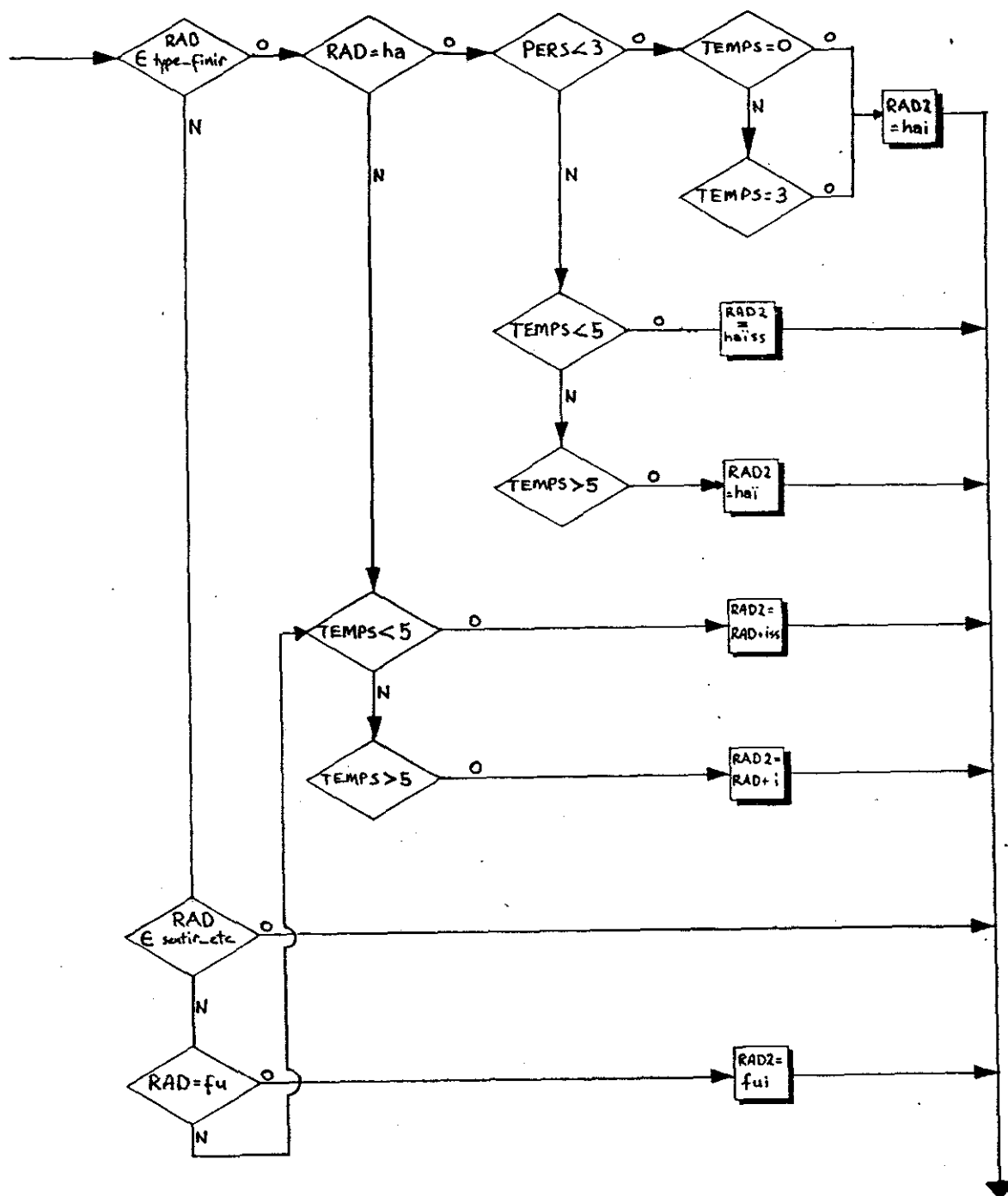
```
ir_modif(TEMPS,RAD,RAD2):- type_finir(LISTE),
    appartenance(INIT,ELEM,LISTE,RAD), Le verbe appartient-il à la liste?
    type_finir_modif(TEMPS,RAD,RAD2). Si oui, aller à TYPE_FINIR_MODIF. Si non, aller
                                         à la clause suivante.
ir_modif(TEMPS,RAD,RAD2):- sentir_etc(LISTE),
    appartenance(INIT2,ELEM2,LISTE,RAD), RAD=RAD2,!. Le verbe appartient-il à la liste?
                                         Si oui, le but est atteint, si
                                         non, aller à la clause suivante.
ir_modif(TEMPS,RAD,RAD2):- concat(INIT,fu,RAD), concat(INIT,fui,RAD2).
                                         Le verbe est-il fuir? Si oui, but atteint
                                         si non, aller à la clause suivante.
ir_modif(TEMPS,RAD,RAD2):- type_finir_modif(TEMPS,RAD,RAD2). Pas de conditions,
                                         aller à TYPE_FINIR_MODIF.
```

TROISIEME ETAPE: TRAITEMENT DES VERBES DE LA 2^e CONJUGAISON

```
type_finir_modif(TEMPS,ha,hai):- pers(PERS), PERS<3, TEMPS=0; TEMPS=3,!.
type_finir_modif(TEMPS,ha,haïss):- TEMPS<5,!.
type_finir_modif(TEMPS,ha,haï):- TEMPS>5,!.
type_finir_modif(TEMPS,RAD,RAD2):- TEMPS<5, concat(RAD,iss,RAD2),!.
type_finir_modif(TEMPS,RAD,RAD2):- TEMPS>5, concat(RAD,i,RAD2),!.
type_finir_modif(TEMPS,RAD,RAD2):- RAD2=RAD.
```

Voyons à la page suivante une représentation organigrammique des 2^e et 3^e étapes.

Organigramme des sous-programmes IR_MODIF et TYPE_FINIR_MODIF.



6.7 Des programmes pour tester le programme

Au lieu de taper chaque forme de chaque verbe afin de vérifier si le programme fait la conjugaison correctement ou non, nous avons développé plusieurs programmes pour tester soit un sous-programme, soit l'ensemble du programme. Le premier tel programme que nous avons fait prend un infinitif d'une liste, fait marcher le noyau afin de donner la conjugaison de la première personne de l'indicatif présent, ensuite la deuxième etc. Quand il a réalisé toute la conjugaison il l'envoie à un fichier et il prend le deuxième verbe de la liste et ensuite la troisième etc. Cela nous a donné l'idée de développer et d'ajouter le module CONJUGAISON COMPLETE au programme CONJUGUE, où l'on peut voir toute la conjugaison à la fois à l'écran.

Un autre programme teste les sous-modules. On peut taper une liste *ad hoc* d'infinitifs, par exemple tous les verbes qui se conjuguent d'une certaine façon, et puis avoir tous les résultats sous les yeux à la fois. L'écran ci-dessus présente les résultats d'un tel test.

LISTINGS DES MODULES

DU PROGRAMME

MODULE INITIAL

Implementation of modular compilation.

```
nowarnings
code=7000
```

GLOBAL DOMAINS

```
liste = symbol*
strlist= string*
touche = cr ; up ; down ; l ; m ; s ; esc ; left ; right ; home ; end ; otherspec
```

DATABASE

```
storecorrect(integer)          storewrong(integer)
```

```
/* ----- Base de données dynamique ----- */
```

```
vb(symbol)      rad(symbol)      rad2(symbol)      rad3(symbol)
prn(symbol)      inf(symbol)      v(symbol)          v2(symbol)
termin(symbol)   termin2(symbol)   termin3(symbol)   pers(integer)
change(symbol,symbol,symbol)      tms(integer)          espace(integer)
change2(symbol,symbol)
conj_forme(symbol)                flag(real)
```

```
/* -- dans VERBS.DBA ----- */
```

```
verb(integer,symbol)
```

```
/* --- dans INFO.DBA ----- */
```

```
particularité(symbol,string)
```

GLOBAL PREDICATES

```
noyau(integer,symbol,symbol,symbol,integer,symbol) - (i,i,i,i,i,o)
output(integer,integer,symbol,symbol,symbol,symbol,symbol) - (i,i,i,i,i,i)
auxiliaire(symbol,integer,symbol) - (i,i,o)
invariable(integer,symbol,symbol,symbol) - (i,o,i,o)
particularité5(integer,symbol,symbol) - (i,i,o)
nondeterm repeat
chk_je(symbol,symbol) - (o,o)
chk_rad(symbol) - (o)
aspiré(liste) - (o)
cons(liste) - (o)
chk_cas_spéc(symbol,symbol,symbol,symbol) - (i,i,o,o)
message
classifier(symbol,symbol,symbol) - (i,o,o)
chk_pron(integer,integer,symbol,symbol) - (i,i,i,o)
choisir_verbe(integer) - (i)
choisir_verbe2(integer,symbol) - (i,i)
isol_elem(integer,symbol,symbol,symbol) - (i,i,o,o)
clear
readinfo
analyse
window6
mainmenu
submenu(integer) - (i)
list_verbs
conjug_compl
delay(integer) - (i)
```

```

/* -- dans DATA.DBA --- */
table1(integer,symbol) - (i,o) (o,o)
pron(symbol,integer,symbol) - (o,o,o) (i,o,o) (o,i,o)
terminaison(integer,symbol,integer) - (o,o,i) (i,o,i)
terminaison2(integer,symbol,integer) - (o,o,i) (i,o,i)
curieux(integer,symbol,symbol) - (i,o,o)
radical_re(symbol,symbol,symbol) - (o,o,o)
car(symbol,symbol) - (o,o)
mutes_1(liste) - (o)
mutes_2(liste) - (o)
cons_douce(symbol,symbol) - (o,o)

/* Lire les touches */
lireT(touche) - (o)
lireT1(touche,char,integer) - (o,i,i)
lireT2(touche,integer) - (o,i)
convert(integer,integer,char,char) - (i,o,i,o)

include "menus.pro"
include "noyau.pro"
include "conjcomp.pro"
include "utils.pro"
include "intergen.pro"
include "concord.pro"
/*include "tutor.pro"*/

GOAL makewindow(1,16,7,"",0,0,25,80),
  cursor(7,20),
  write(" ",cursor(8,20),
  write(" ",cursor(9,20),
  write(" ",cursor(10,20),
  write(" ",cursor(11,20),
  write(" ",cursor(12,20),
  write(" ",cursor(13,20),
  write(" ")),
  field_str(9,27,26,"L A C O N J U G A I S O N"),
  field_attr(9,27,26,31),
  field_str(10,24,31,"D U V E R B E F R A N C A I S"),
  field_attr(10,24,31,31),
  field_str(15,30,21,"William Kinnaird 1991"),
  field_attr(15,30,21,30),
  cursor(21,23),
  field_str(21,30,22,"Appuyez sur une touche"),
  field_attr(21,30,22,15),
  cursor(21,75),
  asserta(storecorrect(0)),
  asserta(storewrong(0)),
  readchar(_),
  field_str(21,27,22," Veuillez patienter "),
  field_attr(21,27,22,143),
  consult("info.dba"),
  consult("verbs.dba"),
  begin.

```

MENUS

PREDICATES

```
mymenu(INTEGER, INTEGER, STRLIST, INTEGER)

/* Menu selection */
maxlen(STRLIST, INTEGER, INTEGER)
listlen(STRLIST, INTEGER)
writelst(INTEGER, INTEGER, STRLIST)
menu1(INTEGER, INTEGER, INTEGER, INTEGER)
menu2(INTEGER, INTEGER, INTEGER, INTEGER, TOUCHE)
ajustfenetre(INTEGER, INTEGER, INTEGER, INTEGER, INTEGER, INTEGER)
```

```
/* ----- LES MENUS ----- */
```

```
ind_menu(integer) interro_menu interro_menu2
subj_menu(integer) partic_menu(integer)
```

```
exec(integer)
exec1(integer, integer) exec2(integer, integer)
exec3(integer, integer) exec4(integer, integer)
exec5(integer) exec6(integer)
```

```
choisir(integer, integer, integer)
```

```
/* ----- PREDICATS DIVERS ----- */
```

```
windows
interro interro_loop(integer, integer)
generation(integer, integer) begin
/* tutor(integer) */
```

CLAUSES

```
/* XXXXXXXXXXXX LES FENETRES XXXXXXXXXXXXXXXXXXXX */
```

```
begin:- windows, mainmenu.
```

```
windows:- makewindow(2,27,7,"",3,0,10,60),
          makewindow(3,112,7,"ANALYSE",13,0,11,46),
          makewindow(4,80,7,"INFO",13,46,11,34),
          makewindow(5,15,7,"SCORE",3,60,10,20),
          makewindow(6,7,7,"",0,0,3,80),
          /* window 7 for menus. */
          shiftwindow(2), shiftwindow(3), shiftwindow(4), shiftwindow(5),
          window6.
```

```
window6:- shiftwindow(6), clearwindow,
          cursor(0,7),
          write("ESC:menu ESPACE:continuer"),
          field_attr(0,7,3,15),
          field_attr(0,17,6,15),
          shiftwindow(2).
```

```

/* XXXXXXXXXXXXXXXXXXXX LES MENUS XXXXXXXXXXXXXXXXXXXXXXXXXXXX */

mainmenu:- repeat,ymenu(8,25,["Introduction",
                             "Tutor",
                             "Génération",
                             "Interro",
                             "La Conjugaison complète",
                             "Concordance ",
                             "DosShell",
                             " QUITTER LE PROGRAMME "],
CHOIX),exec(CHOIX), CHOIX=7.

mainmenu.

exec(0):- !.

exec(1):- shiftwindow(2),file_str("help.hlp",TEXT),
          display(TEXT),clearwindow,mainmenu,!.
exec(1):- cursor(3,20),write("Impossible de lire HELP.HLP"),
          field_str(3,20,27,"Impossible de lire HELP.HLP"),
          field_attr(3,20,27,90),!.

exec(2):- submenu(3), exec(CHOIX),!.

exec(3):- submenu(0), exec(CHOIX),!.
exec(3):- submenu(0),!.

exec(4):- shiftwindow(2),
          interro,!.

exec(5):- conjug_coapl,!.
exec(6):- list_verbs,!.

exec(7):- system(command),!.
exec(8):- shiftwindow(6), clearwindow,
          cursor(0,20),
          write("Voulez vous vraiment sortir (o/n)?"),
          readchar(CHAR),CHAR='o',
          shiftwindow(1),clearwindow,
          field_str(12,35,9,"Au revoir"),
          field_attr(12,35,9,28),cursor(0,0).

```

```

submenu(GENREC):- mymenu(0,5,[" MODE",
    "Indicatif",
    "Subjonctif",
    "Conditionnel",
    "Impératif",
    "Participe",
    "MENU PRINCIPAL"1,
    CHOIX), exec1(GENREC,CHOIX),CHOIX=7.
exec1(11,0):-!.
exec1(11,1):-!.
exec1(GENREC,2):- ind_menu(GENREC), submenu(GENREC),!.
exec1(GENREC,3):- subj_menu(GENREC), submenu(GENREC),!.
exec1(GENREC,4):- choisir(GENREC,7,1), submenu(GENREC),!.
exec1(GENREC,5):- choisir(GENREC,3,3), submenu(GENREC),!.
exec1(GENREC,6):- partic_menu(GENREC), submenu(GENREC),!.
exec1(0,7).
exec1(1,7).

ind_menu(GENREC):- mymenu(0,21,[" INDICATIF ",
    "Présent",
    "Imparfait",
    "Passé simple",
    "Futur simple",
    " MODE menu ",
    "MENU PRINCIPAL"1,
    CHOIX), exec2(GENREC,CHOIX),CHOIX=7.
exec2(11,0):-!.
exec2(11,1):-!.
exec2(GENREC,2):- choisir(GENREC,0,1), ind_menu(GENREC),!.
exec2(GENREC,3):- choisir(GENREC,2,1), ind_menu(GENREC),!.
exec2(GENREC,4):- choisir(GENREC,8,1), ind_menu(GENREC),!.
exec2(GENREC,5):- choisir(GENREC,6,1), ind_menu(GENREC),!.
exec2(GENREC,6):- submenu(GENREC).
exec2(GENREC,7).

subj_menu(GENREC):- mymenu(0,21,["SUBJONCTIF",
    "Présent",
    "Imparfait",
    "MODE menu",
    "MENU PRINCIPAL"1,
    CHOIX), exec3(GENREC,CHOIX), !.
exec3(11,0):-!.
exec3(11,1):-!.
exec3(GENREC,2):- choisir(GENREC,1,1),subj_menu(GENREC),!.
exec3(GENREC,3):- choisir(GENREC,9,1),subj_menu(GENREC),!.
exec3(GENREC,4):- submenu(GENREC),!.
exec3(GENREC,5):- mainmenu.

```

```

partic_menu(GENREC):- mymenu(0,21,["PARTICIPE",
                                "Présent",
                                "Passé",
                                "MODE menu",
                                "MENU PRINCIPAL"],
                                CHOIX), exec4(GENREC,CHOIX), !.
    exec4(11,0):- !.
    exec4(11,1):- !.
    exec4(GENREC,2):- choisir(GENREC,4,2),partic_menu(GENREC),!.
    exec4(GENREC,3):- choisir(GENREC,5,2),partic_menu(GENREC),!.
    exec4(GENREC,4):- submenu(GENREC),!.
    exec4(GENREC,5):- mainmenu.

```

```

choisir(0,ASPECT,FLAG):- génération(ASPECT,FLAG),!.
choisir(1,ASPECT,_):- interro_loop(ASPECT,1),!.
choisir(2,ASPECT,_):- choisir_verbe(ASPECT),!.
/%choisir(3,ASPECT,_):- tutor(ASPECT),!.!

```

```

interro_menu:- mymenu(8,10,["          INTERRO          ",
                            "Choisissez vous-même le temps et le mode",
                            "Choisissez vous-même le verbe",
                            "Laisser l'ordinateur faire le choix",
                            "          MENU PRINCIPAL          "],
                            CHOIX), exec5(CHOIX).
    exec5(2):- submenu(1), interro_menu,!.
    exec5(3):- submenu(2), interro_menu,!.
    exec5(4):- interro_menu2,!.
    exec5(5):- mainmenu.

```

```

interro_menu2:- mymenu(8,10,["NIVEAU DE DIFFICILTE",
                             "1",
                             "2",
                             "3",
                             "MENU INTERRO",
                             "  MENU PRINCIPAL  "],
                             CHOIX), exec6(CHOIX).
    exec6(2):- interro_loop(_,2),!.
    exec6(3):- interro_loop(_,3),!.
    exec6(4):- interro_loop(_,4),!.
    exec6(5):- interro_menu,!.
    exec6(6):- mainmenu.

```

```

mymenu(LI,KOL,LIST,CHOIX) :- maxlen(LIST,0,Strln),
                             listlen(LIST,Lstlngh), Lstlngh>0,!,
                             Strlngh = Strln, HH1= Lstlngh+2,
                             HH2=Strlngh + 2,
                             ajustfenetre(LI,KOL,HH1,HH2,ALI,AKOL),
                             makewindow(7,7,15,"",ALI,AKOL,HH1,HH2),
                             HH3=Strlngh,
                             writelist(0,HH3,LIST),
                             cursor(0,0),
                             menui(0,Lstlngh,Strlngh,CH),
                             CHOIX=1+CH,
                             removewindow.

mymenu(_,_,_,255).

```

```

maxlen([H:T],MAX,MAX1):-str_len(H,LENGDE),LENGDE>MAX,!,
maxlen(T,LENGDE,MAX1).
maxlen([_:T],MAX,MAX1):-maxlen(T,MAX,MAX1).
maxlen([],LEN,LEN).

listlen([],0).
listlen([_:T],N):-listlen(T,X),N=X+1.

writelist(_,_,[]).
writelist(LI,Stringh,[H:T]):-field_str(LI,0,Stringh,H),
LI1=LI+1,writelist(LI1,Stringh,T).

menu1(LI,MAXLI,Stringh,CHOIX):-field_attr(LI,0,Stringh,112),cursor(LI,0),
lireT(TOUCHE),
menu2(LI,MAXLI,Stringh,CHOIX,TOUCHE).

menu2(LI,_,_,LI,cr):-!.
menu2(LI,MAXLI,Stringh,CHOIX,up):-LI>0,!,field_attr(LI,0,Stringh,7),
LI1=LI-1,
menu1(LI1,MAXLI,Stringh,CHOIX).

menu2(LI,MAXLI,Stringh,CHOIX,down):-LI<MAXLI-1,!,
field_attr(LI,0,Stringh,7),LI1=LI+1,
menu1(LI1,MAXLI,Stringh,CHOIX).

menu2(LI,MAXLI,Stringh,CHOIX,_):-menu1(LI,MAXLI,Stringh,CHOIX).

ajustfenetre(LI,KOL,DLI,DKOL,LI,KOL):-LI<25-DLI,KOL<80-DKOL,!.
ajustfenetre(LI,_,DLI,DKOL,LI,AKOL):-LI<25-DLI,!,AKOL=80-DKOL.
ajustfenetre(_,KOL,DLI,DKOL,ALI,KOL):-KOL<80-DKOL,!,ALI=25-DLI.
ajustfenetre(_,_,DLI,DKOL,ALI,AKOL):-ALI=25-DLI,AKOL=80-DKOL.

/* lecture des touches */
lireT(TOUCHE):-readchar(T),char_int(T,VAL),/*convert(VAL,VAL2,T,T2),*/
lireT1(TOUCHE,T,VAL).

lireT1(TOUCHE,_,0):-!,readchar(T),char_int(T,VAL),lireT2(TOUCHE,VAL).
lireT1(l,_,108):-!.
lireT1(m,_,109):-!.
lireT1(s,_,115):-!.
lireT1(cr,_,13):-!.
lireT1(esc,_,27):-!.
lireT1(otherspec,_,_):-!.

lireT2(up,72):-!.
lireT2(down,80):-!.
lireT2(left,75):-!.
lireT2(right,77):-!.
lireT2(home,71):-!.
lireT2(end,79):-!.
lireT2(otherspec,_,_):-!.

```

LE NOYAU

PREDICATES

```

/* ----- choisir la terminaison ----- */
temps(integer, integer, symbol, symbol, symbol, integer, symbol)
chk_ind_pres(integer, symbol, symbol, symbol, integer, symbol)

/* ----- choisir la forme du radical ----- */
rad_modif(integer, symbol, symbol, symbol, symbol, symbol, integer)
rad_modif_imptf(symbol, integer, integer, symbol, symbol, symbol)
rad_modif_ind_pres(symbol, integer, integer, symbol, symbol, symbol)
ir_modif(integer, symbol, symbol)
type_finir_modif(integer, symbol, symbol)
select(symbol, integer, symbol, symbol, integer, integer)

/* ----- divers ----- */
chk_5(integer, symbol)
verbe_défectif(integer, integer, symbol, symbol)

/* ----- voyelles thématiques et changements aux terminaisons ---*/
affecter_voy_thém(integer, symbol, integer, symbol, symbol)
déplacer_chapeau(symbol, symbol, symbol, symbol, symbol, symbol)
change_termin(symbol, symbol, integer, symbol, symbol, symbol)

/* ----- la morphophonologie ----- */
morphoph(integer, symbol, symbol, integer, symbol, symbol, symbol)
adoucir(symbol, symbol, symbol, symbol)
groupe_1(integer, integer, symbol, symbol, symbol, symbol)
groupe_3(integer, symbol, symbol, symbol, symbol)
supprimer(integer, symbol, symbol, symbol, symbol, symbol, symbol)
suppr1(symbol, symbol, symbol, symbol)
suppr2(symbol, symbol, symbol, symbol)
suppr3(symbol, symbol, symbol, symbol)
residu(symbol, symbol, symbol, symbol)

/* -- la "boîte à outils" ---*/
appartient(symbol, liste)
appartenance(symbol, symbol, liste, symbol)
repérer(symbol, liste, integer)
calc_pos(symbol, liste, integer)
isol_elem(integer, symbol, symbol, symbol)
exclure(symbol, symbol, symbol, liste)

/* ----- les listes ----- */
type_finir(liste)      eler_eter(liste)      sentir_etc(liste)
part_passé1(liste)    part_passé2a(liste)    part_passé2b(liste)
prennent_@tre(liste) re_modif1(liste)      re_modif2(liste)

/* ----- LES TERMINAISONS, PRONOMS, RADICAUX MODIFIES ETC ---*/
irreg(liste)
irreg_A(liste)  irreg_B(liste)  irreg_C(liste)
irreg_D(liste)  irreg_E(liste)  irreg_F(liste)
double_irreg(liste)
variantes(symbol, integer, integer, symbol)

```

CLAUSES

```

noyau(TEMPS,RAD,INF,PRON,PERS,CONJ_FORME):-
    asserta(prn(PRON)), asserta(tmps(TEMPS)), asserta(pers(PERS)),
    asserta(rad(RAD)), asserta(inf(INF)),
    temps(TEMPS,TEMPS2,RAD,INF,PRON,PERS,TERMIN),
    asserta(termin(TERMIN)),
    verbe_défectif(TEMPS,PERS,RAD,RAD2),
    rad_modif(TEMPS2,INF,TERMIN,TERMIN2,RAD2,RAD3,PERS),
    chk_5(TEMPS2,RAD3),
    affecter_voy_thém(TEMPS2,RAD3,PERS,INF,V),
    déplacer_chapeau(RAD3,RAD4,V,V2,TERMIN2,TERMIN3),
    change_termin(RAD4,RAD5,PERS,V2,TERMIN3,TERMIN4),
    morphoph(TEMPS2,RAD5,RAD6,PERS,INF,TERMIN4,TERMIN5),
    concat(RAD6,TERMIN5,CONJ_FORME),
    asserta(conj_forme(CONJ_FORME)),!.
noyau(,_,_,_,_,_):- fail.

```

```

chk_5(5,RAD2):- isol_elem(1,RAD2,INIT,FIN), asserta(rad2(INIT)),
    retract(termin(_)),asserta(termin(FIN)),!.
chk_5(TEMPS,RAD2):- asserta(rad2(RAD2)).

```

```

/*----- DETERMINER LA TERMINAISON SELON LE TEMPS CHOISI -----*/
temps(0,0,RAD,oir,PRON,NO,x):- NO<2,
    appartenance(INIT,ELEM,[pouv,voul,val],RAD),!.
temps(0,0,RAD,er,PRON,NO,TERMIN):- not(RAD=all),
    terminaison(TEMPS,TERMIN,NO),!.
temps(3,3,RAD,er,PRON,NO,TERMIN):- terminaison(3,TERMIN,NO),!.
temps(TEMPS,TEMPS2,RAD,INF,PRON,NO,TERMIN):-
    chk_ind_pres(TEMPS,RAD,INF,PRON,NO,TERMIN),
    TEMPS2=TEMPS,!.
temps(1,1,et,INF,PRON,NO,TERMIN):- terminaison2(_,TERMIN,NO),!.
temps(4,4,RAD,INF,PRON,0,ant):- !.
temps(5,5,RAD,er,PRON,0,é):- not(RAD=all),!.
temps(5,8,RAD,INF,PRON,0,""):- exclure(INIT,ELEM,RAD,[fri,cri]),
    not(concat(INIT,ELEM,RAD)),
    part_passé1(LISTE),
    appartenance(INIT2,ELEM2,LISTE,RAD),!.
temps(5,5,RAD,INF,PRON,0,""):-!.
temps(7,7,RAD,INF,PRON,NO,TERMIN):-terminaison(2,TERM,NO),
    concat(r,TERM,TERMIN),!.
temps(TEMPS,TEMPS2,RAD,INF,PRON,NO,TERMIN):-
    terminaison(TEMPS,TERMIN,NO), TEMPS2=TEMPS,!.
temps(,_,_,_,_,_,_):- fail.

```

```

/* les terminaisons pour cueillir etc */
chk_ind_pres(TEMPS,RAD,INF,PRON,NO,TERMIN):-
    appartenance(INIT,ELEM,[cueill,sail,ouvr,ffr],RAD),
    terminaison(TEMPS,TERMIN,NO),!.
chk_ind_pres(TEMPS,RAD,INF,PRON,NO,TERMIN):-
    terminaison2(TEMPS,TERMIN,NO),!.

```

```

/* -----traitement des verbes defectifs ----- */
verbe_defectif(TEMPS,PERS,fall,fall):- PERS=2; TEMPS=5,!.
verbe_defectif(TEMPS,PERS,fall,"").
verbe_defectif(TEMPS,2,brai,brai):- TEMPS>5,TEMPS<8; TEMPS=0,!.
verbe_defectif(TEMPS,PERS,brai,"").
verbe_defectif(TEMPS,2,chal,chal):- TEMPS>5,TEMPS<8; TEMPS=0,!.
verbe_defectif(TEMPS,PERS,chal,"").
verbe_defectif(TEMPS,PERS,pleuv,pleuv):- PERS>1,PERS<4,!.
verbe_defectif(TEMPS,PERS,pleuv,"").
verbe_defectif(TEMPS,PERS,RAD,RAD2):- RAD=RAD2.

```

```

/*

```

TRAITEMENT DES IRREGULARITES DES RADICAUX

```

*/

```

```

rad_modif(4,re,ant,ant,et,et,PRON).
rad_modif(TEMPS,er,TERMIN,TERMIN2,RAD,RAD3,PRON):-
    not(RAD=all),RAD=RAD3,TERMIN=TERMIN2,!.

rad_modif(TEMPS,INF,TERMIN,TERMIN2,maudi,RAD3,PRON):-
    ir_modif(TEMPS,maud,RAD3),TERMIN=TERMIN2,!.
rad_modif(5,INF,TERMIN,TERMIN2,RAD,RAD3,PRON):-
    exclure(INIT,ELEM,RAD,[prend]),
    not(concat(INIT,ELEM,RAD)),
    part_passé2a(LISTE1),
    appartenance(INIT1,ELEM1,LISTE1,RAD),
    calc_pos(ELEM1,LISTE1,POS),
    part_passé2b(LISTE2),
    repérer(ELEM2,LISTE2,POS),
    concat(INIT1,ELEM2,RAD3),TERMIN=TERMIN2,!.
rad_modif(TEMPS,INF,TERMIN,TERMIN2,RAD,RAD3,PRON):-
    not(appartient(INF,[er,re,oir])),
    exclure(INIT,ELEM,RAD,[en,quér,bouill,mour,fail,gés,ouï]),
    not(concat(INIT,ELEM,RAD)),
    ir_modif(TEMPS,RAD,RAD3),TERMIN=TERMIN2,!.

rad_modif(TEMPS,re,TERMIN,TERMIN2,RAD,RAD2,PRON):- TEMPS<8,
    appartenance(INIT,ELEM,[lui,ui,ci,fi,fri,di,li,plai,tai],RAD),
    concat(ELEM,s,ELEM2),
    concat(INIT,ELEM2,RAD2),TERMIN=TERMIN2,!.

rad_modif(TEMPS,re,TERMIN,TERMIN2,RAD,RAD2,PRON):- TEMPS>7,
    re_modif1(LISTE),    appartenance(INIT,ELEM,LISTE,RAD),
    calc_pos(ELEM,LISTE,POS),
    re_modif2(LISTE2),    repérer(ELEM2,LISTE2,POS),
    concat(INIT,ELEM2,RAD2),TERMIN=TERMIN2,!.
rad_modif(TEMPS,re,TERMIN,TERMIN2,RAD,RAD3,PRON):-
    concat(INIT,cri,RAD),
    concat(INIT,criv,RAD3),
    TERMIN=TERMIN2,!.

```

```

rad_modif(0, INF, TERMIN, TERMIN2, RAD, RAD3, PRON) :-
    pers(NO), not(RAD=préval), not(RAD=fail),
    double_irreg(LISTE), appartenance(INIT, ELEM, LISTE, RAD),
    rad_modif_ind_prés(ELEM, TEMPS, NO, TERMIN, TERMIN2, ELEM2),
    concat(INIT, ELEM2, RAD3), !.
rad_modif(1, INF, TERMIN, TERMIN2, RAD, RAD3, PRON) :-
    appartenance(INIT, ELEM, [sav, ét, fai, pouv, av], RAD),
    calc_pos(ELEM, [sav, ét, fai, pouv, av], POS),
    repérer(ELEM2, [sach, soi, fass, puiss, ai], POS),

    concat(INIT, ELEM2, RAD3), TERMIN=TERMIN2, !.
rad_modif(1, INF, TERMIN, TERMIN2, RAD, RAD3, PRON) :-
    pers(NO), NO<4, not(RAD=préval),
    appartenance(INIT, ELEM, [all, voul, val], RAD),
    calc_pos(ELEM, [all, voul, val], POS),
    repérer(ELEM2, [aill, veuill, vail], POS),
    concat(INIT, ELEM2, RAD3), TERMIN=TERMIN2, !.
rad_modif(3, INF, TERMIN, TERMIN2, RAD, RAD3, PRON) :- pers(NO),
    double_irreg(LISTE),
    appartenance(INIT, ELEM, LISTE, RAD),
    rad_modif_imptf(ELEM, TEMPS, NO, TERMIN, TERMIN2, ELEM2),
    concat(INIT, ELEM2, RAD3), !.
rad_modif(TEMPS, INF, TERMIN, TERMIN2, RAD, RAD3, PRON) :-
    exclure(INIT, ELEM, RAD, [suiv, viv, serv, pleuv]),
    not(concat(INIT, ELEM, RAD)),
    irreg(LISTE), appartenance(INIT2, ELEM2, LISTE, RAD),
    calc_pos(ELEM2, LISTE, POS),
    select(ELEM2, TEMPS, TERMIN, ELEM3, POS, PRON),
    concat(INIT2, ELEM3, RAD3), TERMIN=TERMIN2, !.
rad_modif(TEMPS, INF, TERMIN, TERMIN2, RAD, RAD3, PRON) :-
    RAD=RAD3, TERMIN=TERMIN2.

rad_modif_ind_prés(RAD, 0, NO, TERMIN, TERMIN2, RAD3) :- not(NO>3),
    calc_pos(RAD, [fai, sav, pouv, voul], POS),
    repérer(RAD3, [fais, sai, peuv, veul], POS),
    TERMIN=TERMIN2, !.
rad_modif_ind_prés(val, 0, NO, TERMIN, TERMIN2, vaul) :-
    NO<3, TERMIN=TERMIN2, !.
rad_modif_ind_prés(all, 0, 0, s, s, vai) :- !.
rad_modif_ind_prés(all, 0, 1, s, s, va) :- !.
rad_modif_ind_prés(all, 0, 2, t, "", va) :- !.
rad_modif_ind_prés(all, 0, 3, ent, ent, va) :- !.
rad_modif_ind_prés(av, 0, 0, s, "", ai) :- !.
rad_modif_ind_prés(av, 0, 1, s, s, a) :- !.
rad_modif_ind_prés(av, 0, 2, t, "", a) :- !.
rad_modif_ind_prés(ét, 0, 0, s, "", suis) :- !.
rad_modif_ind_prés(ét, 0, 1, s, "", es) :- !.
rad_modif_ind_prés(ét, 0, 2, t, "", est) :- !.
rad_modif_ind_prés(ét, 0, 3, TERMIN, TERMIN2, es) :- TERMIN=TERMIN2, !.
rad_modif_ind_prés(ét, 0, 4, ons, "", sommes) :- !.
rad_modif_ind_prés(ét, 0, 5, ez, es, ét) :- !.
rad_modif_ind_prés(RAD, 0, NO, TERMIN, TERMIN2, RAD2) :- RAD=RAD2,
    TERMIN=TERMIN2.

```

```

rad_modif_imptf(all,3,1,e,"",va).
rad_modif_imptf(et,3,NO,TERMIN,TERMIN2,soi):- TERMIN=TERMIN2,!.
rad_modif_imptf(av,3,1,s,"",aie).
rad_modif_imptf(av,3,NO,TERMIN,TERMIN2,ai):- TERMIN=TERMIN2,!.
rad_modif_imptf(sav,3,1,s,e,sach).
rad_modif_imptf(sav,3,NO,TERMIN,TERMIN2,sach):- TERMIN=TERMIN2,!.
rad_modif_imptf(pouv,3,NO,TERMIN,"","").
rad_modif_imptf(voul,3,1,TERMIN,x,veul).
rad_modif_imptf(voul,3,NO,TERMIN,TERMIN2,voul):- TERMIN=TERMIN2,!.
rad_modif_imptf(val,3,1,TERMIN,x,vaul).
rad_modif_imptf(fai,3,NO,TERMIN,TERMIN2,fais):- TERMIN=TERMIN2,!.
rad_modif_imptf(RAD,3,_,TERMIN,TERMIN2,RAD2):-RAD=RAD2,TERMIN=TERMIN2.

```

```

select(et,1,TERMIN,ELEM2,POS,PRON):-irreg_B(LISTE),
    repérer(ELEM2,LISTE,POS),!.
select(et,2,TERMIN,et,POS,PRON):- !.
select(ELEM,TEMPS,TERMIN,ELEM2,POS,PRON):- TEMPS<5, muted_1(LISTE),
    appartient(TERMIN,LISTE), irreg_B(LISTEB),
    repérer(ELEM2,LISTEB,POS),!.
select(ELEM,TEMPS,TERMIN,ELEM2,POS,PRON):- TEMPS<5, muted_2(LISTE),
    appartient(TERMIN,LISTE), irreg_A(LISTEB),
    repérer(ELEM2,LISTEB,POS),!.
select(ELEM,5,TERMIN,ELEM2,POS,PRON):- irreg_D(LISTE),
    repérer(ELEM2,LISTE,POS),!.
select(ELEM,TEMPS,TERMIN,ELEM2,POS,PRON):- TEMPS>5, TEMPS<8,
    irreg_E(LISTE),
    repérer(ELEM2,LISTE,POS),!.
select(ELEM,TEMPS,TERMIN,ELEM2,POS,PRON):- TEMPS>7, irreg_F(LISTE),
    repérer(ELEM2,LISTE,POS),!.
select(ELEM,TEMPS,TERMIN,ELEM2,POS,PRON):- irreg_C(LISTE),
    repérer(ELEM2,LISTE,POS),!.

```

```

ir_modif(TEMPS,RAD,RAD2):- type_finir(LISTE),
    appartenance(INIT,ELEM,LISTE,RAD),
    type_finir_modif(TEMPS,RAD,RAD2),
    asserta(flag(3)),!.
ir_modif(TEMPS,RAD,RAD2):- sentir_etc(LISTE),
    appartenance(INIT2,ELEM2,LISTE,RAD), RAD=RAD2,!.
ir_modif(TEMPS,RAD,RAD2):- concat(INIT,fu,RAD), concat(INIT,fui,RAD2),
    asserta(flag(3)),!.
ir_modif(TEMPS,RAD,RAD2):- type_finir_modif(TEMPS,RAD,RAD2),
    asserta(flag(3)),!.
ir_modif(TEMPS,RAD,RAD2):- RAD=RAD2.

```

```

type_finir_modif(TEMPS,ha,hai):- pers(PERS), PERS<3, TEMPS=0; TEMPS=3,!.
type_finir_modif(TEMPS,ha,haiss):- TEMPS<5,!.
type_finir_modif(TEMPS,ha,hai):- TEMPS>5,!.
type_finir_modif(TEMPS,RAD,RAD2):- TEMPS<5, concat(RAD,iss,RAD2),!.
type_finir_modif(TEMPS,RAD,RAD2):- TEMPS>5, concat(RAD,i,RAD2),!.
type_finir_modif(TEMPS,RAD,RAD2):- RAD2=RAD.

```

/*

TRAITEMENT DE LA VOYELLE THEMATIQUE ET DE LA TERMINAISON
--

*/

```

/* ----- Insérer la voyelle thématique ----- */
affecter_voy_thém(TEMPS,"",PERS,INF,""):-!.
affecter_voy_thém(5,RAD,PERS,INF,"i"):- flag(3),
    retract(rad2(_)), retract(termin(_)),
    asserta(rad2(RAD)), asserta(termin("i")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,""):-TEMPS<6,!.
/* voyelle thématique pour FS et CD */
affecter_voy_thém(TEMPS,RAD,PERS,er,e):- not(RAD=i),TEMPS>5, TEMPS<8,
    asserta(v("e")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,e):- TEMPS>5, TEMPS<8,
    concat(INIT,cueill,RAD),
    asserta(v("e")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,i):- TEMPS>5,TEMPS<8,
    sentir_etc(LISTE),
6
    appartenance(INIT,ELEM,LISTE,RAD),
    not(ELEM=cour), asserta(v("i")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,""):-TEMPS>5, TEMPS<8,!.
affecter_voy_thém(8,RAD,3,er,è):- asserta(v("è")),!.
/* voyelle thématique pour PS et SI */
affecter_voy_thém(TEMPS,RAD,PERS,er,a):- TEMPS>7, asserta(v("a")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,i):- TEMPS>7, rad(R),
    exclure(INIT,ELEM,R,[ê,t,tai,mouv,dev,pourv,sav]),
    not(concat(INIT,ELEM,R)),
    appartenance(INIT2,ELEM2,[qu,vr,fr,pr,mp,gn,nc,ill,t,m,v,s,d,f],RAD),
    asserta(v("i")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,""):- TEMPS>7,
    appartenance(INIT,ELEM,[in,i,i,u],RAD),
    asserta(v("")),!.
affecter_voy_thém(TEMPS,RAD,PERS,INF,"u").

déplacer_chapeau(RAD3,RAD4,"","",s,s):- not(rad(croît)),
    concat(INIT,croît,RAD3),
    concat(INIT,croît,RAD4),!.
déplacer_chapeau(RAD3,RAD4,u,u,TERMIN,TERMIN1):- not(rad(croît)),
    concat(INIT,"^",RAD3),
    INIT=RAD4,TERMIN=TERMIN1,!.
déplacer_chapeau(RAD3,RAD4,V,V2,TERMIN,TERMIN1):-
    isol_elem(1,RAD3,INIT,FIN),
    FIN="^", car(VOY1,VOY2), not(V=VOY2),V=VOY1,
    V2=VOY2,INIT=RAD4,concat("^",RESTE,TERMIN),
    TERMIN1=RESTE,asserta(rad3(INIT)),!.
déplacer_chapeau(RAD3,RAD4,V,V2,TERMIN,TERMIN1):-
    isol_elem(1,RAD3,INIT,FIN),
    FIN="^", INIT=RAD4, car(VOY1,VOY2),
    V=VOY1,V2=VOY2,TERMIN=TERMIN1,
    asserta(rad3(INIT)),!.
déplacer_chapeau(haï,haï,"","",TERMIN,TERMIN1):-

```

```

concat("^",RESTE,TERMIN),TERMIN1=RESTE,!.
déplacer_chapeau(RAD3,RAD4,V,V2,TERMIN,TERMIN1):- RAD3=RAD4,
V=V2,TERMIN=TERMIN1.

```

```

/* --- Changer a --> â , -in --> -în etc, où besoin est ---- */
change_termin(RAD2,RAD3,PERS,V,TERMIN,TERMIN2):-
frontchar(TERMIN,"^",RESTE),
car(VOY1,VOY2), V=VOY1,
asserta(v2(VOY2)),concat(VOY2,RESTE,TERMIN2),
asserta(termin2(RESTE)),RAD2=RAD3,!.
change_termin(RAD2,RAD3,PERS,V,TERMIN,TERMIN2):-
frontchar(TERMIN,"^",RESTE),
concat(INIT2,in,RAD2),
concat(INIT2,in,RAD3),TERMIN2=RESTE,
asserta(rad3(RAD3)),!.
change_termin(RAD2,RAD3,PERS,"",TERMIN,TERMIN2):-
frontchar(TERMIN,"^",RESTE),
isol_elem(1,RAD2,FRONT,VOY),
car(VOY1,VOY2),
VOY=VOY1,concat(FRONT,VOY2,RAD3),
TERMIN2=RESTE,asserta(termin2(RESTE)),
asserta(rad3(RAD3)),!.
change_termin(RAD2,RAD3,0,a,s,TERMIN2):- concat(a,i,TERMIN2),
asserta(termin2(TERMIN2)),RAD2=RAD3,!.
change_termin(RAD2,RAD3,PERS,a,t,TERMIN2):- concat(a,"",TERMIN2),
asserta(termin2(TERMIN2)),RAD2=RAD3,!.
change_termin(RAD2,RAD3,PERS,V,TERMIN,TERMIN2):-
concat(V,TERMIN,TERMIN2),
asserta(termin2(TERMIN2)),RAD2=RAD3,!.

```

/*

LA MORPHOPHONOLOGIE

*/

```

morphoph(TEMPS,RAD,RAD2,PERS,er,TERMIN,TERMIN2):-
adoucir(RAD,RAD2,TERMIN,TERMIN2),!.
morphoph(TEMPS,RAD,RAD2,PERS,er,TERMIN,TERMIN2):- PERS<4,
groupe_1(TEMPS,PERS,RAD,RAD2,TERMIN,TERMIN2),!.
morphoph(TEMPS,RAD,RAD2,PERS,INF,TERMIN,TERMIN2):-
groupe_3(TEMPS,RAD,RAD2,TERMIN,TERMIN2).

```

```

/* 1.      c --> ç      g --> ge      */
adoucir(RAD2,RAD3,TERMIN3,TERMIN4):- cons_douce(FIN,FIN2),
concat(FRONT,FIN,RAD2),
frontstr(1,TERMIN3,INITIAL,FIN3),
appartient(INITIAL,[a,o,â]),
concat(FRONT,FIN2,RAD3),TERMIN4=TERMIN3,
asserta(change(FIN,FIN2,INITIAL)),asserta(rad3(RAD3)),!.

```

/* Traitement de la 1e conjugaison */

```

/* type "eler_eter" */
groupe_1(TEMPS,PERS,RAD,RAD2,TERMIN2,TERMIN3):-
    isol_elem(2,RAD,FRONT,END),
    frontstr(1,END,V,C),V=e, appartient(C,[l,t]),
    eler_eter(LISTE2),not(appartient(RAD,LISTE2)),
    concat(C,C,CC), concat(V,CC,END2),concat(FRONT,END2,RAD2),
    asserta(change(C,CC,TERMIN2)),
    TERMIN2=TERMIN3,!.

/* type "paser" */
groupe_1(TEMPS,PERS,RAD,RAD2,TERMIN2,TERMIN3):-
    appartenance(INIT,ELEM,[el,et,ec,em,en,ep,er,es,ev,evr1,RAD]),
    frontchar(ELEM,'e',FIN), concat("è",FIN,ELEM2),
    concat(INIT,ELEM2,RAD2), TERMIN2=TERMIN3,
    asserta(change("e","è",TERMIN2)),!.

/* type "céder" */
groupe_1(TEMPS,PERS,RAD,RAD2,TERMIN2,TERMIN3):- TEMPS<6,
    isol_elem(2,RAD,FRONT,END),
    frontstr(1,END,FRONT2,END2), FRONT2=é,
    concat(è,END2,FIN2), concat(FRONT,FIN2,RAD2),
    asserta(change(END,FIN2,TERMIN2)), TERMIN2=TERMIN3,!.

/* type "régler" */
groupe_1(TEMPS,PERS,RAD,RAD2,TERMIN2,TERMIN3):-
    isol_elem(3,RAD,FRONT,END),
    not(END=éci), not(END=éfi),not(END=éri),
    frontstr(1,END,FRONT2,END2), FRONT2=é,TEMPS<6,
    concat(è,END2,FIN2), concat(FRONT,FIN2,RAD2),
    asserta(change(END,FIN2,TERMIN2)), TERMIN2=TERMIN3,!.

/* type "envoyer" */
groupe_1(TEMPS,PERS,RAD,RAD3,TERMIN2,TERMIN3):- TEMPS>5,TEMPS<8,
    frontstr(1,TERMIN2,INIT,FIN), INIT=e,
    concat(FRONT,envoy,RAD), concat(FRONT,enver,RAD3),
    asserta(change("", "Ø",e)),
    asserta(rad3(RAD3)),concat("",FIN,TERMIN3),
    asserta(termin3(TERMIN3)),!.

/* type "payer" (THINK ALSO ABOUT THE ALTERNATIVE CONJUG FOR -AYER) */
groupe_1(TEMPS,PERS,RAD,RAD2,TERMIN2,TERMIN3):-
    not(appartient(RAD,[grassey,longuey,fasey,capey])),
    concat(FRONT,y,RAD),
    mutes_1(LISTE), appartient(TERMIN2,LISTE),
    concat(FRONT,i,RAD2),
    asserta(change(y,i,TERMIN2)), TERMIN2=TERMIN3,!.

/* Le cas général */
groupe_1(TEMPS,PERS,RAD,RAD2,TERMIN2,TERMIN3):- RAD=RAD2,
    TERMIN2=TERMIN3.

```

```

/* Traitement de la 3e conjugaison */

groupe_3(TEMPS,RAD,RAD3,TERMIN,TERMIN2):- isol_elem(1,RAD,RA,D),
    supprimer(TEMPS,RAD,RAD3,RA,D,TERMIN,TERMIN2).

/* ----- "fui + ons ---> fuyons" etc ----- */
groupe_3(TEMPS,RAD2,RAD3,TERMIN3,TERMIN4):- isol_elem(1,RAD2,INIT1,FIN),
    FIN="i", not(FIN="i"),
    exclure(INIT2,ELEM1,RAD2,[li,hi,ci,fi,ri]), /* exclure apprecier etc */
    not(concat(INIT2,ELEM1,RAD2)),
    appartient(ELEM2,[a,ez,i,ol]),
    concat(ELEM2,END,TERMIN3),
    concat(INIT1,y,RAD3),TERMIN4=TERMIN3,
    asserta(change(i,y,ELEM2)),asserta(rad3(RAD3)),!.

/* residu */
groupe_3(TEMPS,RAD,RAD3,TERMIN,TERMIN2):-
    residu(RAD,TERMIN,RAD3,TERMIN2),!.

/* ----- autres */
groupe_3(TEMPS, RAD,RAD2,TERMIN3,TERMIN4):- RAD=RAD2,TERMIN4=TERMIN3.

/* La suppression de la/les dernière(s) consonne(s) */

supprimer(TEMPS,RAD,RAD3,RA,D,TERMIN,TERMIN2):-TEMPS<6,
    appartient(TERMIN,[s,x,t]),
    suppr1(RAD2,RA,D,TERMIN),
    suppr2(RAD2,RAD3,TERMIN,TERMIN2),!.
supprimer(TEMPS,RAD,RAD2,RA,D,TERMIN,TERMIN2):-TEMPS>5,TEMPS<8,
    suppr3(RAD2,RA,D,TERMIN),TERMIN=TERMIN2.

/* Indicatif présent */

/* première passe */
suppr1(RAD2,RA,D,TERMIN):- appartient(D,[v,s,l,m,t]),
    exclure(INIT,ELEM,RA,[vê,gi]),
    not(concat(INIT,ELEM,RA)),RAD2=RA,!.
suppr1(RAD2,RA,d,TERMIN):- isol_elem(2,RA,R,A), A=in, RAD2=RA,!.
suppr1(RAD2,RA,D,TERMIN):- concat(RA,D,RAD),RAD=RAD2.

/* deuxième passe */
suppr2(RAD2,RAD3,t,""):- isol_elem(1,RAD2,RA,D), appartient(D,[d,c,t]),
    concat(RA,D,RAD),RAD3=RAD,!.
suppr2(crois,croî,T,T2):- T=T2,!.
suppr2(RAD2,RAD3,t,t):- not(flag(3)), concat(INIT,is,RAD2),
    concat(INIT,i,RAD3), !.
suppr2(RAD2,RAD3,t,t):- concat(INIT,plai,RAD2),
    concat(INIT,plai,RAD3),!.
suppr2(clo,clô,t,t):-!.
suppr2(RAD2,RAD3,T,T2):- concat(INIT,s,RAD2), RAD3=INIT,T=T2,!.
suppr2(RAD2,RAD3,T,T2):- RAD2=RAD3,T=T2.

```

```

/* futur et conditionnel */
suppr3(RAD2,RA,s,TERMIN):- RAD2=RA,!.
suppr3(RAD2,RA,v,TERMIN):- concat(INIT,cri,RA),RAD2=RA,!.
suppr3(RAD2,RA,D,TERMIN):- concat(RA,D,RAD2).

/* residu */
residu(va,ent,vont,""):- asserta(change2("va + ent","vont")),!.
residu(fais,ent,font,""):- asserta(change2("fais + ent","font")),!.
residu(fais,ez,faites,""):- asserta(change2("fais + ez","faites")),!.
residu(es,ent,sont,""):- asserta(change2("es + ent","sont")),!.
residu(es,ez,etes,""):- asserta(change2("es + ez","etes")),!.
residu(dis,ez,dites,""):- asserta(change2("dis + ez","dites")),!.
residu(redis,ez,redites,""):-asserta(change2("redis + ez","redites")),!.
residu(av,ent,ont,""):- asserta(change2("av + ent","ont")),!.
residu(ai,ions,ay,ons):- asserta(change2("ai + ions","ayons")),!.
residu(ai,iez,ay,ez):- asserta(change2("ai + iez","ayez")),!.

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

LA BOITE A OUTILS

```

*/

/* ----- Outils pour le traitement des listes ----- */

appartient(X,[X!_]).
appartient(X,[_!L]):- appartient(X,L).

repérer(X,[X!_],1):-!.
repérer(X,[_!Q],N):- N>1, N1=N-1, repérer(X,Q,N1).

10
calc_pos(X,[X!_],1).
calc_pos(X,[_!L],N):- calc_pos(X,L,Y),N=Y+1.

/* ----- Outils pour la manipulation des éléments ----- */

isol_elem(Z,RAD,INIT,FIN):- str_len(RAD,X), Y=X-Z,
                             frontstr(Y,RAD,INIT,FIN).

appartenance(INIT,ELEM,LISTE,RAD):- appartient(ELEM,LISTE),
                                     concat(INIT,ELEM,RAD).

exclure(INIT,ELEM,RAD,LISTE):- appartenance(INIT,ELEM,LISTE,RAD),!.
exclure(xxx,xxx,RAD,LISTE).

```

/----- DIVERS -----/

```
clear:- retract(tps(_)),fail.
clear:- retract(rad(_)),fail.
clear:- retract(rad2(_)),fail.
clear:- retract(rad3(_)),fail.
clear:- retract(inf(_)),fail.
clear:- retract(termin(_)),fail.
clear:- retract(termin2(_)),fail.
clear:- retract(termin3(_)),fail.
clear:- retract(prn(_)),fail.
clear:- retract(v(_)),fail.
clear:- retract(v2(_)),fail.
clear:- retract(change(_,_)),fail.
clear:- retract(change2(_,_)),fail.
clear:- retract(conj_forme(_)),fail.
clear:- retract(espace(_)),fail.
clear:- retract(flag(_)),fail.
clear.
```

```
repeat.
repeat:- repeat.
```

```
delay(0):-!.
delay(X):- X > 0, Y=X-1, delay(Y).
```

```
chk_pron(ASPECT,PERS,PRON," "):- ASPECT>3, ASPECT<6,!.
chk_pron(3,PERS,PRON,PRON2):- not(PERS=0),PRON=PRON2,!.
chk_pron(3,PERS,PRON,PRON2):- not(PERS=2),PRON=PRON2,!.
chk_pron(3,PERS,PRON,PRON2):- not(PERS=3),PRON=PRON2,!.
chk_pron(ASPECT,PERS,PRON,PRON2):- PRON=PRON2.
```

```
classifier(INF,RAD,er):- concat(RAD,er,INF),!.
classifier(INF,RAD,oir):- concat(RAD,oir,INF),!.
classifier(INF,RAD,ir):- concat(RAD,ir,INF),!.
classifier(INF,RAD,re):- concat(RAD,re,INF),!.
classifier(INF,RAD,ir):- concat(RAD,ir,INF),!.
classifier(INF,"",""):-
    write("\n          >> Mot non reconnu <<"),
    write("\n    Veuillez retapez avec l'INFINITIF du verbe."),
    write("\n          Appuyez sur ESPACE"),
    readchar(C), C='Q'.
```

LA FENETRE "ANALYSE"

```

analyse:- shiftwindow(3),prn (PRON),rad (RAD),inf (TERM),
        cursor(0,0),cursor (LIN,COL),LIN1=LIN+1, COL1=COL+3,
        cursor (LIN1,COL1), write (PRON," ",RAD," - ",TERM),fail.
analyse:- rad2(RAD2), termin (TERMIN),
        cursor (LIN,COL),LIN1=LIN+1,
        prn (PRON),str_len (PRON,LEN),COL1=COL-COL+LEN+4,cursor (LIN1,COL1),
        write (RAD2," + ",TERMIN),fail.
analyse:- v(V),rad2(RAD2),termin (TERMIN),
        cursor (LIN,COL),LIN1=LIN+1,
        prn (PRON),str_len (PRON,LEN),COL1=COL-COL+LEN+4,cursor (LIN1,COL1),
        write (RAD2," + ",V," + ",TERMIN),fail.
analyse:- v2(V2), termin2 (TERMIN2),
        cursor (LIN,COL), LIN1=LIN+1,
        prn (PRON), str_len (PRON,LEN),
        rad2(RAD2), str_len (RAD2,LEN2), COL1=COL-COL+4+LEN+LEN2+3,
        cursor (LIN1,COL1),write (V2," + ",TERMIN2),fail.
analyse:- change(X,Y,Z), cursor (LIN,COL),LIN1=LIN+1,
        prn (PRON), str_len (PRON,LEN), str_len (X,LEN3),
        rad2(RAD2), str_len (RAD2,LEN2), COL1=COL-COL+4+LEN+LEN2-LEN3,
        cursor (LIN1,COL1), write (X," --> ",Y,"/_",Z),fail.
analyse:- change2(X,Y), cursor (LIN,COL),LIN1=LIN+1,
        prn (PRON), str_len (PRON,LEN), str_len (X,LEN3),
        rad2(RAD2), str_len (RAD2,LEN2), COL1=COL-COL+4+LEN+LEN2-LEN3+4,
        cursor (LIN1,COL1), write (X," --> ",Y),fail.
analyse:- rad3(RAD3), termin3 (TERMIN3),
        cursor (LIN,COL),LIN1=LIN+1,
        prn (PRON),str_len (PRON,LEN),COL1=COL-COL+LEN+4,cursor (LIN1,COL1),
        write (RAD3," + ",TERMIN3),fail.
analyse:- rad3(RAD3), termin2 (TERMIN2), v2(V2),
        cursor (LIN,COL),LIN1=LIN+1,
        prn (PRON),str_len (PRON,LEN),COL1=COL-COL+LEN+4,cursor (LIN1,COL1),
        write (RAD3," + ",V2," + ",TERMIN2),fail.
analyse:- conj_forme(CONJ_FORME),
        cursor (LIN,COL),LIN1=LIN+2, prn (PRON),str_len (PRON,LEN),
        COL1=COL-COL+LEN+4,cursor (LIN1,COL1), write (CONJ_FORME),
        str_len (CONJ_FORME,X), field_attr (LIN1,COL1,X,113),!.

```

/*

//

TERMINAISONS, PRONOMS, LISTES

LISTES

*/

```

eler_eter((model, cel, décel, recel, cisel, démantel, écartel, se_encastel, gel, dégel,
          congel, surgel, martel, pel, achat, rachat, béguet, corset, crochet, filet,
          furet, halet)).
type_finir((ert, lent, retent, amort, assort, asserv, fepart, répart, tart, queill, vieill, jaill)).
sentir_etc((lent, rt, vêt, ouvr, ffr, ill, dorm, cour, serv)).
part_passé1((cour, croi, ill, r, rv, nt, rt, pleuv, fu, fall, ch, clu, uiv, viv, li, ri, plai, tai)).
part_passé2a((vêt, ouvr, ffr, asse, surse, ind, nd, rd, mp, batt, mett, rai, di, cri, fi, circonci,
              fri, uil)).
part_passé2b((vêtu, ouvert, ffert, assis, sursis, int, ndu, rdu, mp, battu, mis, rait, dit, crit, fit,
              circoncis, frit, uit)).
prennent_être((adven, all, arriv, croît, croul, croup, débond, décéd, deven, éch, enlaid, entr,
               interven, mour, naît, obven, part, parv, prov, redeven, reentr, repaît, repart,
               ressort, rest, retomb, retour, reven, surven, tomb, ven)).
re_modif1((lui, ui, ci, fi, fri, cri, di, li, viv, croi, plai, rai, tai, mett)).
re_modif2((lui, uis, cis, f, fr, criv, d, l, véc, cr, pl, "ø", t, a)).

```

/* ----- RADICAUX MODIFIES ----- */

```

IRREG((gés, chal, fall, all, en, quér, prend, boi, soud, coud, moud, pourv, cev, dev, croît, repaît, paît,
       connaît, naît, aît,
       vainc, bouill, surse, asse, ind, mour, mouv, pouv, sav, vout, préval, val, fai, av, èt, faill, v)).

irreg_A((gis, chau, fau, va, ien, quier, prend, boiv, sou, coud, moud, pourvoi, çoi, doiv, croît, repaiss,
        paiss, connaît, naiss, aiss,
        vainc, bou, sursoi, assied, in, meur, meur, peuv, sai, veul, prévau, vau, fai, av, es, "ø", voi)).
irreg_B((f", chaill, faill, aill, ienn, quier, prenn, boiv, solv, cous, moul, pourvoi, çoi, doiv, croiss,
        repaiss, paiss, connaît, naiss, aiss, vainqu, bouill, sursoi, assey, ign, meur, meur, puiss, sach,
        veuill, préval, vaill, fass, ai, soi, "ø", voi)).
irreg_C((gis, chal, fall, all, en, quér, pren, buv, solv, cous, moul, pourvoy, cev, dev, croiss, repaiss,
        paiss, connaît, naiss, aiss, vainqu, bouill, sursoi, assey, ign, mour, mouv, pouv, sav, vout, préval,
        val, fais, av, "EST", "ø", voi)).
irreg_D((f", "f", fallu, allé, enu, quis, pris, bu, sous, cousu, moulu, pourvu, çu, dû, crû, repu, "ø", connu,
        né, u, vaincu, bouilli, sursis, assis, int, mort, mû, pu, su, voulu, prévalu, valu, fait, eu, été,
        failli, vul)).
irreg_E((f", chaud, faud, i, iend, quer, prend, boi, soud, coud, moud, pourvoi, cev, dev, croît, repaît, paît,
        connaît, naît, aît, vainc, bouill, surseoi, assié, ind, mour, mouv, pour, sau, vout, prévau, vaud,
        fe, au, se, failli, ver)).
irreg_F((f", "f", fall, all, in, qu, pr, b, "ø", cous, moul, pourv, ç, d, "cr", rep, "ø", conn, naqu, "f",
        vainqu, bouill, surs, ass, ign, mour, m, p, s, vout, préval, val, f, e, f, faill, v)).

double_irreg((all, sav, èt, fai, pouv, av, vout, val)).

```

```
/* Temps, personnes, messages */
```

```
table1(0,"INDICATIF présent").  
table1(1,"SUBJONCTIF présent").  
table1(2,"INDICATIF imparfait").  
table1(3,"IMPERATIF").  
table1(4,"PARTICIPE présent").  
table1(5,"PARTICIPE passé").  
table1(6,"INDICATIF futur").  
table1(7,"CONDITIONNEL").  
table1(8,"INDICATIF passé simple").  
table1(9,"SUBJONCTIF imparfait").  
pron("je",0,"1sg").  
pron("tu",1,"2sg").  
pron("il",2,"3sg").  
pron("nous",4,"1pl").  
pron("vous",5,"2pl").  
pron("ils",3,"3pl").
```

```
/* Terminaisons */
```

```
terminaison(0,"e",0).  
terminaison(0,"es",1).  
terminaison(0,"e",2).  
terminaison(0,"ons",4).  
terminaison(0,"ez",5).  
terminaison(0,"ent",3).  
terminaison(1,"e",0).  
terminaison(1,"es",1).  
terminaison(1,"e",2).  
terminaison(1,"ions",4).  
terminaison(1,"iez",5).  
terminaison(1,"ent",3).  
terminaison(2,"ais",0).  
terminaison(2,"ais",1).  
terminaison(2,"ait",2).  
terminaison(2,"ions",4).  
terminaison(2,"iez",5).  
terminaison(2,"aient",3).  
terminaison(3,"e",1).  
terminaison(3,"ons",4).  
terminaison(3,"ez",5).  
terminaison(6,"rai",0).  
terminaison(6,"ras",1).  
terminaison(6,"ra",2).  
terminaison(6,"rons",4).  
terminaison(6,"rez",5).  
terminaison(6,"ront",3).  
terminaison(8,"s",0).  
terminaison(8,"s",1).  
terminaison(8,"t",2).  
terminaison(8,"^mes",4).  
terminaison(8,"^tes",5).  
terminaison(8,"rent",3).  
12  
terminaison(9,"sse",0).
```

LISTINGS DES MODULES

LE NOYAU

```
terminaison(9,"sses",1).
terminaison(9,"^t",2).
terminaison(9,"ssions",4).
terminaison(9,"ssiez",5).
terminaison(9,"ssent",3).
terminaison2(0,"s",0).
terminaison2(0,"s",1).
terminaison2(0,"t",2).
terminaison2(0,"ons",4).
terminaison2(0,"ez",5).
terminaison2(0,"ent",3).
terminaison2(3,"s",1).
terminaison2(3,"ons",4).
terminaison2(3,"ez",5).

/* quelques particularités */
curieux(5,"inclu","inclus").
curieux(5,"occlu","occlus").
curieux(5,"résous","résolu").
curieux(5,"maudi","maudit").
curieux(5,"suffit","suffi").
curieux(5,"luit","lui").
curieux(5,"nuit","nui").

/* détails de la morphophonologie */
car("a","â").
car("i","î").
car("u","û").
cons_douce("c","ç").
cons_douce("g","ge").
mutes_1(["e","es","ent"]).
mutes_2(["ent","s","t","x"]).

/* variantes */
variantes(voul,3,1,veille).
variantes(voul,3,5,veuillez).
variantes(pouv,0,0,puis).
```

INTERRO ET GENERATION

```

/*                      last modified on 18.9.91 */
PREDICATES
chk_flag(integer)

/* ----- pré-traitement ----- */
saisir(symbol,integer,symbol,symbol)
couper(symbol,symbol,symbol,integer)
ster_espace(symbol,symbol)

choose(symbol,integer,symbol,integer,symbol,integer)
aléa(symbol,integer,symbol,integer,integer)
chk_resp(integer,symbol,symbol)
chk_conj_forme(integer,integer,symbol,symbol,symbol)

/* ----- PREDICATS POUR CALCULER LE SCORE ----- */
tallycorrect testconj(symbol,symbol)
tallywrong    count      bravo

CLAUSES
/* XXXXXXXXXXXXXXXX LA GENERATION XXXXXXXXXXXXXXXXXXXXXXXX */

génération(ASPECT,FLAG):- repeat,
    clear,
    window6,
    table1(ASPECT,MESSAGE),
    str_len(MESSAGE,LEN),
    shiftwindow(6), cursor(0,36),
    write("MODE: Génération - ",MESSAGE),
    shiftwindow(4),clearwindow,
    shiftwindow(5),clearwindow,
    shiftwindow(3),clearwindow,
    shiftwindow(2),clearwindow,
    cursor(0,1),
    chk_flag(FLAG),nl,nl,
    write(">> "),
    saisir(PRON,PERS,RAD,TERM),cursor(LI,CO),
    noyau(ASPECT,RAD,TERM,PRON,PERS,CONJ_FORME),
    chk_cas_spéc(PRON,CONJ_FORME,PRON2,CONJ_FORME2),nl,nl,
    write(">>> "),
    invariable(ASPECT,INVAR,CONJ_FORME2,ALTERN),
    particularité5(ASPECT,CONJ_FORME2,CONJ_FORME3),
    output(ASPECT,PERS,INVAR,PRON2,CONJ_FORME3,ALTERN),
    analyse, shiftwindow(4),clearwindow,
    readinfo,shiftwindow(2),cursor(LI,CO),
    readchar(ESP), char_int(ESP,27).
génération(ASPECT,FLAG).

chk_flag(1):- write("Entrez INFINITIF + PRONOM, eg >> venir je"),
    cursor(LI,CO),CO2=CO-8, field_attr(LI,CO2,8,16),!.
chk_flag(2):- write("Entrez seulement l'INFINITIF (pas de pronom)"),
    field_attr(0,20,9,155),!.
chk_flag(3):- write("Entrez INFINITIF + PRONOM"), nl,
    write("PRONOM ne peut être que 'tu', 'nous' ou 'vous'").

```

```
readinfo:- vb(VERB), particularité(VERB,MESSAGE),!,
           write(MESSAGE),!. /%try fail here to stop persistence of message%/
readinfo.
```

```
/* ----- SAISIR LE VERBE ----- */
saisir (PRON,NO,RAD,TERM):- readln(GROUPE),
                           upper_lower(GROUPE,GROUPE2),
                           couper (GROUPE2,VB,PRON,NO),asserta(vb(VB)),
                           classifie (VB,RAD,TERM),!.

saisir("",00,"","").
```

```
couper (GROUPE2,VB,PRN,NO):- fronttoken(GROUPE2,VB,PRON),
                             ôter_espace (PRON,PRN),
                             prn (PRN,NO,_),!.
couper (GROUPE2,VB,PRN,NO):- GROUPE2=VB, NO=0, PRN="",!.
```

```
ôter_espace (PRON,PRN):- frontchar (PRON,' ',PRN),!.
ôter_espace (PRON,PRN):- PRN=PRON.
```

```
/* ----- MODULE INTERRO ----- */
```

```
interro:- shiftwindow(2), shiftwindow(3), shiftwindow(4),shiftwindow(5),
          window6,shiftwindow(6),cursor (0,56),
          write("MODE: Interro"),      interro_menu.
```

```
interro_loop (ASPECT,FLAG):-
    shiftwindow(4),clearwindow,
    shiftwindow(3),clearwindow,
    shiftwindow(2),clearwindow,
    aléa (VB,ASPECT,PRON,PERS,FLAG),
    readln(REPONSE),
    testconjug (REPONSE,CONJ_FORME),
    count,
    analyse, shiftwindow(4),clearwindow,
    readinfo,
    readchar (RESP), clear, char_int (RESP,27).
```

```
interro_loop (ASPECT,FLAG):- interro_loop (ASPECT,FLAG).
```

```
testconjug (REPONSE,CONJ_FORME):- conj_forme (CONJ_FORME),REPONSE=CONJ_FORME,
                                     bravo,!.
```

```
testconjug (REPONSE,CONJ_FORME):- tallywrong, nl, cursor (LIN,COL),
    write("          Désolé, incorrect."),nl,
    field_attr (LIN,COL,32,30), cursor (LIN2,COL2),
    write(" Appuyer sur une touche pour la bonne réponse"),
    field_attr (LIN2,COL2,46,30), readchar (_),
    conj_forme (CONJ_FORME), str_len (CONJ_FORME,X),
    prn (PRON),              str_len (PRON,Y),
    str_len (REPONSE,Z), LIN3=LIN-2, COL3=X+Y+8,
    field_str (LIN3,COL3,X,CONJ_FORME),
    field_attr (LIN3,COL3,X,16).
```

```
testconjug (_,_):- fail.
```

```

aléa(VB,ASPECT,PRON,PERS,FLAG):- clear,
    choose(VB,ASPECT,MESSAGE,PERS,PRON,FLAG),
    chk_pron(ASPECT,PERS,PRON,PRON2),
    classifier(VB,RAD,TERM),
    noyau(ASPECT,RAD,TERM,PRON2,PERS,CONJ_FORME),
    cursor(1,2), str_len(VB,LEN),
    write(" ",VB," --- ",MESSAGE),
    field_attr(1,3,LEN,16),nl,nl,
    write(">> ",PRON2," "),
    asserta(vb(VB)),!.

aléa(VB,ASPECT,PRON,PERS,FLAG):- aléa(VB,ASPECT,PRON,PERS,FLAG).

choose(VB,ASPECT,MESSAGE,PERS,PRON,FLAG):- FLAG=1,
/* choose own ASP */      random(A), X=A*476, verb(X,VB),
/* random prons */      table1(ASPECT,MESSAGE),
                        random(B), PERS=B*6, pron(PRON,PERS,_),!.

choose(VB,ASPECT,MESSAGE,PERS,PRON,FLAG):- FLAG=2,
/* All random. */      random(A), X=A*476, verb(X,VB),
/* ASP <6 */          random(B), ASPECT=B*5.5, table1(ASPECT,MESSAGE),
                        random(C), PERS=C*6, pron(PRON,PERS,_),!.

choose(VB,ASPECT,MESSAGE,PERS,PRON,FLAG):- FLAG=3,
/* All random. */      random(A), X=A*476, verb(X,VB),
/* ASP <8 */          random(B), ASPECT=B*7.5, table1(ASPECT,MESSAGE),
                        random(C), PERS=C*6, pron(PRON,PERS,_),!.

choose(VB,ASPECT,MESSAGE,PERS,PRON,FLAG):- FLAG=4,
/* All random.*/      random(A), X=A*476, verb(X,VB),
/* Full range*/      random(B), ASPECT=B*10, table1(ASPECT,MESSAGE),
                        random(C), PERS=C*6, pron(PRON,PERS,_).

bravo:- shiftwindow(2),
    field_str(5,10,24,"Bravo! La bonne réponse."),
    field_attr(5,10,24,30),
    tallycorrect.

tallycorrect:- storecorrect(OLDScore),!, NEWScore=OLDScore+1,
    retract(storecorrect(_)),
    asserta(storecorrect(NewScore)),
    shiftwindow(5),
    cursor(2,1),write("VRAI ",NewScore),
    cursor(3,1),write("FAUX "),
    cursor(5,1),write("TOTAL "),
    shiftwindow(2),!.

tallywrong:- storewrong(OLDScore),!, NEWScore=OLDScore+1,
    retract(storewrong(_)),
    asserta(storewrong(NewScore)),
    shiftwindow(5),
    cursor(2,1),write("VRAI "),
    cursor(3,1),write("FAUX ",NewScore),
    cursor(5,1),write("TOTAL "),
    shiftwindow(2),!.

```

```

count:- storecorrect(SCORE1),!, storewrong(SCORE2),!,
        TOTAL=SCORE1+SCORE2,
        PERCTGE=SCORE1/TOTAL,
        PERCENTAGE=PERCTGE*100,
        shiftwindow(5), cursor(5,1),
        write("TOTAL ",TOTAL), cursor(7,5),
        writef("%3.0",PERCENTAGE),write("%"),
        shiftwindow(2).

choisir_verbe(ASPECT):- repeat,shiftwindow(1), clearwindow, clear,
        table1(ASPECT,MESSAGE),
        cursor(1,30),write(MESSAGE),
        cursor(2,1),write("Entrez infinitif >> "), readln(VB),nl,
        choisir_verbe2(ASPECT,VB),/count,*/ readchar(C), char_int(C,27).
choisir_verbe(_).

choisir_verbe2(ASPECT,VB):- cursor(4,0),
        pron(PRON,PERS,_),
        write(PRON," "), cursor(LI,CO),
        classifier(VB,RAD,TERM), asserta(vb(VB)),
        noyau(ASPECT,RAD,TERM,PRON,PERS,CONJ_FORM),
        chk_conj_forme(ASPECT,PERS,PRON,CONJ_FORM,CONJ_FORM2),
        readln(RESPONSE), cursor(LI,20),
        chk_resp(LI,RESPONSE,CONJ_FORM2),
        nl, fail.
choisir_verbe2(_, _).

chk_conj_forme(ASPECT,PERS,PRON,CONJ_FORM,CONJ_FORM3):-
        curieux(ASPECT,BAS_FORM,CONJ_FORM2),
        concat(INIT,BAS_FORM,CONJ_FORM),
        concat(INIT,CONJ_FORM2,CONJ_FORM3),!.
chk_conj_forme(ASPECT,PERS,PRON,CONJ_FORM,CONJ_FORM2):- CONJ_FORM=CONJ_FORM2.

chk_resp(LI,RESP,CONJ):- RESP=CONJ, cursor(LI,28), write("Bravo!"),
        field_attr(LI,28,6,28), delay(1000),!.
chk_resp(LI,RESP,CONJ):- cursor(LI,28), write("Incorrect \26 "),
        field_attr(LI,28,11,28),write(CONJ), delay(1000).

```

CONJUGAISON COMPLETE

PREDICATES

```
conjug_compl_loop
afficher(symbol)
chk_ligne(integer, integer, integer, integer, integer)
pos_curs(integer, symbol, symbol, integer)
/*chk_aspect(integer, symbol, symbol)*/
long(symbol, integer)
branch(symbol)
```

CLAUSES

```
conjug_compl:- makewindow(8,80,7,"INFO",15,46,10,34), conjug_compl_loop.
```

```
conjug_compl_loop:- repeat,
    window6,
    shiftwindow(1), window_attr(112), clearwindow, cursor(0,27),
    write("La Conjugaison Complète"),
    field_attr(0,27,23,111), cursor(9,26),
    write("Entrez l'infinitif du verbe"),
    field_attr(9,26,27,112),
    cursor(10,24), write(" "),
    cursor(11,24), write(" "),
    cursor(12,24), write(" "),
    cursor(11,26),
    /*
    branch(VERBE),*/
    readln(VERBE),
    long(VERBE,FLAG), FLAG=0,
    asserta(vb(VERBE)),
    clearwindow, cursor(22,0),
    write(" ESPACE: encore          ESC: menu"),
    field_attr(22,0,46,7),
    field_attr(22,1,6,15),
    field_attr(22,24,3,15),
    cursor(0,0),
    afficher(VERBE),
    /*
    shiftwindow(8), clearwindow,
    readinfo,*/
    readchar(CHAR), char_int(CHAR,27).
conjug_compl_loop.
```

```
branch(_):- readln(X), X="", list_verbs,!. /* to give user option to choose from verb list*/
branch(VERBE):- readln(VERBE),!.
```

```
long(VERBE,FLAG):- str_len(VERBE,LEN), LEN>14, cursor(13,26),
    write("Votre verbe est trop long."), cursor(14,20),
    write("Max = 14 caractères. Veuillez retaper."), cursor(16,30),
    write("Appuyez sur espace"),
    readchar(_), FLAG=1,!.
long(_,0).
```

LISTINGS DES MODULES

CONJUGAISON COMPLETE

```

afficher(VB):- table1(TEMPS,MESSAGE),
               cursor(LIN,COL),
               chk_ligne(TEMPS,LIN,COL,LIN2,COL2),
               cursor(LIN2,COL2),
               write(MESSAGE),str_len(MESSAGE,LEN),
               field_str(LIN2,COL2,LEN,MESSAGE),
               field_attr(LIN2,COL2,LEN,29),
               cursor(L,C),L2=L+1,C2=C-LEN,
               cursor(L2,C2),
               pron(PRON,PERS,_),
               chk_pron(TEMPS,PERS,PRON,PRON2),
               classifie(VB,RAD,INF),
               noyau(TEMPS,RAD,INF,PRON2,PERS,FORME_CONJ),
               asserta(flag(4)),
               chk_cas_spec(PRON2,FORME_CONJ,PRON3,FORME_CONJ2),
               invariable(TEMPS,INVAR,FORME_CONJ2,ALTERN),
               particularites(TEMPS,FORME_CONJ2,FORME_CONJ3),
               auxiliaire(RAD,TEMPS,AUX),
               output(TEMPS,PERS,INVAR,PRON3,FORME_CONJ3,ALTERN,AUX),
               conj_forme(FORME_CONJ3),
               pos_curs(TEMPS,FORME_CONJ3,PRON3,PERS),
               clear, fail.

afficher(_).

/* ----- IRREGULARITES ET MESSAGES ----- */

/* 1. retourne ">inusité" en mode GENERATION si la forme n'existe pas */
chk_cas_spec(PRON,FORME_CONJ,"",>inusité):- flag(4),rad2(RAD),
        concat(INIT,">",RAD), retract(conj_forme(_)),
        asserta(conj_forme(">inusité")),!.

/* 2. retourne "" (vide) en mode INTERRO si la forme n'existe pas */
chk_cas_spec(PRON,FORME_CONJ,"",<):- rad2(RAD),
        concat(INIT,"<",RAD), message,!.

/* 3. le cas général */
chk_cas_spec(PRON,FORME_CONJ,PRON2,FORME_CONJ2):- PRON2=PRON,
        FORME_CONJ2=FORME_CONJ.

message:- tps(TEMPS), vb(VB),prn(PRN),table1(TEMPS,MESSAGE),
        pron(PRN,_PERS),clearwindow, cursor(2,3),
        write(VB," n'existe pas en"),nl,
        str_len(VB,LEN),field_attr(2,3,LEN,90),
        cursor(3,3),write(PERS," ",MESSAGE),readchar(C),clear,C='m',!.
message.

auxiliaire(RAD,5,"étant"):- prennent_être(LISTE), appartient(RAD,LISTE),!.
auxiliaire(RAD,5,"ayant"),!.
auxiliaire(RAD,TEMPS,"").

```

```

/* irrégularités du participe passé, par exemple "inclu" --> "inclus" */
particularité5(S,FORME_CONJ,FORME_CONJ2):-
    curieux(TEMPS,BAS_FORM,FORME_CONJ2),
    concat(INIT,BAS_FORM,FORME_CONJ),
    concat(INIT,FORME_CONJ2,FORME_CONJ3),
    retract(conj_forme(_)),
    asserta(conj_forme(FORME_CONJ3)),!.
particularité5(S,FORME_CONJ,FORME_CONJ2):- FORME_CONJ=FORME_CONJ2.

/* PP est-il invariable? */
invariable(S,"invar",C,C2):- vb(VB), /*liste de verbes à PP invariable */
    appartient(VB,[être,défaillir,faillir,dormir,redormir,pleuvoir,repleuvoir,
        falloir,plaire,complaire,déplaire,paître,survivre,suffire,
        luire,nuire]),C=C2,!.
/* pour le féminin, ajoute 'e', à l'exception de -soudre (sauf 'résoudre')*/
invariable(S,"e",FORME_CONJ,sout):- concat(INIT,sous,FORME_CONJ),!.
invariable(S,"e",C,C2):- C=C2.

/* ===== POSITION CURSOR ON SCREEN ===== */
/*----- reposition cursor after each person of conjugation -----*/
pos_curs(TEMPS,CONJFORM,PRON,PERS):- cursor(LI,CO),LI=21,cursor(LI,CO),!.
pos_curs(TEMPS,CONJFORM,PRON,PERS):- conj_forme("»inusité"),cursor(LI,CO),
    LI2=LI+1, CO2=CO-8,cursor(LI2,CO2),!.
pos_curs(TEMPS,CONJFORM,PRON,0):- espace(Y),
    str_len(PRON,X), str_len(CONJFORM,Z),
    cursor(LI,CO),LI2=LI+1,CO2=CO-(X+Y+Z),
    cursor(LI2,CO2),!.
pos_curs(TEMPS,CONJFORM,PRON,PERS):- TEMPS>2,TEMPS<6,
    str_len(CONJFORM,X),
    cursor(LI,CO),LI2=LI+1,CO2=CO-X,
    cursor(LI2,CO2),!.
pos_curs(TEMPS,CONJFORM,PRON,PERS):- str_len(CONJFORM,X), str_len(PRON,Y),
    cursor(LI,CO),LI2=LI+1,CO2=CO-(X+Y+1),
    cursor(LI2,CO2).

/* sauter une ligne, ou sauter de bas en haut */
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=0, LIN2=LIN, COL2=COL,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=4, TEMPS=4,LIN2=LIN, COL2=COL,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=6, TEMPS=5,LIN2=LIN, COL2=COL,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=7, LIN2=LIN, COL2=COL,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=8, TEMPS=6,LIN2=LIN, COL2=COL,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=14, LIN2=LIN, COL2=COL,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=15, TEMPS=7,LIN2=LIN, COL2=COL,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=21,COL=0,LIN2=0,COL2=27,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN=21,LIN2=0,COL2=53,!.
chk_ligne(TEMPS,LIN,COL,LIN2,COL2):- LIN2=LIN+1,COL=COL2.

```

```

/* ----- SORTIE DE LA FORME CONJUGUEE ----- */

output(TEMPS,NO,_,PRON,">inuité",_,_):- write("      "),!.
output(TEMPS,NO,_,PRON,">inuité",_,_):- write("      "),!.
output(5,NO,invar,PRON,FORME_CONJ,_,AUX):- write(AUX," ",FORME_CONJ," invar"),
      /*cursor(L,C),C2=C-12, cursor(L,C2),*/!.
output(5,NO,e,PRON,FORME_CONJ,ALTERN,AUX), isol_elem(1,ALTERN,INIT,FIN),
      write(AUX," ",FORME_CONJ," ",FIN,"e"),
      /*cursor(L,C),C2=C-10, cursor(L,C2),*/!.
output(TEMPS,NO,_,PRON,FORME_CONJ,_,_):- TEMPS>2,TEMPS<6,
      write(FORME_CONJ),!.
output(TEMPS,NO,_,je,FORME_CONJ,_,_):- chk_je(PRON2,ESPACE),
      write(PRON2,ESPACE,FORME_CONJ),!.
output(TEMPS,NO,_,PRON,FORME_CONJ,_,_):- write(PRON," ",FORME_CONJ).

chk_je("j",""):- chk_rad(RAD),frontstr(1,RAD,INIT,RESTE),INIT=h,
      aspiré(LISTE),not(appartient(RAD,LISTE)),
      asserta(espace(0)),!.
chk_je("j",""):- chk_rad(RAD),frontstr(1,RAD,INIT,RESTE),cons(LISTE),
      not(appartient(INIT,LISTE)),
      asserta(espace(0)),!.
chk_je(je,""):- asserta(espace(1)).

chk_rad(RAD):- rad("ét"),rad2(RAD),!.
chk_rad(RAD):- rad("all"),rad2(RAD),!.
chk_rad(RAD):- rad(RAD).

/*chk_aspect(TEMPS,PRON,FORME_CONJ3):- TEMPS>2,TEMPS<6,write(FORME_CONJ3),!.
chk_aspect(TEMPS,PRON,FORME_CONJ3):- write(PRON," ",FORME_CONJ3).*/

/* ----- LISTES ----- */

cons([b,c,d,f,g,h,j,k,l,m,n,p,q,r,s,t,v,w,x,z]).

aspiré([habl,hach,hachur,hal,hâl,halêt,ha,hanch,handicap,happ,hant,
harangu,harass,harccl,harccl,hard,harnach,harpaill,harp,harponn,hasard,hât,
hauss,hav,haviss,hél,hêl,hériss,hériss,hers,heurt,hiss,hoch,hongroi,hongroy,
honniss,hoquet,hoquett,houblonn,hou,houpp,hourd,hourdis,houspill,huch,hu,
hulul,hum,hurl]).

```

CONCORDANCE

PREDICATES

```

/* Menu selection */
highlight(INTEGER,INTEGER,TOUCHE)
cursor_move(INTEGER,INTEGER,TOUCHE)
bottom_line

remove_spaces(symbol,symbol)
chk_line(integer,integer,integer,integer,integer)
write_column

```

CLAUSES

```

list_verbs:- shiftwindow(1),
              window_attr(112),
              clearwindow,
              write_column.
list_verbs.

write_column:- bottom_line,
               cursor(0,0),
               verb(N,VERB),
               write(VERB),
               str_len(VERB,LEN),
               cursor(LI,CO),
               chk_line(LEN,LI,CO,LIN,COL),
               cursor(LIN,COL),LI=21,CO>60,
               highlight(0,0,TOUCHE), TOUCHE=esc,!,fail.
write_column.

chk_line(LEN,21,CO,21,COL):- CO>60, COL=CO,!.
chk_line(LEN,LI,CO,LIN,COL):- LI<21, LIN=LI+1, COL=CO-LEN,!.
chk_line(LEN,21,CO,0,COL):- COL=CO-LEN+15,!.
chk_line(LEN,LI,CO,LIN,COL):- LI=LIN,CO=COL.

highlight(LI,CO,TOUCHE):- field_attr(LI,CO,15,7),
                           lireT(TOUCHE),trace(on),
                           cursor_move(LI,CO,TOUCHE).

cursor_move(LI,CO,esc):- LI2=LI-LI, CO2=CO-CO,cursor(LI2,CO2),!.
cursor_move(LI,CO,s):- clearwindow, bottom_line.
cursor_move(LI,CO,cr):- field_attr(LI,CO,15,112),
                        field_str(LI,CO,15,VB),
                        remove_spaces(VB,VERB),
                        clearwindow,cursor(22,5),
                        write("      L : liste verbes "),
                        field_attr(22,0,78,7),
                        field_attr(22,15,1,15),
                        cursor(0,0),afficher(VERB),
                        highlight(LI,CO,TOUCHE).

cursor_move(LI,CO,l):- clearwindow, write_column.
cursor_move(0,CO,up):- !, highlight(0,CO,_).
cursor_move(LI,CO,up):- LI>0,!,field_attr(LI,CO,15,112), LI1=LI-1,
                        highlight(LI1,CO,TOUCHE).
cursor_move(21,CO,down):- !, highlight(21,CO,_).

```

LISTINGS DES MODULES

CONCORDANCE

```

cursor_move(LI,CO,down):- LI<21,!, field_attr(LI,CO,15,112), LI1=LI+1,
                           highlight(LI1,CO,TOUCHE).
cursor_move(LI,0,left):- !, highlight(LI,0,_).
cursor_move(LI,CO,left):- CO>1,!, field_attr(LI,CO,15,112), CO1=CO-15,
                           highlight(LI,CO1,TOUCHE).
cursor_move(LI,60,right):- !, highlight(LI,60,_).
cursor_move(LI,CO,right):- CO<60,!, field_attr(LI,CO,15,112), CO1=CO+15,
                           highlight(LI,CO1,TOUCHE).
cursor_move(LI,CO,home):- !, field_attr(LI,CO,15,112), CO1=0, LI1=0,
                           highlight(LI1,CO1,TOUCHE).
cursor_move(LI,CO,end):- !, field_attr(LI,CO,15,112), CO1=60, LI1=21,
                           highlight(LI1,CO1,TOUCHE).
cursor_move(LI,CO,otheresc):- !, highlight(LI,CO,TOUCHE).

remove_spaces(VB,VERB):- str_len(VB,X), Y=X-1,
                           frontstr(Y,VB,FRONT,END),END=" ",
                           remove_spaces(FRONT,VERB),!.
remove_spaces(VB,VERB):- VB = VERB.

bottom_line:- cursor(22,0),
               write("<-:choisir  <-- --> HOME END:déplacer curseur  S:page suivante ESC:sortir"),
               field_attr(22,0,78,7),
               field_attr(22,0,3,15),
               field_attr(22,13,15,15),
               field_attr(22,46,1,15),
               field_attr(22,62,3,15),
               cursor(0,0).

```

ANNEXE

ANNEXE

A. Particularités du programme.

liaison entre deux voyelles

- je + aime s'affiche correctement comme j'aime
je + ai ---> j'ai etc

- h-aspiré

Le programme sait distinguer entre h-aspiré et non-aspiré, et affiche donc respectivement, par exemple, *j'hésite* et *je hâte*.

- impératif

Il n'accepte pas une demande de conjuguer un impératif à la 1ère ou 3ème pers sg, ou 3ème pers pl. De même, en mode INTERRO il n'en fait pas de telles demandes de l'utilisateur (il n'attendra pas l'impératif de la 3ème personne, par exemple)

- inusité

Si une forme est inusitée ou n'existe pas ou n'est pas réalisée, le programme en avertira l'utilisateur en laissant un blanc. De même, en mode INTERRO il n'en demandera pas une telle conjugaison il ne posera pas la question, "Donnez la première personne de l'imparfait du *pleuvoir*?"

- longueur du verbe

En mode CONJUGAISON COMPLETE, afin que les trois colonnes ne dépassent pas les limites de l'écran, le programme rejettera un infinitif plus long que 14 caractères (il n'acceptera pas *radiotélégraphier* ni *transsubstantier*, par exemple)

- auxiliaire

le participe passé s'affiche à l'écran précédé par *ayant* ou *étant*, selon le cas - *étant tombé*, *ayant vu*, *étant allé* etc

- alternatifs

là où il existe une forme alternative le programme en avertira l'utilisateur. La première personne de l'indicatif présent du pouvoir peut être soit *je peux*, soit *je puis*. Le programme donne tous les deux.

B. Quelques traits additionnels non-inclus.

- des formes impossibles ou improbables auraient pu être exclues, par exemple "nous brayons" ou "vous aboyez". Cela aurait exigé des connaissances sémantiques, ce que n'est pas le cas avec le présent programme qui conjugue uniquement selon des règles purement formelles.
- des renseignements à propos de l'utilisation du verbe (renseignements syntaxiques), par exemple:
 - "essayer + de"
 - nombre d'arguments
 - emploi transitif ou intransitif
- les formes composées: j'ai servi, que j'aie servi, j'aurais servi etc
- verbe à employe impersonnel: il pleut, il neige - éviter les formes comme "je neige" (bien qu'il soit concevable dans certains contextes!)
- les formes pronominales: "je me méfie" etc (incluses dans une version antérieure du program qui ne traitait que du subjonctif présent)
- "infos" fenêtre - censée fournir des renseignements additionnels, par exemple pour les verbes défectifs et unipersonnels.

C. Définitions

1. Numérotation

1.1 Temps

No.	code		temps
0	IP	=	indicatif présent
1	SP	=	subjonctif présent
2	II	=	indicatif imparfait
3	IM	=	impératif
4	PR	=	participe présent
5	PA	=	participe passé
6	FS	=	futur simple
7	CD	=	conditionnel
8	PS	=	indicatif passé simple
9	SI	=	subjonctif imparfait

1.2 Personne

No.	personne
0	1 pers. sing.
1	2 pers. sing.
2	3 pers. sing.
3	3 pers. pl.
4	1 pers. pl.
5	2 pers. pl.

2. Symboles

$\left\{ \begin{matrix} x \\ x \\ x \\ x \end{matrix} \right\}$ ou $\{x, x, x, x, \dots\}$ = alternatifs (SOIT x OU x OU $x, \dots$)

{LISTE n } = renvoi à une liste d'alternatifs trop longue à insérer facilement dans une règle

$\left[\begin{matrix} x \\ x \\ x \\ x \end{matrix} \right]$ = $(x \text{ ET } x \text{ ET } x \dots)$

C_n = au minimum n consonnes, au maximum n consonnes

$a \rightarrow b$ = a se réécrit en b

$y / x _ _ z$ = y dans l'environnement $x \dots z$

$+$ = trait à inclure

$-$ = trait à exclure

$\emptyset$ = zéro ou caractère supprimé

$\#$ = frontière de morphème

$\#\#$ = frontière de mot

$\langle \rangle$ = "métatrait" - signifie que le trait n'existe pas réellement à l'étape du processus. Par exemple $\langle [+er] \rangle$ signifie que l'on veut inclure les verbes de la 1^{re} conjugaison, $\langle [-ir] \rangle$ que l'on veut exclure tous les verbes en *ir*, bien que ces marques de l'infinitif soient déjà enlevées au début du traitement et donc "n'existe plus" (ils sont néanmoins sauvegardés et on peut y référer si besoin est).

D. Les règles principales

Règle générale:

INFINITIF + ASPECT + PERSONNE $\longrightarrow$ [RADICAL+ASPECT] + VOYELLE + [PERSONNE+ASPECT] $\longrightarrow$ FORME CONJUGUÉE

Règles spécifiques:

Les particularités de la première conjugaison

LISTE1(modeler, celer, déceler, receler, ciseler, démanteler, écarteler, se_encasteler, geler, dégeler, congeler, surgeler, marteler, peler, acheter, racheter, bégueter, corseter, crocheter, fileter, fureter, halet)).

1.
$$\begin{matrix} c \\ g \end{matrix} \longrightarrow \begin{matrix} g \\ ge \end{matrix} / ____ \#[\{a, o, \hat{a}\}]$$
2.
$$\begin{matrix} t \\ l \end{matrix} \longrightarrow \begin{matrix} tt \\ ll \end{matrix} / [-\langle \text{LISTE1} \rangle] ____ \#[+e_muet]$$
3.
$$e \longrightarrow \hat{e} / [\{l, t, c, m, n, p, r, s, v, vr\}] ____ \#[+e_muet]$$
4.
$$\hat{e} \longrightarrow \hat{e} / [+C\hat{e}] ____ \# \left[\begin{matrix} [+TEMPS > 5] \\ [+e_muet] \end{matrix} \right]$$
5.
$$oy \longrightarrow er / env ____ \# \left[\begin{matrix} + TEMPS > 5 \\ + TEMPS < 8 \end{matrix} \right]$$
6.
$$y \longrightarrow i / [- ey] ____ \#[+ e_muet]$$
7.
$$V \longrightarrow e / [- envoy] ____ \# \left[\begin{matrix} + TEMPS > 5 \\ + TEMPS < 8 \end{matrix} \right]$$
8.
$$V \longrightarrow a / ____ \#[+ TEMPS > 7]$$
9.
$$e_muet \longrightarrow \left[\begin{matrix} [+ PERS < 4] \\ [+TEMPS = \{0, 1, 3, 6, 7\}] \end{matrix} \right]$$
10.
$$\hat{i} \longrightarrow i / ha ____ \left\{ \begin{matrix} [+ TEMPS = 0] \\ [+ TEMPS = 3] \end{matrix} \right\} [+ PERS < 3]$$
11.
$$\emptyset \longrightarrow ss / \{i, \hat{i}\} ____ [+TEMPS < 5] - \left[\begin{matrix} [PERS < 3] \\ \left\{ \begin{matrix} [TEMPS = 0] \\ [TEMPS = 3] \end{matrix} \right\} \end{matrix} \right]$$

Voyelles thématiques au PS et SI

12. V ----> a/___«er» [+ TEMPS > 7]

13. V ----> i / $\left[\begin{array}{l} \text{-(taire, savoir, devoir, être, pouvoir, pourvoir)} \\ \text{+(ill, vr, fr, pr, mp, gn, qu, nc, m, v, s, f, d, t)} \end{array} \right] \text{--- +TEMPS>7}$

14. V ----> Ø/{in, i, u, i}___ + TEMPS>7

15. V ----> u/ + TEMPS>7

Morphophonologie

Première passe

16. {v, s, l, t, m} ----> Ø/{-və, gi}___# {s, x, t}

17. d ----> Ø/in___# {s, x, t}

Deuxième passe

18. t ----> Ø/___(d, c, t)#

19. is ----> i/##cro_#(s, t)

20. s ----> ^/i___#t

21. i ----> î/pla___#t

22. o ----> ô/##cl___#t

23. s ----> Ø/___#(s, t)

E. Prédicats - dans l'ordre alphabétique

afficher	eler_eter	part_passé2b
afficher2	espace	pers
ajustfenêtre	être	personne
aléa	exclure	pos_curs
analyse	exec	auxiliaire
appartenance	exec1	prn
appartient	exec2	pron
tmps	exec3	rad
aspct	exec4	rad2
temps	exec5	rad3
aspiré	exec6	re_modif1
aux	flag	re_modif2
begin	génération	rad_modif
bottom_line	go	rad_modif_imptf
branch	highlight	rad_modif_ind_pres
bravo	ind_menu	interro
calc_pos	invariable	interro_loop
car	irreg	interro_menu
change	irreg_A	interro_menu2
change2	irreg_B	remove_spaces
change_rad	irreg_C	repérer
change_rad_er	irreg_D	saisir
change_termin	irreg_E	select
chk_5	irreg_F	sentir_etc
chk_aspect	ir_modif	adoucir
chk_cas_spéc	isol_elem	couper
chk_conj_forme	lireT	subj_menu
chk_flag	lireT1	submenu
chk_ind_pres	lireT2	table1
chk_je	listlen	tallycorrect
chk_ligne	list_verbs	tallywrong
chk_line	long	inf
chk_pron	noyau	temps
chk_rad	mainmenu	termin
chk_resp	maxlen	termin2
choisir	menu1	termin3
choisir_verbe	menu2	terminaison
choisir_verbe2	message	terminaison2
choose	morphoph	testconjug
classifier	déplacer_chapeau	test_term
clear	mute_1	tmps
conjug_compl	mute_2	type_finir
conjug_compl_loop	mymenu	type_finir_modif
conj_forme	repeat	v
cons	curieux	v2
cons_douce	ôter_espace	variantes
cons_finale	output	vb
cons_finale2	particularité	verb
cons_finale3	particularité	verbe_défectif
count	particularité5	window6
cursor_move	partic_menu	windows
delay	part_passé1	writelist
double_irreg	part_passé2a	write_column

LES 478 VERBS DANS VERBS.DBA

abaisser	calculer	différer	hâter	pénétrer	résoudre
abattre	camper	dîner	hausser	percer	ressentir
abîmer	céder	dire	hériter	perdre	rester
aboutir	célébrer	diriger	hésiter	périr	rétablir
aboyer	changer	discuter	heurter	permettre	retenir
abriter	charger	disparaître	indiquer	peser	réunir
accepter	chasser	disposer	inonder	pincer	réussir
accompagner	chausser	distinguer	inquiéter	placer	révéler
accomplir	choisir	distraire	instruire	plaindre	revenir
accourir	citer	dormir	interdire	plaire	rêver
accueillir	clocher	douter	intéresser	pleurer	revoir
accuser	combattre	échoir	interrompre	pleuvoir	rire
acheter	combiner	éclaircir	intervenir	plonger	rompre
achever	commencer	éclater	introduire	polir	ronger
acquiescer	commettre	écouter	inviter	pondre	rougir
admettre	communiquer	écrire	irriter	posséder	saillir
adopter	compléter	effacer	isoler	pourrir	saisir
adoucir	comprendre	efforcer	jaillir	poursuivre	saler
aérer	concevoir	effrayer	jeter	pouvoir	salir
affaiblir	conclure	élancer	joindre	précéder	satisfaire
affliger	conduire	élargir	jouer	préférer	sauter
agacer	confesser	élever	jurer	prendre	savoir
agenouiller	confier	élire	jurer	prétendre	sécher
agir	confondre	éloigner	lâcher	prêter	secourir
agiter	connaître	embrasser	laisser	prévenir	semer
aimer	conquérir	émouvoir	lancer	prévoir	sentir
ajouter	consentir	emparer	lever	produire	séparer
aller	conserver	empirer	lier	profiter	serrer
allonger	considérer	enfermer	lire	projeter	servir
amener	consoler	engager	loger	prolonger	soigner
annoncer	construire	englober	longer	promettre	songer
apercevoir	contenir	enlever	lui	promouvoir	sortir
aplatir	conter	ennuyer	lutter	prononcer	souffrir
apparaître	contraindre	enrichir	mâcher	protéger	souiller
appartenir	contribuer	enseigner	maigrir	provenir	soulager
appeler	convaincre	entendre	maintenir	punir	soulever
applaudir	convenir	entreprendre	manger	quérir	soumettre
appliquer	corriger	entretenir	manifester	quitter	sourire
apprécier	coudre	entrevoir	maudire	raccourcir	soustraire
apprendre	couler	envahir	mêler	raffaîchir	soutenir
argenter	couper	envoyer	ménager	ranger	souvenir
armer	courir	épanouir	mentir	rappeler	subir
arranger	couster	espérer	mêler	ravager	succéder
arrêter	couvrir	établir	mettre	ravir	sucer
arrondir	craindre	éteindre	modéliser	recevoir	suffire
arroser	créer	étendre	modérer	reciter	suggérer
assaillir	crêper	être	mordre	reconduire	sulver
assembler	crier	évanouir	mouiller	reconnaître	surprendre
asseoir	croire	évanouir	mourir	recouvrir	suspendre
associer	croître	évanouir	mouvoir	recueillir	taire
assurer	cueillir	évanouir	munir	recupérer	taper
attaquer	cuire	évanouir	nager	redevenir	tâter
atteindre	daigner	évanouir	naître	redire	tendre
atteler	débâcher	évanouir	négliger	réduire	tenir
attendre	débattre	évanouir	nettoyer	refaire	tendre
attraper	décharger	évanouir	noircir	réfléchir	tordre
attribuer	découvrir	évanouir	noter	refroidir	tousser
attrister	décrire	évanouir	nourrir	régler	tracer
autoriser	défaire	évanouir	nuire	régner	traduire
avaler	défendre	évanouir	obéir	regretter	trahir
avancer	dégager	évanouir	obliger	rejeter	traiter
avertir	dégouter	évanouir	obscurcir	rejoindre	transporter
avoir	déguiser	évanouir	observer	réjouir	unir
avouer	déménager	évanouir	obtenir	relever	vaincre
baigner	démolir	évanouir	offrir	relier	valoir
baïiller	dépêcher	évanouir	opérer	remettre	veiller
balayer	dépendre	évanouir	ôter	remplacer	vendre
battre	déplacer	évanouir	ouvrir	remplir	venger
bénir	déplaire	évanouir	pâler	rendre	venir
bercer	déranger	évanouir	paraître	renouveler	verdir
blanchir	descendre	évanouir	parcourir	renseigner	vernir
boire	désespérer	évanouir	parler	rentrer	vêtir
bondir	désobeir	évanouir	partager	renvoyer	vieillir
bouger	désoler	évanouir	partir	réparer	visiter
bouillir	déterminer	évanouir	parvenir	reparaître	vivre
briller	détruire	évanouir	payer	repartir	voir
briser	devenir	évanouir	pêcher	repêcher	vouloir
broder	deviner	évanouir	peindre	répondre	zigzaguer
brûter	devoir	évanouir	pendre	reprandre	
brûler	dévouer	évanouir			

VERBES DE GROUPE 2 ET 3 SEPARES PAR L'ORDINATEUR SELON SCHEMA 1

abasourdir 2	attendrir 2	désenlaidir 2	fléchir 2	pervertir 2	rétablir 2	tenir 3
abâtardir 2	atterrir 2	désépaissir 2	flétrir 2	pétrir 2	retentir 2	venir 3
abêtir 2	attiédir 2	désétablir 2	fleurir 2	polir 2	rétrécir 2	acquérir 3
abolir 2	avachir 2	désinvestir 2	forcer 2	pourrir 2	rétroagir 2	sentir 3
abonner 2	avertir 2	désobéir 2	fouir 2	préétablir 2	réunir 2	partir 3
aboutir 2	aveullir 2	dessaisir 2	fourbir 2	prémunir 2	réussir 2	mentir 3
abrutir 2	avilir 2	dessertir 2	fournir 2	punir 2	reverdir 2	repentir 3
accomplir 2	bannir 2	désunir 2	fraîchir 2	rabonner 2	roidir 2	ressortir 3
accourir 2	barrir 2	déverdir 2	franchir 2	rabougir 2	rondir 2	vêtir 3
accroupir 2	bâtir 2	dévernir 2	frémir 2	raccourcir 2	rosir 2	couvrir 3
adoucir 2	bénir 2	divertir 2	froidir 2	racornir 2	rôtir 2	souffrir 3
affadir 2	blanchir 2	doucir 2	garantir 2	radoucir 2	rougir 2	offrir 3
affaiblir 2	blêmir 2	durcir 2	garnir 2	raffermir 2	rouir 2	cueillir 3
affermer 2	blettir 2	ébahir 2	gauchir 2	rafraîchir 2	roussir 2	assaillir 3
affranchir 2	bleuir 2	ébaubir 2	gémir 2	ragaillardir 2	roustir 2	faillir 3
agir 2	blondir 2	ébaudir 2	glapir 2	raidir 2	rugir 2	bouillir 3
agonir 2	blottir 2	éblouir 2	glatir 2	rajeunir 2	saillir 3	dormir 3
agrandir 2	bondir 2	écatir 2	grandir 2	ralentir 2	saisir 2	courir 3
aguerrir 2	bouffir 2	échampir 2	gravir 2	ramollir 2	salir 2	mourir 3
ahurir 2	brandir 2	éclaircir 2	grossir 2	rancir 2	saurir 2	servir 3
aigrir 2	bruir 2	écrouir 2	guérir 2	raplatir 2	serfouir 2	fuir 3
alanguir 2	brunir 2	effleurir 2	haver 2	rapointir 2	sertir 2	gésir 3
alentir 2	calair 2	élargir 2	hennir 2	ravilir 2	sortir 3	
alléger 2	candir 2	embellir 2	honnir 2	ravir 2	subir 2	
alourdir 2	chancier 2	emboutir 2	hourdir 2	réagir 2	subvertir 2	
alunir 2	chauvir 2	embrunir 2	impartir 2	réassortir 2	superfinir 2	
amaigrir 2	chérir 2	emplir 2	infléchir 2	rassortir 2	surenchérir 2	
amater 2	choisir 2	empuantir 2	intervertir 2	rebâtir 2	surgir 2	
amerrir 2	clapir 2	enchérir 2	invertir 2	reblanchir 2	surir 2	
ameubler 2	compatir 2	endolorir 2	jaillir 2	rebondir 2	tapis 2	
amincir 2	côir 2	endurcir 2	jaunir 2	rechaupir 2	tarir 2	
amoindir 2	convertir 2	enforcer 2	jouir 2	réchaupir 2	tartir 2	
amollir 2	cotir 2	enfouir 2	languir 2	reconvertir 2	ternir 2	
amortir 2	crépier 2	engloutir 2	lotir 2	recrépier 2	terrir 2	
amuir	croupir 2	engourdir 2	louchir 2	redémolir 2	tiédir 2	
anéantir 2	débâtir 2	enhardir 2	maigrir 2	réfêchir 2	trahir 2	
anoblir 2	débleuir 2	enlaidir 2	matir 2	refleurir 2	transir 2	
anordir 2	décatir 2	ennoblir 2	mégir 2	refroidir 2	travestir 2	
aplanir 2	décrépier 2	enorgueillir 2	meurtrir 2	regarnir 2	unir 2	
aplatir 2	définir 2	enrichir 2	mincir 2	régir 2	vagir 2	
appauvrir 2	défleurir 2	ensevelir 2	moisir 2	regrossir 2	verdir 2	
appesantir 2	défraîchir 2	envahir 2	moitir 2	réinvestir 2	vernir 2	
applaudir 2	dégarnir 2	envieillir 2	mollir 2	rejaillir 2	vieillir 2	
appointir 2	dégauchir 2	épaissir 2	mugir 2	réjouir 2	vioquir 2	
approfondir 2	déglutir 2	épanouir 2	munir 2	rélargir 2	vomir 2	
arrondir 2	dégourdir 2	époutir 2	mûrir 2	rebrunir 2	vrombir 2	
assagir 2	dégrossir 2	équarrir 2	nantir 2	reaplr 2		
assainir 2	déguerpir 2	esbaudir 2	noircir 2	renchérir 2	maudire	
asservir 2	déjaunir 2	estourbir 2	nordir 2	répartir 2		
assoébrir 2	démolir 2	établir 2	nourrir 2	repolir 2		
assortir 2	démunir 2	étourdir 2	obéir 2	resaler 2		
assoupir 2	dépérir 2	étrécir 2	obscurir 2	resplendir 2		
assouplir 2	dépolir 2	évanouir 2	ourdir 2	ressaisir 2		
assourdir 2	déraider 2	faiblir 2	pâir 2	ressortir 3		
assourvir 2	désassortir 2	farcir 2	pâtir 2	resurgir 2		
assujettir 2	désaplr 2	finir 2	périr 2	ressurgir 2		

Exemple de la sortie d'un programme de test. Les verbes sont ceux dans VERBS.DBA
On repère facilement les erreurs.

INDICATIF PRESENT 3 sg

assaille	agenouille	arrose	berce	combine
abaisse	agit	assemble	blanchit	commence
abat	agite	assied	boit	commet
abîme	aime	associe	bondit	communique
aboutit	ajoute	assure	bouge	complète
aboie	va	attaque	bout	comprend
abrite	allonge	atteint	brille	conçoit
accepte	amène	attelle	brise	conclut
accompagne	annonce	attend	brode	conduit
accomplit	aperçoit	attrape	broute	confesse
accourt	aplatit	attribue	brûle	confie
accueille	apparaît	attriste	calcule	confond
accuse	appartient	autorise	campe	connaît
achète	appelle	avale	cède	conquiert
achève	aplaudît	avance	célèbre	consent
acquiert	applique	avertît	change	conserve
admet	apprécie	a	charge	considère
adopte	apprend	avoue	chasse	console
adoucit	argente	baigne	chausse	construit
aère	arme	bâille	choisit	contient
affaiblit	arrange	balaie	cite	conte
afflige	arrête	bat	cloche	contraint
agace	arrondit	bénit	combat	contribue
convainc	défait	dîne	embrasse	évanouit
convient	défend	dit	émeut	fâche
corrige	dégage	dirige	empare	fait
coud	dégoûte	discute	endort	faut
coule	déguise	disparaît	enfuit	fatigue
coupe	déménage	dispose	engage	félicite
court	démolît	distingue	engloutit	finît
coûte	dépêche	distrain	enlève	fonce
couvre	dépend	dort	ennuie	fond
craint	déplace	doute	enrichit	force
crée	déplaît	écht	enseigne	fournit
crève	dérange	éclaircit	entend	franchit
crie	descend	éclate	entreprend	frémît
croit	désespère	écoute	entretient	fuit
croît	désobeît	écrit	entrevoit	gagne
cueille	désole	efface	envahit	garantit
cuit	détermine	eforce	envoie	garnit
daigne	détruit	effraie	épanouit	gèle
débarque	devient	élance	espère	gémît
débat	devine	élargit	éteint	gêne
décharge	doit	élève	étend	gât
découvre	dévoque	élit	est	goûte
décrit	diffère	éloigne	établit	grandit

gratte	introduit	mange	obéit	perd
grogne	invite	manifeste	oblige	pérît
grossît	irrite	maudît	obscurcit	permet
guérît	isole	mêle	observe	pèse
habille	jaillît	ménage	obtient	pince
habite	jette	ment	offre	place
habitué	joint	mérite	opère	plaint
hait	jouît	met	ôte	plaît
hache	juge	modèle	ouvre	pleure
harcelle	jure	modère	pâlît	pleuvoit
hâte	lâche	mord	paraît	plonge
hausse	laisse	mouille	parcourt	polît
hérîte	lance	meurt	parle	pond
hésite	lève	meut	partage	possède
heurte	lie	munît	part	pourrît
indique	lit	nage	parvient	poursuit
inonde	loge	naît	paie	peut
inquiète	longe	néglige	pêche	précède
instruit	luit	nettoie	peigne	préfère
interdit	lutte	noircît	peint	prend
intéresse	mâche	note	pend	prétend
interrompt	maigrît	nourrît	pénètre	prête
intervient	maintient	nuit	perce	prévient
prévoit	recouvre	rentre	rougît	sourit
produit	recueille	renvoie	saille	soustrait
profite	recupère	répand	saisît	soutient
projette	redevient	reparaît	sale	souvient
prolonge	redit	repart	salît	subît
promet	réduit	repent	satisfait	succède
promeut	refait	répète	saute	suce
prononce	réfléchît	répond	sait	suffit
protège	refroidît	reprend	sèche	suggère
provient	règle	résout	secourt	suit
punît	règne	ressent	sème	surprend
quiert	regrette	reste	sent	suspend
quitte	rejette	rétablît	sépare	tait
raccourcit	rejoint	retient	serre	tape
rafraîchît	réjouît	réunit	sert	tâte
range	relève	réussît	soigne	tend
rappelle	relit	révèle	songe	tient
ravage	remet	revient	sort	tond
ravît	remplace	rêve	souffre	tord
reçoit	remplît	revoit	souille	tousse
récite	rend	rit	soulage	trace
reconduit	renouvelle	rompt	soulève	traduit
reconnaît	renseigne	ronge	soumet	trahît
traite	venge	vit		
transporte	vient	voit		
unît	verdît	veut		
vainc	vernît	zigzague		
vaut	vêt			
veille	vieillît			
vend	visite			

BIBLIOGRAPHIE

OUVRAGES GENERAUX

- ANTWORTH Evan - 1990 PC-KIMMO: A Two-level Processor for Morphological Analysis, Dallas, Texas, Summer Institute of Linguistics (Occasional Publications in Academic Computing Nr. 16), 273 p.
- BESCHERELLE - 1990 (nouvelle éd.), La Conjugaison - 12000 verbes Paris, Librairie Hatier, 175 p.
- CAPUT J. et J.-P. - 1981 (nouvelle éd.), Dictionnaire des verbes français, Paris, Librairie Larousse, XVI + 589 p.
- CHOMSKY Noam - 1985 (14^e éd.) Syntactic Structures, The Hague, Mouton, (Janua Linguarum Series Minor - Nr. 4), 117 p.
- GERTNER Michael - 1973, The Morphology of the Modern French Verb, The Hague, Mouton, (Janua Linguarum, Series Practica, 204), 149 p.
- GREVISSE Maurice - 1986 (12^e éd. refondue par André Goosse, 2^e tirage 1988), Le bon usage, grammaire française, Paris-Gembloux, Duculot, XXXVIII + 1768 p.
- ISAAC Luc - 1985, Calcul de la flexion verbale en français contemporain, Genève, Librairie Droz, XVII + 499 p.
- KAYSER Daniel - 1985, Des machines qui comprennent notre langue, La Recherche, No 170, vol 16, pp 1198-1212.
- LAROUSSE - 1979 (nouvelle éd. 1989), Dictionnaire de la langue française - lexis, Paris, Larousse, XVI + 2109 p.
- RADFORD Andrew - 1981, Transformational Grammar - A first course Cambridge, CUP, 625 p.
- SCHANE S. A. - 1968, French Phonology and Morphology, Cambridge Mass, M.I.T. Press.

LA PROGRAMMATION EN PROLOG

- CONDILLAC M. - 1986, Prolog: fondements et applications, Paris, Dunod, p.
- DELAHAYE J.-P. - 1988, Cours de PROLOG avec TURBO PROLOG - éléments fondamentaux, Paris, Eyrolles, XX + 338 p.

INDEX GENERAL

Conjugué, le programme	24	imperatif	41,151
2ème conjugaison	89	impersonnel	14,17
algorithme	15	indicatif	41
algorithme, confection de	31	indicatif futur	41
algorithmes	85	indicatif imparfait	41
appartient	89	indicatif passé simple	41
auxiliaire	151	indicatif présent	41
base de données	23,35	infinitif	14,34
boîte à outils	85,96,98,100	infinitif, point de départ	34
catégorisation, problème de	38	infos fenêtre	152
code à deux lettres	44,153	interface en langue naturelle	23
CONCORDANCE, module	24	INTERRO, module	24
conditionnel	42	langage formel	31,33,154
CONJUGAISON COMPLETE, module	24	liaison	28,151
connaissances linguistiques	28	liste de verbes	24
connaissances sémantiques	152	listes d'exceptions	89
corpus	16	locuteurs non francophones	32
défectif	15,17	Macintosh	8,9
définitions	153	matrice	79
description traditionnelle	50	mémoire de l'ordinateur	85
désinence	51	MENUS, module	24
diagnostics	98	mode	14,41
didacticiel	6,27	mode CONJUGAISON COMPLETE	151
didactique	32	mode INTERRO	151
DOS, message d'erreur	79	modes	112
e-muet	53	niveau de langue	28
échec	89	nombre d'arguments	152
éléments minimaux	87	non-phonétique, aspect	41
emploi intransitif	152	NOYAU	85
emploi transitif	152	NOYAU, module	24
exceptions	89	numérotation	153
exceptions, problème de	37	numérotation des temps	44
exclusion des éléments	89	ordinateur, conjugaison par	31
fenêtre ANALYSE	27	ordinateur,	
formalisation	21,41	l'emploi dans l'analyse	59,116
forme inusitée	151	orthographe	28,30,47
formes composées	152	participe	41
formes impossibles	79,152	participe passé	41
formes pronominales	152	participe passé invariable	152
français écrit	30	participe passé variable	152
français parlé	30	participe présent	41
futur	41,42	participes	34
génération	7	particularités des radicaux	52
GENERATION, module	24	passé simple	14,41,42
global, traitement	29	pédagogique	32
grec	7	personne	153
h-aspiré	151	phonologie	31
hébreu	7	physique	32
IBM PC	8	predicats	85
imparfait	41	predicats prédéfinis	85
Caput	81	présent	41
French Assistant	8	Prolog	20,35,80
Grevisse	81	racine	38
Hugo	8,9	racine, définition	51
Sans Faute	9	radical	51

GAL A., Lapalme G., Saint-Dizier P. 1989, PROLOG pour l'analyse automatique du langage naturel, Paris, Eyrolles, p .

PARIZOT E. et BIKAN P. - 1987, Exercices de Turbo-Prolog, Paris, Eyrolles, p .

ROUSSEAU M. - 1986-7, Série d'articles parue dans Micro-Système d'octobre 1986 à avril 1987.

SAINT-DIZIER P. -

TOWNSEND Carl - 1988, Turbo Prolog - Guide du programmeur, Sybex, Paris,

LA PORTE J. et DELPORT D. - 1987, TURBO PROLOG - construisez des applications, Paris, Eyrolles,

radical, définition	51	symboles	154
reconnaissance	7	système expert	23
récurtivité	88	tableaux de verbes	36
redondance	36	temps	14, 41, 112, 153
règle de réécriture	39	temps composés	34
règle générale	89	terminaison	38
règles	21, 85, 155	unification	87
règles formelles	32	unipersonnel	17
règles morphophonologiques	51	verbe à employe impersonnel	152
règles, système de	30	verbes archaïques	30
répétition	36	verbes défectifs	30, 38, 83, 154
sous-jacente	28	verbes impersonnels	83
sous-jacentes, séquences	30	verbes irréguliers	37
subjonctif	14, 41	verbes réguliers	37
subjonctif imparfait	14, 41, 42	verbes unipersonnels	38, 154
subjonctif présent	41	voyelle thématique	43, 52, 72
succès	89		

INDEX DES PREDICATS

adoucir	125, 131	noyau	126
affecter_voy_thém	22, 126, 130	part_passé1	137
analyse	11, 136	part_passé2a	127, 137
appartenance	22, 88, 125, 127, 130	part_passé2b	127, 137
appartient	125, 132, 133, 134	prennent_être	137
calc_pos	127, 128, 134	programme	27
car	139	pron	138
change_termin	125, 126, 131	rad	22
chk_5	125, 126	rad_modif	125
chk_ind_pres	125, 126	rad_modif_ind_prés	128
classifier	135	repérer	125, 127, 128, 129, 134
concat, prédicat prédéfini	88	residu	125, 133, 134
cons_douce	131, 139	re_modif1	127, 137
curieux	139	re_modif2	127, 137
déplacer_chapeau	125, 126, 130	select	125, 129
double_irreg	137	sentir_etc	129, 130, 137
eler_eter	137	suppr1	125, 133
exclure	22, 90, 127, 128, 130, 133	suppr2	125, 133
groupe_1	125, 132	suppr3	125, 133, 134
groupe_3	125, 133	supprimer	125, 133, 133
irreg	128, 137	table1	138
irreg_A	129, 137	temps	125, 126
irreg_B	129, 137	terminaison	126, 138
irreg_C	129, 137	terminaison2	126
irreg_D	129, 137	type_finir	129, 137
irreg_E	129, 137	type_finir_modif	125, 129
irreg_F	129, 137	variantes	139
ir_modif	125, 127, 129	verbe_défectif	125, 126, 127
isol_elem	130, 131, 132, 132, 133	rad_modif	126, 127, 128
morphoph	125, 126, 131	rad_modif_imptf	125, 128, 129
mutés_1	129, 132, 139	rad_modif_ind_prés	125
mutés_2	129, 139		

